



## SIRC-1 CPU Reference Manual

Featuring the SIRCIS Instruction Set

### Technical Specifications

|                          |                          |
|--------------------------|--------------------------|
| <b>Architecture:</b>     | 16-bit RISC Load/Store   |
| <b>Address Bus:</b>      | 24-bit                   |
| <b>Data Bus:</b>         | 16-bit                   |
| <b>Instruction Size:</b> | 32-bit (fixed)           |
| <b>Registers:</b>        | 16 x 16-bit registers    |
| <b>Execution:</b>        | 6-stage execution phases |

*Version 1.0*

*Copyright Silicon Integrated Research Corporation (SIRC) 1989*

June 20, 2026

## About This Manual

This reference manual provides comprehensive documentation for the SIRC-1 CPU architecture and the SIRCIS (SIRC Instruction Set) instruction set. It is intended for:

- Assembly language programmers
- Compiler and toolchain developers
- Hardware designers and implementers
- System software developers

## Document Conventions

Throughout this manual, the following conventions are used:

- Register names appear in typewriter font: `r1`, `ph`, `sr`
- Hexadecimal values are prefixed with `0x`: `0x1A`
- Immediate values are prefixed with `#`: `#123`
- Instruction mnemonics appear in bold typewriter font: **ADDI**
- Addressing modes use parentheses for indirection: `(#4, a)`

# Contents

|          |                                    |          |
|----------|------------------------------------|----------|
| <b>I</b> | <b>Architecture Overview</b>       | <b>1</b> |
| <b>1</b> | <b>Introduction</b>                | <b>3</b> |
| 1.1      | Overview                           | 3        |
| 1.2      | Design Philosophy                  | 3        |
| 1.2.1    | RISC Principles                    | 3        |
| 1.2.2    | Design Goals                       | 3        |
| 1.2.3    | Applications                       | 4        |
| 1.3      | Key Features                       | 4        |
| 1.3.1    | Architecture Characteristics       | 4        |
| 1.3.2    | Execution Model                    | 4        |
| 1.3.3    | Coprocessors                       | 5        |
| 1.3.4    | Privilege Levels                   | 5        |
| 1.4      | Memory Organization                | 6        |
| 1.4.1    | Address Space                      | 6        |
| 1.4.2    | Memory Map                         | 6        |
| 1.4.3    | Alignment                          | 6        |
| 1.5      | Document Organization              | 6        |
| <b>2</b> | <b>CPU Architecture</b>            | <b>9</b> |
| 2.1      | Overview                           | 9        |
| 2.2      | Six-Stage Execution Phases         | 9        |
| 2.3      | Architectural Components           | 10       |
| 2.3.1    | Control Unit (CU)                  | 11       |
| 2.3.2    | Arithmetic Logic Unit (ALU)        | 11       |
| 2.3.3    | Addressing Unit (AU)               | 11       |
| 2.3.4    | Register File                      | 12       |
| 2.3.5    | Barrel Shifters                    | 12       |
| 2.4      | Design Characteristics             | 12       |
| 2.4.1    | Combinatorial Logic Implementation | 12       |
| 2.4.2    | Fixed Execution Sequence           | 12       |
| 2.4.3    | Parallel Functional Units          | 12       |
| 2.5      | External Interface                 | 13       |
| 2.5.1    | Chip Layout                        | 13       |
| 2.5.2    | Pin Descriptions                   | 13       |
| 2.5.3    | Timing Notes                       | 19       |

|          |                                                    |           |
|----------|----------------------------------------------------|-----------|
| <b>3</b> | <b>Register Model</b>                              | <b>21</b> |
| 3.1      | Overview                                           | 21        |
| 3.2      | General Purpose Registers                          | 21        |
| 3.2.1    | Register Set                                       | 21        |
| 3.2.2    | Usage Conventions                                  | 22        |
| 3.3      | Address Register Pairs                             | 22        |
| 3.3.1    | Overview                                           | 22        |
| 3.3.2    | Link Register (l)                                  | 22        |
| 3.3.3    | Address Register (a)                               | 23        |
| 3.3.4    | Stack Pointer (s)                                  | 23        |
| 3.3.5    | Program Counter (p)                                | 23        |
| 3.4      | Status Register                                    | 24        |
| 3.5      | Privilege Restrictions                             | 24        |
| 3.5.1    | High Address Register Restrictions                 | 24        |
| 3.6      | Register Addressing                                | 24        |
| 3.7      | Usage Examples                                     | 25        |
| 3.7.1    | Basic Register Operations                          | 25        |
| 3.7.2    | Address Register Usage                             | 25        |
| 3.7.3    | Subroutine Call                                    | 26        |
| <b>4</b> | <b>Status Register</b>                             | <b>27</b> |
| 4.1      | Overview                                           | 27        |
| 4.2      | Status Register Layout                             | 27        |
| 4.3      | Condition Flags (Non-Privileged)                   | 27        |
| 4.3.1    | Zero Flag (Z) – Bit 0                              | 27        |
| 4.3.2    | Negative Flag (N) – Bit 1                          | 28        |
| 4.3.3    | Carry Flag (C) – Bit 2                             | 28        |
| 4.3.4    | Overflow Flag (V) – Bit 3                          | 28        |
| 4.3.5    | Reserved Flags – Bits 4–7                          | 28        |
| 4.4      | Control Flags (Privileged)                         | 28        |
| 4.4.1    | Protected Mode (P) – Bit 8                         | 28        |
| 4.4.2    | Hardware Interrupt Enable Bits (E1-E4) – Bits 9–12 | 29        |
| 4.4.3    | Exception Active (EA) – Bit 13                     | 29        |
| 4.4.4    | Trap on Address Overflow (A) – Bit 14              | 30        |
| 4.4.5    | Trace Mode (T) – Bit 15                            | 30        |
| 4.5      | Status Register Updates                            | 30        |
| 4.5.1    | Automatic Updates                                  | 30        |
| 4.5.2    | Manual Updates                                     | 30        |
| 4.6      | Reading the Status Register                        | 31        |
| 4.7      | Exception Handling Effects                         | 31        |
| 4.8      | Usage Examples                                     | 31        |
| 4.8.1    | Conditional Execution                              | 31        |
| 4.8.2    | Manual Status Flag Control                         | 32        |
| 4.8.3    | Testing Bits                                       | 32        |
| 4.8.4    | Enabling Protected Mode                            | 32        |
| 4.8.5    | Saving and Restoring Status                        | 32        |

|           |                                         |           |
|-----------|-----------------------------------------|-----------|
| <b>5</b>  | <b>Exception and Interrupt Handling</b> | <b>33</b> |
| 5.1       | Overview                                | 33        |
| 5.2       | Exception Types                         | 33        |
| 5.2.1     | Faults                                  | 33        |
| 5.2.2     | Hardware Exceptions (Interrupts)        | 35        |
| 5.2.3     | User Exceptions (Traps)                 | 35        |
| 5.3       | Exception Registers                     | 35        |
| 5.3.1     | Link Registers                          | 36        |
| 5.3.2     | Cause Register                          | 38        |
| 5.3.3     | Hardware Interrupt Enable Bits          | 38        |
| 5.4       | Exception Processing                    | 39        |
| 5.4.1     | Exception Priority                      | 39        |
| 5.4.2     | Exception Handling Flow                 | 40        |
| 5.4.3     | Vector Table                            | 40        |
| 5.5       | Return from Exception                   | 41        |
| 5.5.1     | Accessing Link Registers                | 41        |
| 5.6       | Exception Coprocessor Instructions      | 42        |
| 5.6.1     | Internal Instructions                   | 42        |
| <br>      |                                         |           |
| <b>II</b> | <b>Instruction Set Architecture</b>     | <b>43</b> |
| <br>      |                                         |           |
| <b>6</b>  | <b>Instruction Formats</b>              | <b>45</b> |
| 6.1       | Overview                                | 45        |
| 6.2       | Format Overview                         | 45        |
| 6.3       | Common Fields                           | 45        |
| 6.4       | Immediate Format                        | 46        |
| 6.4.1     | Structure                               | 46        |
| 6.4.2     | Encoding Examples                       | 47        |
| 6.5       | Short Immediate Format (with Shift)     | 47        |
| 6.5.1     | Structure                               | 47        |
| 6.5.2     | Encoding Examples                       | 48        |
| 6.6       | Register Format                         | 49        |
| 6.6.1     | Structure                               | 49        |
| 6.6.2     | Register Operand Semantics              | 49        |
| 6.6.3     | Encoding Examples                       | 50        |
| 6.7       | Operand Encoding Details                | 51        |
| 6.7.1     | Condition Codes                         | 51        |
| 6.7.2     | Shift Types                             | 52        |
| 6.8       | Opcode Encoding Patterns                | 52        |
| 6.8.1     | ALU Instructions                        | 52        |
| 6.8.2     | Test vs. Save                           | 52        |
| 6.9       | Instruction Fetch and Alignment         | 53        |
| 6.9.1     | Fetch Process                           | 53        |
| 6.9.2     | Alignment Requirements                  | 53        |
| 6.10      | Format Selection Guidelines             | 53        |

|          |                               |           |
|----------|-------------------------------|-----------|
| <b>7</b> | <b>Addressing Modes</b>       | <b>55</b> |
| 7.1      | Overview                      | 55        |
| 7.2      | Immediate Addressing          | 55        |
| 7.2.1    | Description                   | 55        |
| 7.2.2    | Syntax                        | 55        |
| 7.2.3    | Effective Value               | 56        |
| 7.2.4    | Usage                         | 56        |
| 7.3      | Register Direct Addressing    | 56        |
| 7.3.1    | Description                   | 56        |
| 7.3.2    | Syntax                        | 56        |
| 7.3.3    | Effective Value               | 56        |
| 7.3.4    | Usage                         | 56        |
| 7.4      | Indirect Immediate Addressing | 56        |
| 7.4.1    | Description                   | 56        |
| 7.4.2    | Syntax                        | 56        |
| 7.4.3    | Effective Address             | 57        |
| 7.4.4    | Usage                         | 57        |
| 7.5      | Indirect Register Addressing  | 57        |
| 7.5.1    | Description                   | 57        |
| 7.5.2    | Syntax                        | 57        |
| 7.5.3    | Effective Address             | 57        |
| 7.5.4    | Usage                         | 57        |
| 7.6      | Post-Increment Addressing     | 57        |
| 7.6.1    | Description                   | 57        |
| 7.6.2    | Syntax                        | 58        |
| 7.6.3    | Operation Sequence            | 58        |
| 7.6.4    | Usage                         | 58        |
| 7.7      | Pre-Decrement Addressing      | 58        |
| 7.7.1    | Description                   | 58        |
| 7.7.2    | Syntax                        | 58        |
| 7.7.3    | Operation Sequence            | 58        |
| 7.7.4    | Usage                         | 58        |
| 7.8      | Short Immediate Addressing    | 59        |
| 7.8.1    | Description                   | 59        |
| 7.8.2    | Syntax                        | 59        |
| 7.8.3    | Effective Value               | 59        |
| 7.8.4    | Usage                         | 59        |
| 7.9      | Address Register Selection    | 59        |
| 7.10     | Addressing Mode Examples      | 59        |
| 7.10.1   | Structure Access              | 59        |
| 7.10.2   | Array Iteration               | 60        |
| 7.10.3   | Stack Frame                   | 60        |
| 7.11     | Addressing Mode Restrictions  | 60        |
| 7.11.1   | Privilege Mode Restrictions   | 60        |
| 7.11.2   | Alignment                     | 61        |

|          |                                 |           |
|----------|---------------------------------|-----------|
| <b>8</b> | <b>Shift Operations</b>         | <b>63</b> |
| 8.1      | Overview                        | 63        |
| 8.2      | Shift Types                     | 63        |
| 8.2.1    | Shift Type Encoding             | 63        |
| 8.3      | Shift Syntax                    | 64        |
| 8.4      | Shift Type Details              | 64        |
| 8.4.1    | Logical Shift Left (LSL)        | 64        |
| 8.4.2    | Logical Shift Right (LSR)       | 64        |
| 8.4.3    | Arithmetic Shift Left (ASL)     | 65        |
| 8.4.4    | Arithmetic Shift Right (ASR)    | 65        |
| 8.4.5    | Rotate Left (RTL)               | 66        |
| 8.4.6    | Rotate Right (RTR)              | 66        |
| 8.5      | Shift Operands                  | 67        |
| 8.5.1    | Mode 0: Shift by Immediate      | 67        |
| 8.5.2    | Mode 1: Shift by Register Count | 67        |
| 8.6      | Shift Count                     | 67        |
| 8.7      | Status Flag Effects             | 67        |
| 8.7.1    | Default Behavior                | 68        |
| 8.7.2    | Carry Flag                      | 68        |
| 8.7.3    | Zero Flag                       | 68        |
| 8.7.4    | Negative Flag                   | 68        |
| 8.7.5    | Overflow Flag                   | 68        |
| 8.8      | Common Use Cases                | 68        |
| 8.8.1    | Multiplication by Constants     | 68        |
| 8.8.2    | Division by Powers of 2         | 69        |
| 8.8.3    | Bit Field Extraction            | 69        |
| 8.8.4    | Bit Manipulation                | 69        |
| 8.9      | Multi-Word Shifts               | 69        |
| 8.9.1    | 32-bit Left Shift               | 69        |
| 8.9.2    | 32-bit Right Shift (Unsigned)   | 70        |
| 8.9.3    | 32-bit Right Shift (Signed)     | 70        |
| 8.10     | Performance Considerations      | 70        |
| 8.11     | Limitations                     | 70        |
| 8.12     | Status Flag Update Control      | 70        |
| 8.12.1   | Status Override Syntax          | 71        |
| 8.12.2   | Examples                        | 71        |
| 8.12.3   | Use Cases                       | 71        |
| 8.12.4   | Assembler Syntax Notes          | 72        |
| <b>9</b> | <b>Condition Codes</b>          | <b>73</b> |
| 9.1      | Overview                        | 73        |
| 9.2      | Condition Code Encoding         | 73        |
| 9.3      | Condition Code Categories       | 74        |
| 9.3.1    | Simple Conditions               | 74        |
| 9.3.2    | Unsigned Comparisons            | 74        |
| 9.3.3    | Signed Comparisons              | 74        |
| 9.3.4    | Special Conditions              | 75        |
| 9.4      | Assembly Syntax                 | 75        |

|                                                |                                                   |           |
|------------------------------------------------|---------------------------------------------------|-----------|
| 9.5                                            | Usage with Compare Instructions . . . . .         | 75        |
| 9.5.1                                          | Unsigned Comparison . . . . .                     | 75        |
| 9.5.2                                          | Signed Comparison . . . . .                       | 75        |
| 9.5.3                                          | Equality Testing . . . . .                        | 76        |
| 9.6                                            | Condition Code Truth Tables . . . . .             | 76        |
| 9.6.1                                          | After Unsigned Comparison (CMPR rA, rB) . . . . . | 76        |
| 9.6.2                                          | After Signed Comparison (CMPR rA, rB) . . . . .   | 76        |
| 9.7                                            | Advanced Usage . . . . .                          | 77        |
| 9.7.1                                          | Conditional Assignment . . . . .                  | 77        |
| 9.7.2                                          | Conditional Increment . . . . .                   | 77        |
| 9.7.3                                          | Conditional Function Call . . . . .               | 77        |
| 9.7.4                                          | Loop Construction . . . . .                       | 77        |
| 9.8                                            | Condition Code Selection Guide . . . . .          | 78        |
| 9.9                                            | Performance Considerations . . . . .              | 78        |
| 9.10                                           | Common Pitfalls . . . . .                         | 78        |
| <br><b>III SIRCIS Instruction Reference</b>    |                                                   | <b>81</b> |
| <br><b>10 Reading Instruction Descriptions</b> |                                                   | <b>83</b> |
| 10.1                                           | Instruction Entry Fields . . . . .                | 83        |
| 10.2                                           | Notation . . . . .                                | 84        |
| 10.3                                           | Status Flag Effect Symbols . . . . .              | 84        |
| 10.4                                           | Conditional Execution . . . . .                   | 84        |
| 10.5                                           | Status Update Overrides . . . . .                 | 84        |
| <br><b>11 Instruction Summary</b>              |                                                   | <b>87</b> |
| 11.1                                           | Overview . . . . .                                | 87        |
| 11.2                                           | Complete Instruction List . . . . .               | 88        |
| 11.3                                           | Instruction Grouping Patterns . . . . .           | 89        |
| 11.3.1                                         | Opcode Organization . . . . .                     | 89        |
| 11.3.2                                         | Save vs. Test Variants . . . . .                  | 89        |
| 11.4                                           | Meta-Instructions . . . . .                       | 89        |
| 11.5                                           | Instruction Timing . . . . .                      | 89        |
| 11.6                                           | Next Chapters . . . . .                           | 90        |
| <br><b>12 ALU Instructions</b>                 |                                                   | <b>91</b> |
| 12.1                                           | Overview . . . . .                                | 91        |
| 12.2                                           | ALU Instruction Families . . . . .                | 91        |
| 12.2.1                                         | Arithmetic Instructions . . . . .                 | 91        |
| 12.2.2                                         | Logical Instructions . . . . .                    | 91        |
| 12.2.3                                         | Data Movement . . . . .                           | 91        |
| 12.2.4                                         | Test and Compare . . . . .                        | 92        |
| 12.3                                           | Legal Forms . . . . .                             | 92        |
| 12.4                                           | Common ALU Semantics . . . . .                    | 93        |
| 12.5                                           | Status Flag Updates . . . . .                     | 93        |
| 12.5.1                                         | Manual Flag Update Control . . . . .              | 94        |
| 12.6                                           | ADD – Addition . . . . .                          | 95        |

|           |                                           |            |
|-----------|-------------------------------------------|------------|
| 12.7      | ADC – Add with Carry                      | 96         |
| 12.8      | SUB – Subtraction                         | 97         |
| 12.9      | SBC – Subtract with Carry (Borrow)        | 98         |
| 12.10     | AND – Bitwise AND                         | 99         |
| 12.11     | ORR – Bitwise OR                          | 100        |
| 12.12     | XOR – Bitwise Exclusive OR                | 101        |
| 12.13     | LOAD – Load Immediate / Move Register     | 102        |
| 12.14     | CMP – Compare                             | 104        |
| 12.15     | TSA – Test AND                            | 106        |
| 12.16     | TSX – Test XOR                            | 107        |
| 12.17     | Summary                                   | 108        |
| <b>13</b> | <b>Memory Access Instructions</b>         | <b>109</b> |
| 13.1      | Overview                                  | 109        |
| 13.2      | Effective Address Calculation             | 109        |
| 13.3      | LOAD Instructions                         | 110        |
| 13.4      | STOR Instructions                         | 111        |
| 13.5      | Common Memory Access Patterns             | 111        |
| 13.5.1    | Structure Field Access                    | 111        |
| 13.5.2    | Array Iteration                           | 111        |
| 13.5.3    | Stack Operations                          | 112        |
| 13.6      | Shift Operations with Memory Instructions | 112        |
| 13.6.1    | LOAD with Shift                           | 112        |
| 13.6.2    | STOR with Shift                           | 112        |
| 13.6.3    | Supported Shift Types                     | 113        |
| 13.6.4    | Shift Limitations                         | 113        |
| 13.6.5    | Use Cases                                 | 113        |
| 13.6.6    | Status Flags                              | 114        |
| <b>14</b> | <b>Control Flow Instructions</b>          | <b>115</b> |
| 14.1      | Overview                                  | 115        |
| 14.2      | Branch Instructions                       | 116        |
| 14.3      | Long Jump                                 | 117        |
| 14.4      | Subroutine Call Instructions              | 118        |
| 14.5      | Load Effective Address                    | 119        |
| 14.6      | Control Flow Patterns                     | 119        |
| 14.6.1    | If-Then-Else                              | 119        |
| 14.6.2    | While Loop                                | 119        |
| 14.6.3    | Switch/Jump Table                         | 120        |
| <b>15</b> | <b>Coprocessor Instructions</b>           | <b>121</b> |
| 15.1      | Overview                                  | 121        |
| 15.2      | Exception Unit (Coprocessor 1)            | 122        |
| 15.2.1    | Common Operations                         | 123        |

|                                              |                |
|----------------------------------------------|----------------|
| <b>16 Meta-Instructions</b>                  | <b>125</b>     |
| 16.1 Overview                                | 125            |
| 16.2 NOOP – No Operation                     | 125            |
| 16.3 RETS – Return from Subroutine           | 125            |
| 16.4 LJMP – Long Jump                        | 125            |
| 16.5 EXCP – User Exception (Trap)            | 126            |
| 16.6 WAIT – Wait for Interrupt               | 126            |
| 16.7 RETE – Return from Exception            | 126            |
| 16.8 RSET – System Reset                     | 127            |
| 16.9 ETFR – Exception Transfer From Register | 127            |
| 16.10 ETTR – Exception Transfer To Register  | 127            |
| 16.11 SHFT – Shift with Status Update        | 128            |
| <br><b>A Opcode Map</b>                      | <br><b>129</b> |
| A.1 Complete Opcode Reference                | 129            |
| A.2 Opcode Patterns                          | 131            |
| A.2.1 Format Identification                  | 131            |
| A.2.2 Save vs. Test                          | 131            |
| A.2.3 Memory Operations Decoding             | 131            |
| A.3 Undocumented Instructions                | 131            |
| A.4 Quick Lookup Table                       | 132            |
| <br><b>B Instruction Timing</b>              | <br><b>133</b> |
| B.1 Overview                                 | 133            |
| B.2 Six-Stage Execution Phases               | 133            |
| B.3 Timing Characteristics                   | 134            |
| B.4 Conditional Execution                    | 134            |
| B.5 Program Timing Examples                  | 134            |
| B.5.1 Simple Loop                            | 134            |
| B.5.2 Function Call Overhead                 | 135            |
| B.5.3 Conditional Block                      | 135            |
| B.6 Performance Considerations               | 135            |
| B.6.1 No Pipeline Stalls                     | 135            |
| B.6.2 Memory Access                          | 135            |
| B.6.3 Optimization Guidelines                | 136            |
| B.7 Theoretical Performance                  | 136            |
| B.8 Comparison to Other CPUs                 | 137            |
| <br><b>C Undocumented Instructions</b>       | <br><b>139</b> |
| C.1 Overview                                 | 139            |
| C.2 Undocumented Opcode List                 | 139            |
| C.3 Likely Behavior                          | 140            |
| C.3.1 0x08, 0x28, 0x38 (ADD Test)            | 140            |
| C.3.2 0x09, 0x29, 0x39 (ADC Test)            | 140            |
| C.3.3 0x0B, 0x2B, 0x3B (SUB Test)            | 140            |
| C.3.4 0x0D, 0x2D, 0x3D (SBC Test)            | 140            |
| C.4 Implementation Notes                     | 140            |
| C.4.1 Current Simulator                      | 140            |

---

|          |                                      |            |
|----------|--------------------------------------|------------|
| C.4.2    | Hardware Implementation . . . . .    | 141        |
| C.5      | Why Undocumented? . . . . .          | 141        |
| C.6      | Recommendations . . . . .            | 141        |
| C.6.1    | For Application Developers . . . . . | 141        |
| C.6.2    | For Compiler Writers . . . . .       | 141        |
| C.6.3    | For Hardware Implementers . . . . .  | 142        |
| C.6.4    | For Researchers/Hackers . . . . .    | 142        |
| C.7      | Future Standardization . . . . .     | 142        |
| C.8      | Opcode Space for Expansion . . . . . | 142        |
| <b>D</b> | <b>Example Programs</b>              | <b>143</b> |
| D.1      | Byte Sieve . . . . .                 | 143        |
| D.2      | Faults . . . . .                     | 144        |
| D.3      | Hardware Exception . . . . .         | 150        |
| D.4      | Software Exception . . . . .         | 155        |
| D.5      | Store Load . . . . .                 | 156        |



# List of Tables

|      |                                                     |    |
|------|-----------------------------------------------------|----|
| 1.1  | SIRC-1 CPU Specifications . . . . .                 | 4  |
| 1.2  | List of coprocessors . . . . .                      | 5  |
| 2.1  | SIRC CPU 64-pin DIP pinout . . . . .                | 14 |
| 2.2  | Bus Access Type Bits Meaning . . . . .              | 17 |
| 3.1  | Register Encoding . . . . .                         | 25 |
| 5.1  | Exception Coprocessor Instruction Summary . . . . . | 42 |
| 6.1  | Instruction Format Summary . . . . .                | 45 |
| 6.2  | Immediate Format Encoding Examples . . . . .        | 47 |
| 6.3  | Short Immediate Format Encoding Examples . . . . .  | 48 |
| 6.4  | Register Format Encoding Examples . . . . .         | 50 |
| 6.5  | Condition Code Encoding . . . . .                   | 51 |
| 6.6  | Shift Type Encoding . . . . .                       | 52 |
| 6.7  | Format Selection Examples . . . . .                 | 53 |
| 7.1  | Addressing Modes Summary . . . . .                  | 55 |
| 7.2  | Address Register Encoding . . . . .                 | 59 |
| 8.1  | Shift Type Encoding . . . . .                       | 63 |
| 8.2  | Examples of first source operands . . . . .         | 64 |
| 8.3  | Shift Operand Modes . . . . .                       | 67 |
| 8.4  | Status Flag Override Modifiers . . . . .            | 71 |
| 9.1  | Condition Code Encoding . . . . .                   | 73 |
| 9.2  | Unsigned Comparison Results . . . . .               | 76 |
| 9.3  | Signed Comparison Results . . . . .                 | 76 |
| 9.4  | Condition Code Selection Guide . . . . .            | 78 |
| 10.1 | Instruction Description Notation . . . . .          | 84 |
| 10.2 | Status Flag Effect Symbols . . . . .                | 84 |
| 10.3 | Status Update Source Encoding . . . . .             | 85 |
| 11.1 | Complete SIRCIS Instruction Set . . . . .           | 88 |
| 11.2 | Meta-Instructions . . . . .                         | 89 |
| 12.1 | Legal ALU Instruction Forms . . . . .               | 92 |

|                                                     |     |
|-----------------------------------------------------|-----|
| 12.2 Common ALU Semantics . . . . .                 | 93  |
| 12.3 ALU Status Flag Effects . . . . .              | 94  |
| 15.1 Exception Unit Operations . . . . .            | 123 |
| A.1 Complete SIRCIS Opcode Map . . . . .            | 130 |
| A.2 Instruction Variant Quick Reference . . . . .   | 132 |
| B.1 Instruction Timing (All instructions) . . . . . | 134 |
| B.2 Cycles Per Instruction Comparison . . . . .     | 137 |
| C.1 Undocumented Instruction Opcodes . . . . .      | 139 |

# List of Figures

|     |                                                   |    |
|-----|---------------------------------------------------|----|
| 2.1 | SIRC-1 CPU Execution Phase Architecture . . . . . | 10 |
| 2.2 | SIRC CPU 64-pin DIP pinout, top view . . . . .    | 13 |
| 3.1 | Address Register Pair Formation . . . . .         | 22 |
| 4.1 | Status Register Bit Layout . . . . .              | 27 |
| 6.1 | Immediate Instruction Format . . . . .            | 46 |
| 6.2 | Short Immediate with Shift Format . . . . .       | 47 |
| 6.3 | Register Instruction Format . . . . .             | 49 |
| 8.1 | Logical Shift Left . . . . .                      | 64 |
| 8.2 | Logical Shift Right . . . . .                     | 64 |
| 8.3 | Arithmetic Shift Right . . . . .                  | 65 |
| 8.4 | Rotate Left . . . . .                             | 66 |
| 8.5 | Rotate Right . . . . .                            | 66 |



## Part I

# Architecture Overview



# Chapter 1

## Introduction

### 1.1 Overview

The SIRC-1 is a 16-bit RISC processor ideal for general purpose computing.

It implements the SIRCIS (SIRC Instruction Set) instruction set architecture, which provides a powerful set of operations while still ensuring simplicity of CPU implementation.

### 1.2 Design Philosophy

#### 1.2.1 RISC Principles

The SIRC-1 adheres to Reduced Instruction Set Computer (RISC) design principles:

- **Load/Store Architecture:** All ALU operations work exclusively on registers. Memory access is performed only through dedicated load and store instructions.
- **Fixed Instruction Length:** All instructions are exactly 32 bits (2 words) in length, simplifying instruction fetch and decode logic.
- **Simple Instruction Formats:** Only three instruction formats are supported: Immediate, Short Immediate with Shift, and Register.
- **Register-Rich:** Sixteen addressable registers provide ample workspace for computations without frequent memory access.
- **Consistent Execution Time:** All instructions use exactly 6 execution phases and complete in 6 clock cycles when no bus wait states are required, eliminating the need for microcode and simplifying hardware implementation.

#### 1.2.2 Design Goals

The SIRC-1 was designed with several key goals in order of importance:

**Simplicity** The CPU should be simple enough to be implemented in hardware without requiring complex microcode engines or multi-level instruction decoders.

**Developer Ergonomics** It should have powerful instruction encodings that include conditional execution and bit shifting that can help reduce the number of instructions and branches to keep track of.

**Performance** While simple, the design should allow for reasonable performance through efficient instruction encoding and the potential for future enhancements like pipelining.

**Memory Protection** Basic privilege levels and memory segmentation provide rudimentary protection for system software.

### 1.2.3 Applications

The SIRC-1 is designed for ideal for general purpose computing, such as the core of a microcomputer.

It should not be used in memory constrained environments, as the fixed length instructions and lack of byte addressing can potentially mean larger program sizes.

## 1.3 Key Features

### 1.3.1 Architecture Characteristics

| Feature                     | Specification               |
|-----------------------------|-----------------------------|
| Data Width                  | 16-bit                      |
| Addressing                  | Word Addressing             |
| Address Bus                 | 24-bit (32MB address space) |
| Instruction Size            | 32-bit fixed                |
| General Purpose Registers   | 7 (r1–r7)                   |
| Address Register Pairs      | 4 (l, a, s, p)              |
| Total Addressable Registers | 16                          |
| Instruction Formats         | 3                           |
| Addressing Modes            | 7                           |
| Condition Codes             | 16                          |

Table 1.1: SIRC-1 CPU Specifications

### 1.3.2 Execution Model

The SIRC-1 executes every instruction through six sequential execution phases (normally one phase per clock cycle). For a full description of each phase, see Chapter 2.

This fixed six-stage model means that all instructions take 6 clock cycles to complete when bus operations do not require wait states, regardless of their complexity. A bus device may hold the current phase by delaying bus acknowledgement, adding wait cycles without adding execution phases. While this may seem wasteful for simple register operations that don't require memory access, it dramatically simplifies the hardware design by eliminating the need for variable-length execution sequences or microcode.

### 1.3.3 Coprocessors

The SIRC-1 is made up of multiple coprocessors (also known as modules) that all share the same address/data bus and register file. Only one coprocessor can be active at one time and are usually activated using the COP instructions (except for the "exception unit" which can be activated when physical pins are asserted on the chip). All implementations of the SIRC-1 will have the "processing unit" as coprocessor 0x0 and "exception unit" as coprocessor 0x1. Other coprocessors are optional and will vary depending on which SIRC-1 model you have. The instruction set supports up to 16 coprocessors. The coprocessor IDs are stable between models, so that programs are cross compatible. For example, coprocessor 0x2 will always be the DMA unit. This means if the CPU model is missing a particular coprocessor, and a program requires it, the CPU will raise an exception so that the behaviour can be emulated in software.

| ID  | Name            | Optional | Purpose                                      |
|-----|-----------------|----------|----------------------------------------------|
| 0x0 | Processing Unit | No       | Executes instructions                        |
| 0x1 | Exception Unit  | No       | Handles interrupts and exceptions            |
| 0x2 | DMA Unit        | Yes      | Transfers large amounts of data via the bus  |
| 0x3 | Maths Unit      | Yes      | Performs mathematical operations in hardware |
| 0x4 | Reserved        | Yes      | TBA                                          |
| 0x5 | Reserved        | Yes      | TBA                                          |
| 0x6 | Reserved        | Yes      | TBA                                          |
| 0x7 | Reserved        | Yes      | TBA                                          |
| 0x8 | Reserved        | Yes      | TBA                                          |
| 0x9 | Reserved        | Yes      | TBA                                          |
| 0xA | Reserved        | Yes      | TBA                                          |
| 0xB | Reserved        | Yes      | TBA                                          |
| 0xC | Reserved        | Yes      | TBA                                          |
| 0xD | Reserved        | Yes      | TBA                                          |
| 0xE | Reserved        | Yes      | TBA                                          |
| 0xF | Reserved        | Yes      | TBA                                          |

Table 1.2: List of coprocessors

Each coprocessor can execute 16 opcodes, the first 8 (0x0-0x7) can be called in any mode. The last 8 (0x8-0xF) can only be called in supervisor mode.

### 1.3.4 Privilege Levels

The SIRC-1 supports two privilege levels:

**Supervisor Mode** Full access to all registers and instructions. Can set the high bytes of address register pairs, access privileged status register bits, and execute privileged coprocessor operations.

**Protected Mode** Restricted access for user programs. Cannot write to the high bytes of address register pairs (providing memory segmentation/protection), cannot access privileged status register bits, and cannot execute privileged coprocessor instructions.

The privilege level is controlled by the Protected Mode bit in the status register.

## 1.4 Memory Organization

### 1.4.1 Address Space

The SIRC-1 uses word addressing, where each address refers to a 16-bit word. With a 24-bit address space, this allows access to  $2^{24} = 16,777,216$  words, or 32 megabytes of memory. Addresses are formed by combining the 8-bit high byte and 16-bit low word of address register pairs:

$$\text{Address} = (\text{RegHigh}[7 : 0] \ll 16) | \text{RegLow}[15 : 0] \quad (1.1)$$

Note that the upper 8 bits of the high register (bits 15–8) are not used for addressing but are available for storage, enabling techniques like tagged pointers.

### 1.4.2 Memory Map

The CPU expects certain memory regions to be defined:

- **Reset Vector** (0x000000): Initial program counter after reset
- **Fault Vectors** (0x000001–0x00000F): Fault vectors and reserved privileged exception vectors
- **Hardware Vectors** (0x000010–0x000050): 5 x Hardware exception vectors (offset by 0x10 due to hardware implementation)
- **User Vectors** (0x000060–0x0000FF): User exception vectors

Beyond these reserved regions, the memory layout is flexible and can be defined by system software.

### 1.4.3 Alignment

- All memory addresses refer to 16-bit words
- There are no alignment restrictions for general memory I/O
- All instructions span 2 words and must begin at an even word address
- Unaligned instruction fetches (odd word address) trigger an alignment fault exception

## 1.5 Document Organization

This manual is organized into three main parts:

**Part I: Architecture Overview** Covers the register model, status register, and overall CPU architecture.

**Part II: Instruction Set Architecture** Details instruction formats, addressing modes, shift operations, and condition codes.

**Part III: SIRCIS Instruction Reference** Provides detailed documentation for each instruction, including encoding, operation, and examples.

Appendices provide quick reference materials including complete opcode maps, timing diagrams, and notes on undocumented instructions.



## Chapter 2

# CPU Architecture

### 2.1 Overview

The SIRC-1 CPU implements a combinatorial logic design with no microcode, utilizing six sequential execution phases to execute instructions in 6 clock cycles when no bus wait states are required. The architecture is composed of several key functional units that work together to fetch, decode, execute, and complete instructions.

### 2.2 Six-Stage Execution Phases

All SIRCIS instructions follow a six-phase execution model and complete in 6 clock cycles when no bus wait states are required. A phase that performs a bus operation may be held until the bus acknowledges it. This fixed execution sequence greatly simplifies software development and hardware design.

1. **Instruction Fetch (High Word)** – 1 cycle
  - Fetch bits 31–16 from memory address in PC
  - Increment PC by 1
2. **Instruction Fetch (Low Word)** – 1 cycle
  - Fetch bits 15–0 from memory address in PC
  - Increment PC by 1
  - Instruction is now fully available for decode
3. **Decode and Register Fetch** – 1 cycle
  - Decode instruction opcode and operands
  - Read source registers (optionally shifted)
  - Evaluate condition codes
4. **Execute and Address Calculation** – 1 cycle
  - Perform ALU operation, and/or
  - Calculate effective memory address

**5. Memory Access – 1 cycle**

- Read from memory (LOAD), or
- Write to memory (STOR), or
- No operation (register-only instructions)

**6. Write Back – 1 cycle**

- Write result to destination register
- Update status register flags if applicable
- Update address registers for post-increment/pre-decrement

## 2.3 Architectural Components

Figure 2.1 illustrates the major functional units within the SIRC-1 CPU and their interconnections.

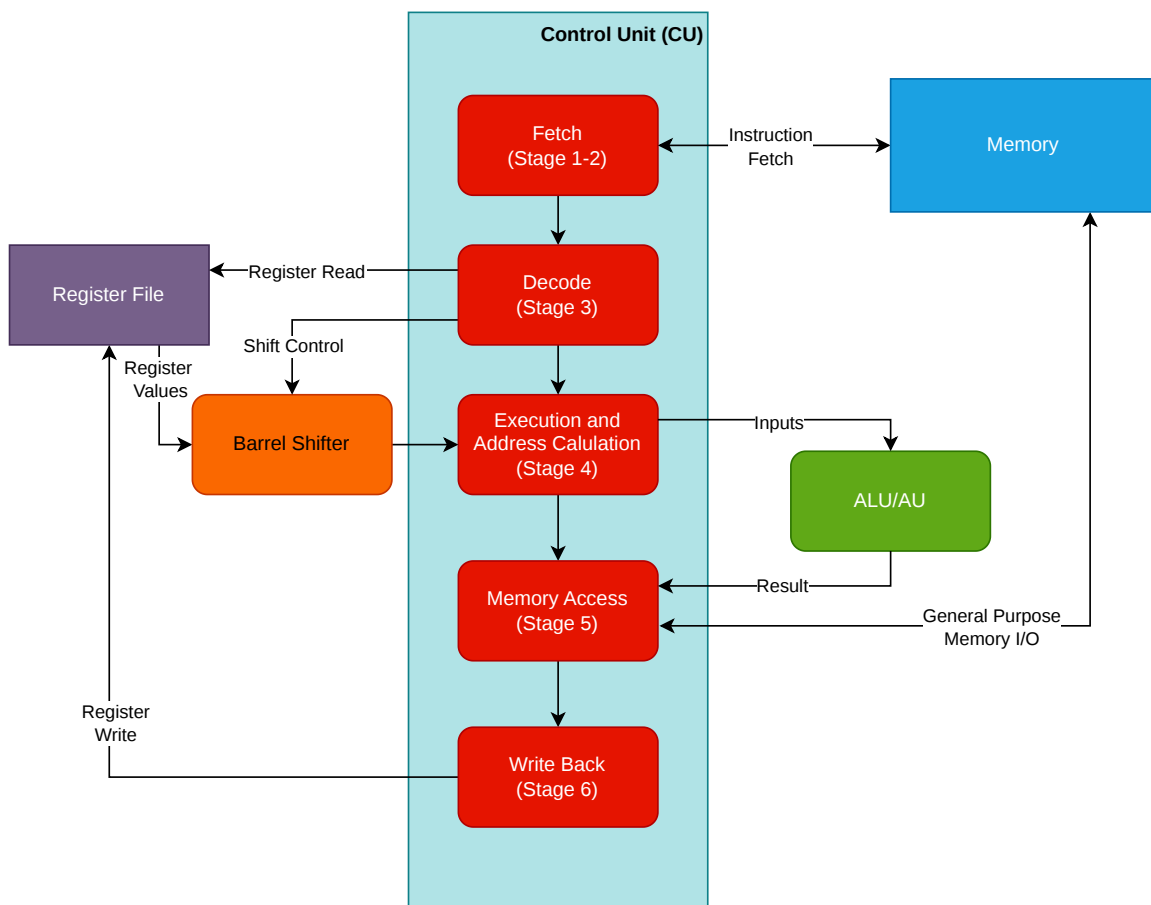


Figure 2.1: SIRC-1 CPU Execution Phase Architecture

### 2.3.1 Control Unit (CU)

The Control Unit sits at the heart of the SIRC-1 CPU, coordinating all other functional units throughout the instruction execution phases. It decodes instructions and generates control signals that orchestrate data movement and operations across the CPU.

Key responsibilities include:

- Instruction fetch and decode
- Execution phase coordination
- Control signal generation for all functional units
- Condition code evaluation for conditional execution

### 2.3.2 Arithmetic Logic Unit (ALU)

The ALU performs all arithmetic and logical operations on register values. It supports the following operations:

- **Arithmetic:** Addition, Subtraction (with and without carry)
- **Logical:** AND, OR, XOR
- **Comparison:** Test and compare operations
- **Status Flag Generation:** Carry, Overflow, Negative, Zero

The ALU operates exclusively on data from the Register File (after optional shifting) and outputs results either back to the Register File or to memory via the data bus.

### 2.3.3 Addressing Unit (AU)

The Addressing Unit calculates memory addresses and manages address-related operations. It is responsible for:

- Computing effective addresses from base + displacement
- Handling pre-decrement and post-increment addressing modes
- Updating address register pairs (l, a, s, p) for LDEA, jump, and branch instructions
- Presenting calculated addresses to the address bus

The AU works in parallel with the ALU during the execution stage, allowing address calculation to occur simultaneously with ALU operations.

### 2.3.4 Register File

The Register File contains all CPU registers organized into three categories:

- **General Purpose Registers:** r1–r7
- **Address Register Pairs:** l (link), a (address), s (stack), p (program counter)
- **Status Register:** sr

The Register File provides multi-port access, allowing the ALU, AU, and Control Unit to read and write registers as needed during each execution phase. All registers are 16 bits wide, though address register pairs can be combined to form 24-bit addresses.

### 2.3.5 Barrel Shifters

Two barrel shifter units sit between the Register File and the CU, enabling efficient bit manipulation operations.

The barrel shifters support logical shift left (LSL), logical shift right (LSR), arithmetic shift right (ASR), and rotate operations. Shift amounts can be specified as immediate values or dynamically from register contents.

The shift is applied to the first source operand when fetching the source registers for an instruction. Shifts cannot be applied to the result or any other source operand.

## 2.4 Design Characteristics

### 2.4.1 Combinatorial Logic Implementation

The SIRC-1 uses pure combinatorial logic with no microcode engine. Each execution phase is implemented as combinatorial circuits that process data in a single clock cycle. This approach:

- Simplifies the hardware design
- Provides a deterministic, fixed execution sequence
- Eliminates the complexity of microcode sequencing
- Makes the CPU behavior fully predictable and easy to reason about

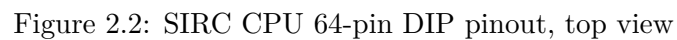
### 2.4.2 Fixed Execution Sequence

All instructions execute in exactly 6 phases, regardless of instruction type or operands, and complete in 6 clock cycles when no bus wait states are required. Delayed bus acknowledgement can hold a phase and add wait cycles. This design choice simplifies software timing and eliminates the need for branch prediction or complex multi-stage control logic found in variable-time architectures.

### 2.4.3 Parallel Functional Units

The ALU and AU operate in parallel during Stage 4, allowing address calculation to proceed simultaneously with arithmetic operations. This parallelism enables efficient execution of memory reference instructions without additional clock cycles.

The SIRC CPU is most commonly provided as a DIP-64 package.



### 2.5.2 Pin Descriptions

- **Direction:** Output
- **Signal Type:** Active High

These 24 pins declare the address on the bus that the CPU will read or write from when BAS (bus access strobe) is asserted.

| Pin   | Name      | Description                        |
|-------|-----------|------------------------------------|
| 1-8   | A0-A7     | Address bus bits 0-7               |
| 9     | VCC1      | Power supply                       |
| 10-25 | A8-A23    | Address bus bits 8-23              |
| 26    | GND1      | Ground                             |
| 27-42 | D0-D15    | Data bus bit 0-15                  |
| 43    | VCC2      | Power supply                       |
| 44    | GND2      | Ground                             |
| 45    | CLKI      | Clock input                        |
| 46    | SYNC      | Instruction-complete strobe output |
| 47    | BAS       | Bus access strobe output           |
| 48    | BRW       | Bus read/write output              |
| 49    | BACK      | Bus acknowledge input              |
| 50    | BERR      | Bus error input                    |
| 51    | BPER      | Bus protection error               |
| 52-54 | BAT0-BAT2 | Bus access type bits 0-2           |
| 55    | PROT      | Protected-mode output              |
| 56    | HALT      | External CPU halt input            |
| 57    | TRCE      | Force trace mode input             |
| 58-61 | IRQ1-IRQ4 | Maskable interrupt inputs 1-4      |
| 62    | NMI       | Non-maskable interrupt input       |
| 63    | RSTI      | Reset input                        |
| 64    | RSTO      | Reset output                       |

Table 2.1: SIRC CPU 64-pin DIP pinout

**Power Pins (VCC1-2, GND1-2)**

These pins provide the 5V power that the CPU requires to function.

The VCC pins must be connected to 5V +/- 5% and GND pins must be connected to a 0V or ground rail.

**Data Bus Bits (D0-D15)**

- **Direction:** Bi-directional
- **Signal Type:** Active High

Connected to the data bus.

These 16 pins will be driven to a 16-bit value when the CPU is outputting to the bus, and will be an input for a 16-bit value when the CPU is reading from the bus.

**Clock Input (CLKI)**

- **Direction:** Input
- **Signal Type:** Rising Edge
- **Max Rate:** 24 Mhz

Progresses the CPU to the next cycle when the input transitions from low to high (rising edge).

Each transition from low to high will progress the CPU internal six stage sequence, and so every six clock transitions will be a full instruction execution (assuming no wait for bus devices).

**Bus Access Strobe Output (SYNC)**

- **Direction:** Output
- **Signal Type:** Rising Edge

When the CPU completes a full instruction (six internal cycles) it will transition this pin from low to high to signal external devices.

It will be high for cycle 1, and low for every other cycle.

It can be useful for debugging purposes.

**Bus Access Strobe Output (BAS)**

- **Direction:** Output
- **Signal Type:** Rising Edge

When the CPU is ready to perform a bus operation (i.e. the address pins, data pins and BRW pin are all in the right state) it will transition this pin from low to high.

It will then not progress to the next cycle until the bus asserts the BACK (Bus acknowledge input) pin, allowing slow devices time to complete operations.

### Bus Access Strobe Output (BRW)

- **Direction:** Output
- **Signal Type:** Active High

Only relevant when the BAS (Bus access strobe output) is asserted.

When low, the CPU is requesting a read from the bus. Otherwise, when high, the CPU is requesting to write to the bus.

### Bus Acknowledge Input (BACK)

- **Direction:** Input
- **Signal Type:** Active Low

When low, the bus is asserting that the previously requested read/write was successful, allowing the CPU I/O operation to complete.

If all bus devices are fast and can complete reads/writes in a single cycle, this pin can be permanently pulled low.

### Bus Error Input (BERR)

- **Direction:** Input
- **Signal Type:** Active Low

When low, the bus is asserting that the previously requested read/write was unsuccessful.

This will cause the CPU to abort the I/O operation and raise a bus fault exception.

An example of a situation where a bus error might be raised would be I/O to an address that is not connected to any device.

### Bus Protection Error Input (BPER)

- **Direction:** Input
- **Signal Type:** Active Low

When low, the bus is asserting that the previously requested read/write was to a valid address/device, but that the I/O is disallowed given the current state of the CPU or bus.

This will cause the CPU to abort the I/O operation and raise a bus protection fault exception.

It acts very similar to the BERR pin but raises a different fault so that programs can handle it differently.

An example of a situation where a bus protection error might be raised would be a write to ROM, or a write to a privileged section of memory when the CPU is not in supervisor mode.

| BAT2 | BAT1 | BAT0 | Description                  |
|------|------|------|------------------------------|
| 0    | 0    | 0    | No I/O                       |
| 0    | 0    | 1    | Instruction Fetch            |
| 0    | 1    | 0    | Data Read                    |
| 0    | 1    | 1    | Data Write                   |
| 1    | 0    | 0    | Exception Vector Fetch       |
| 1    | 0    | 1    | DMA Co-processor Read Burst  |
| 1    | 1    | 0    | DMA Co-processor Write Burst |
| 1    | 1    | 1    | Reserved                     |

Table 2.2: Bus Access Type Bits Meaning

**Bus Access Type Bits (BAT0-BAT2)**

- **Direction:** Output
- **Signal Type:** Active High

Output by the CPU to describe what I/O operation it is performing.

Can be used to provide useful hints to external devices during I/O.

For example, if a memory controller knows that the DMA co-processor will be reading more than one sequential address, it can prefetch the next address.

Another example might be a memory controller providing memory protection if these pins are used in conjunction with the PROT (Protected-mode output) pin. A memory controller might trigger a bus error if BAT is 010 (data read), PROT is 1 (user mode) and the address is in a protected memory region.

**Protected-mode Output (PROT)**

- **Direction:** Output
- **Signal Type:** Active High

Indicates whether the CPU is in protected mode (high) or in supervisor mode (low).

Connected directly to the status register Protected Mode (P) bit.

When the CPU is in protected mode, programs have access to less types of instructions (e.g. they cannot write to the upper word of the address registers).

Exposing this as an external pin allows other devices such as memory controllers to block access to protected regions of memory by raising bus protection errors.

**External CPU halt input (HALT)**

- **Direction:** Input
- **Signal Type:** Active Low

When low, the CPU will stop executing instructions.

It will complete the current instruction and halt before the next instruction fetch. It will resume as normal when the input goes high.

This pin is useful for hardware driven debugging, as the CPU can be halted and the state of the system examined.

### **Force trace mode input (TRCE)**

- **Direction:** Input
- **Signal Type:** Active Low

When low, the Trace Mode (T) flag of the status register is forced to be 1, regardless of whether a program has cleared it or not.

When in trace mode the CPU will raise exceptions every instruction to allow software debugging.

### **Maskable interrupt inputs (IRQ1-IRQ4)**

- **Direction:** Input
- **Signal Type:** Active Low

When low, a hardware exception will be raised by the CPU for the corresponding exception level.

They can be "masked" by the Hardware Interrupt Enable Bits (E1-E4) of the status register, which will disable them.

If an exception for the interrupt level or higher is already being handled, the exception will be "queued" and handled whenever all the higher level interrupts have been serviced.

If a lower level interrupt is being handled, it will be interrupted and the higher level interrupt will be serviced.

If an interrupt pin is held low continuously it will be serviced again as soon as the exception has been handled.

See the exceptions section for more details.

### **Non-maskable interrupt inputs (NMI)**

- **Direction:** Input
- **Signal Type:** Falling Edge

When transitioning from high to low, will raise a level 5 hardware exception.

This interrupt cannot be masked, and if this pin is asserted while another level 5 exception is being handled, a "level five hardware exception conflict" fault will be raised.

If asserted again while that fault is being handled, it will trigger a double fault.

Therefore, it is important that the level 5 exception handlers are executed quickly and that the NMI is only asserted for important non-interruptible events.

**Reset input (RSTI)**

- **Direction:** Input
- **Signal Type:** Active low

Can be used by external devices to force a reset of the CPU.

When low, the CPU will immediately be interrupted and stop processing. It will be set to cycle 0 and the cause register will be set to 0x1B00 (RSET).

The RSTO pin will also be asserted.

When this pin becomes high, the CPU will wait 6 cycles, stop asserting the RSTO pin, and the the exception co-processor will execute the RSET instruction, resetting the CPU state and fetching the reset vector.

If your system needs external devices to be reset for longer than 6 cycles to allow the program, to run, the resets will have to be managed externally to the CPU and RSTI will need to be asserted for however long is needed for the other devices to reset.

**Reset output (RSTO)**

- **Direction:** Output
- **Signal Type:** Active high

Asserted while the RSTI pin is asserted, and for 6 cycles after the RSTI pin is no longer asserted.

It will also be asserted for 6 cycles after a software reset

A reset can be triggered by asserting the Reset input (RSTI) pin, or in software by a special exception co-processor meta instruction (RSET).

This pin is useful to reset other devices on the bus whenever the CPU is reset or to trigger more complex reset/boot controllers.

**2.5.3 Timing Notes**

BACK, BERR and BPER are all mutually exclusive. If they happen to be asserted by the bus at the same time, BERR will take first priority, then BPER, then BACK.



## Chapter 3

# Register Model

### 3.1 Overview

The SIRC-1 CPU provides sixteen 16-bit registers that serve various purposes. These registers can be classified into three categories:

- General purpose registers (**r1–r7**)
- Address register pairs (**l, a, s, p**)
- Status register (**sr**)

All programmer-visible registers are 16 bits wide. Address registers can be used in pairs to form 24-bit addresses for memory operations.

### 3.2 General Purpose Registers

#### 3.2.1 Register Set

The SIRC-1 provides seven general-purpose registers:

| Register | Description                |
|----------|----------------------------|
| r1       | General purpose register 1 |
| r2       | General purpose register 2 |
| r3       | General purpose register 3 |
| r4       | General purpose register 4 |
| r5       | General purpose register 5 |
| r6       | General purpose register 6 |
| r7       | General purpose register 7 |

These registers can be used freely for any purpose by the programmer. They hold 16-bit values and can be used as source or destination operands for all ALU instructions.

### 3.2.2 Usage Conventions

While there are no hardware-enforced conventions, typical usage patterns include:

- **r1–r4**: Temporary values and expression evaluation
- **r5–r7**: Preserved across function calls (by convention)

These are software conventions only and are not enforced by the CPU.

## 3.3 Address Register Pairs

### 3.3.1 Overview

The SIRC-1 provides four address register pairs, each consisting of a high and low 16-bit register. When used together, these pairs form 24-bit addresses by combining the low 8 bits of the high register with the full 16 bits of the low register.

|                                                                                       |  |  |  |  |  |  |  |               |  |  |  |  |  |  |  |              |  |  |  |  |  |  |  |  |  |  |  |  |  |
|---------------------------------------------------------------------------------------|--|--|--|--|--|--|--|---------------|--|--|--|--|--|--|--|--------------|--|--|--|--|--|--|--|--|--|--|--|--|--|
| 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 |  |  |  |  |  |  |  |               |  |  |  |  |  |  |  |              |  |  |  |  |  |  |  |  |  |  |  |  |  |
| Unused                                                                                |  |  |  |  |  |  |  | High Register |  |  |  |  |  |  |  | Low Register |  |  |  |  |  |  |  |  |  |  |  |  |  |
| bits 31–24                                                                            |  |  |  |  |  |  |  | bits 23–16    |  |  |  |  |  |  |  | bits 15–0    |  |  |  |  |  |  |  |  |  |  |  |  |  |

Figure 3.1: Address Register Pair Formation

The upper 8 bits (bits 31–24) of the high register are not used for addressing but remain available for storage, enabling techniques such as tagged pointers.

Although each address register pair has a specific name, and a suggested usage by convention, they are all treated the same by **STOR**/**LOAD** instructions and they can all be used for indirect addressing the same way.

In fact, you can perform a jump by simply loading a value in to the Program Counter registers.

However, it is worth noting that the Link Register pair will be updated automatically though specific instructions (**LJSR**, **BRSR**) and the Program Counter pair is incremented after every instruction.

### 3.3.2 Link Register (l)

| Component     | Register        |
|---------------|-----------------|
| High Register | lh (privileged) |
| Low Register  | ll              |

The link register pair stores return addresses for subroutine calls. When a subroutine is called using **LJSR** or **BRSR**, the return address is automatically saved to this register pair.

**Typical Use:**

- Storing return addresses from subroutine calls
- Alternate storage for general memory addressing

### 3.3.3 Address Register (a)

| Component     | Register        |
|---------------|-----------------|
| High Register | ah (privileged) |
| Low Register  | al              |

A general-purpose address register pair used for pointer operations and memory access.

**Typical Use:**

- Base pointer for data structures
- Array indexing
- General memory addressing

### 3.3.4 Stack Pointer (s)

| Component     | Register        |
|---------------|-----------------|
| High Register | sh (privileged) |
| Low Register  | sl              |

The stack pointer register pair points to the current top of the stack. Pre-decrement and post-increment addressing modes are particularly useful with this register.

**Typical Use:**

- Stack operations (push/pop)
- Local variable allocation
- Function parameter passing

### 3.3.5 Program Counter (p)

| Component     | Register        |
|---------------|-----------------|
| High Register | ph (privileged) |
| Low Register  | pl              |

The program counter register pair points to the next instruction to be executed. It is automatically incremented by 1 after each 16-bit word fetch. Since instructions are 32 bits (two words), the PC is incremented twice during a complete instruction fetch, advancing by 2 words total per instruction.

**Typical Use:**

- Automatically updated during normal execution
- Modified by branch and jump instructions
- Used for position-independent code

## 3.4 Status Register

The status register (**sr**) is a special 16-bit register that contains CPU status flags and control bits. It is divided into two bytes:

- **Lower Byte (bits 7–0):** Condition flags, accessible in both supervisor and protected modes
- **Upper Byte (bits 15–8):** Control and privileged state

In protected mode, reads of **sr** are redacted to the lower byte (upper byte reads as zero). Writes in protected mode only update the lower byte and preserve the upper byte.

The status register is discussed in detail in Chapter 4.

## 3.5 Privilege Restrictions

### 3.5.1 High Address Register Restrictions

The high component of each address register pair (**lh**, **ah**, **sh**, **ph**) is considered privileged:

- In **supervisor mode**: Can be read from and written to freely
- In **protected mode**:
  - Reading is allowed
  - Writing triggers a privilege exception

This restriction provides a form of memory segmentation/protection. User programs running in protected mode cannot modify the segment portion of addresses, preventing them from accessing memory outside their designated segment.

The status register differs slightly from the high address registers:

- High address registers (**lh**, **ah**, **sh**, **ph**) remain readable in protected mode, but writes trigger a privilege violation fault.
- The status register's privileged byte is hidden on reads in protected mode, and cannot be modified directly from protected mode writes.

## 3.6 Register Addressing

Registers are addressed using 4-bit register identifiers in instruction encoding:

| ID  | Register | Privileged      |
|-----|----------|-----------------|
| 0x0 | sr       | Upper byte only |
| 0x1 | r1       | No              |
| 0x2 | r2       | No              |
| 0x3 | r3       | No              |
| 0x4 | r4       | No              |
| 0x5 | r5       | No              |
| 0x6 | r6       | No              |
| 0x7 | r7       | No              |
| 0x8 | lh       | Yes             |
| 0x9 | ll       | No              |
| 0xA | ah       | Yes             |
| 0xB | al       | No              |
| 0xC | sh       | Yes             |
| 0xD | sl       | No              |
| 0xE | ph       | Yes             |
| 0xF | pl       | No              |

Table 3.1: Register Encoding

## 3.7 Usage Examples

### 3.7.1 Basic Register Operations

```

1 ; Load immediate value into r1
2 LOAD r1, #100
3
4 ; Add 2 to the value stored in r1 and store the result into r2
5 LOAD r2, r1
6 ADDI r2, #2
7
8 ; Add r1 and r2, store in r3
9 ADDR r3, r1, r2

```

### 3.7.2 Address Register Usage

```

1 ; Load value from memory at address in 'a' register pair
2 LOAD r1, (a)
3
4 ; Store r1 to memory at address (a + 8)
5 STOR (#8, a), r1
6
7 ; Push r1 onto stack (pre-decrement)

```

```
8 STOR -(#2, s), r1
9
10 ; Pop stack into r2 (post-increment)
11 LOAD r2, (s)+
```

### 3.7.3 Subroutine Call

```
1 ; Call subroutine at address in 'a' register
2 ; Return address saved to 'l' register automatically
3 LJSR (a)
4
5 ; ...subroutine code...
6
7 ; Return from subroutine
8 ; Copies 'l' register to 'p' register
9 RETS
```

## Chapter 4

# Status Register

### 4.1 Overview

The Status Register (**sr**) is a 16-bit special-purpose register that contains condition flags and control bits for the CPU. It is divided into two bytes with different privilege levels:

- **Lower Byte (bits 7–0):** Condition flags – readable and writable in both supervisor and protected modes
- **Upper Byte (bits 15–8):** Control and mode flags – accessible only in supervisor mode

### 4.2 Status Register Layout

|            |   |    |    |    |    |    |   |                |   |    |    |    |    |    |    |
|------------|---|----|----|----|----|----|---|----------------|---|----|----|----|----|----|----|
| 0          | 1 | 2  | 3  | 4  | 5  | 6  | 7 | 8              | 9 | 10 | 11 | 12 | 13 | 14 | 15 |
| T          | A | EA | E4 | E3 | E2 | E1 | P |                |   |    |    | V  | C  | N  | Z  |
| Privileged |   |    |    |    |    |    |   | Non-Privileged |   |    |    |    |    |    |    |

Figure 4.1: Status Register Bit Layout

### 4.3 Condition Flags (Non-Privileged)

These flags reflect the results of ALU operations and comparisons. They can be read and written by code running in any privilege level.

#### 4.3.1 Zero Flag (Z) – Bit 0

**Set when:** The result of an operation is zero

**Cleared when:** The result of an operation is non-zero

**Tested by:** Equal (==), Not Equal (!=) condition codes

### 4.3.2 Negative Flag (N) – Bit 1

**Set when:** The result of an operation is negative (bit 15 = 1) when interpreted as a signed two's complement value

**Cleared when:** The result is positive or zero (bit 15 = 0)

**Tested by:** Negative Set (NS), Negative Clear (NC), and signed comparison condition codes

### 4.3.3 Carry Flag (C) – Bit 2

**Set when:** An addition produces a carry out of bit 15, or a subtraction requires a borrow

**Cleared when:** No carry/borrow occurs

**Tested by:** Carry Set (CS), Carry Clear (CC), Unsigned Higher (HI), Unsigned Lower or Same (LO) condition codes

**Note:** For unsigned comparisons, the carry flag acts as an unsigned overflow/borrow indicator

### 4.3.4 Overflow Flag (V) – Bit 3

**Set when:** A signed arithmetic operation produces a result that cannot be represented in 16 bits (signed overflow)

**Cleared when:** No signed overflow occurs

**Tested by:** Overflow Set (OS), Overflow Clear (OC), and signed comparison condition codes

**Note:** Overflow occurs when adding two positive numbers yields a negative result, or adding two negative numbers yields a positive result

### 4.3.5 Reserved Flags – Bits 4–7

Bits 4 through 7 of the lower byte are reserved for future use. They should not be relied upon by software.

## 4.4 Control Flags (Privileged)

These flags control CPU behavior and can only be modified in supervisor mode. Attempts to write these bits in protected mode will be ignored (the write affects only the lower byte).

### 4.4.1 Protected Mode (P) – Bit 8

**When Clear (0):** CPU operates in supervisor mode with full privileges

**When Set (1):** CPU operates in protected mode with restricted access to:

- High bytes of address register pairs (lh, ah, sh, ph)
- Upper byte of status register
- Privileged coprocessor operations (opcodes > 0x07xx)

**Note:** This bit provides basic memory protection by preventing user code from changing memory segments. With the exception of writing to the status register, performing a restricted operation will cause a "privilege violation" fault.

#### 4.4.2 Hardware Interrupt Enable Bits (E1-E4) – Bits 9–12

These four bits individually control whether each of the four maskable hardware interrupt lines is enabled:

- **E1 (Bit 9):** Enable Hardware Interrupt Line 1 (Level 1, lowest maskable priority)
- **E2 (Bit 10):** Enable Hardware Interrupt Line 2 (Level 2)
- **E3 (Bit 11):** Enable Hardware Interrupt Line 3 (Level 3)
- **E4 (Bit 12):** Enable Hardware Interrupt Line 4 (Level 4, highest maskable priority)

**Note:** Hardware Interrupt Line 5 (Level 5) is a Non-Maskable Interrupt (NMI) and is always enabled. It cannot be disabled via the status register.

When a bit is set to 1, the corresponding hardware interrupt is enabled and can trigger an exception when asserted. When cleared to 0, interrupts on that line are disabled and will not be serviced.

**Important behavior:**

- Disabled interrupts are not queued – they are completely ignored while the enable bit is 0
- Hardware interrupts can only interrupt execution if their priority level is higher than the currently executing exception level
- This prevents the same interrupt from re-triggering while its handler is executing
- Lower priority interrupts that occur while handling a higher priority interrupt are queued and serviced after the current handler completes (after RETE)

These bits are typically managed by the operating system's interrupt handling code. At reset, all interrupt enable bits are cleared (interrupts disabled).

Interrupt handling is discussed in detail in Chapter 5.

#### 4.4.3 Exception Active (EA) – Bit 13

**When Set (1):** The CPU is currently executing an exception handler (`current_exception_level != 0`)

**When Clear (0):** The CPU is executing normal code (`current_exception_level == 0`)

**Automatically Set:** When any exception is entered (software, hardware, or fault)

**Automatically Cleared:** When returning to exception level 0 via **RETE**

**Purpose:** Allows software to detect whether it's running within an exception handler context

**Typical Use:** OS code can check this bit to determine appropriate behavior (e.g., avoid re-enabling interrupts during nested exception handling)

**Note:** This bit reflects the internal `current_exception_level` state. Programs can read this bit but cannot directly modify it – it is managed automatically by the exception handling hardware.

#### 4.4.4 Trap on Address Overflow (A) – Bit 14

**When Set (1):** Memory address calculations that overflow or underflow will trigger an address overflow exception

**When Clear (0):** Address overflows wrap around without any side effects

**Purpose:** Helps detect pointer errors during development

**Typical Use:** Enabled during debugging, disabled in production for performance

#### 4.4.5 Trace Mode (T) – Bit 15

**When Set (1):** An exception is generated after every instruction

**When Clear (0):** Normal execution

**Purpose:** Allows debuggers to single-step through code

**Note:** Used by SIRCIT (SIRC Instruction Tracer) for debugging

### 4.5 Status Register Updates

#### 4.5.1 Automatic Updates

The condition flags (Z, N, C, V) are automatically updated by most ALU instructions, unless explicitly disabled:

- **Arithmetic Instructions** (ADDI, SUBI, ADCI, SBCI): Update all four condition flags
- **Logical Instructions** (ANDI, ORRI, XORI): Update Z and N flags; C and V flags are cleared
- **Compare Instructions** (CMPI, CMPR): Same as Arithmetic Instructions but does not update destination
- **Test Instructions** (TSAI, TSAR, TSXI, TSXR): Same as Logical Instructions but does not update destination
- **Load Instructions** (LOAD, LDEA): Does not update condition flags
- **Shift Operations:** Can optionally update flags based on the instruction's status register update source field

#### 4.5.2 Manual Updates

The status register can be directly manipulated like any other register:

```
1 ; Set protected mode bit (only in supervisor mode)
2 ORRI sr, #0x0100
3
4 ; Enable all maskable hardware interrupts (set bits 9-12)
5 ; Note: Hardware interrupt line 5 is an NMI and is always enabled
6 ORRI sr, #0x1E00
7
```

```

8 ; Disable all maskable hardware interrupts (clear bits 9-12)
9 ANDI sr, #0xE1FF
10
11 ; Restore status register saved to stack
12 LOAD sr, (s)+

```

**Important:** When running in protected mode, writes to the status register only affect the lower byte (bits 7–0). The upper byte remains unchanged.

## 4.6 Reading the Status Register

In supervisor mode, reading `sr` returns the full 16-bit value. In protected mode, reading `sr` returns the lower byte in bits 7–0, with bits 15–8 masked to zero (for security).

```

1 ; In supervisor mode - gets full 16 bits
2 LOAD r1, sr ; r1 = 0xFFFF (all bits)
3
4 ; In protected mode - upper byte masked
5 LOAD r1, sr ; r1 = 0x00XX (upper byte forced to 0)

```

## 4.7 Exception Handling Effects

When an exception occurs:

1. The current status register is saved
2. Protected mode bit (P) is cleared (entering supervisor mode)
3. Trace mode bit (T) is cleared (disabling single-stepping in exception handler)
4. Interrupts may be masked depending on exception type

When returning from an exception with `RETE`, the saved status register is restored.

## 4.8 Usage Examples

### 4.8.1 Conditional Execution

```

1 ; Compare r1 with r2
2 CMPI r1, r2
3
4 ; Branch if equal
5 BRAN |== @label_equal
6
7 ; Branch if r1 > r2 (unsigned)
8 BRAN |HI @label_greater
9
10 ; Add 0x16 to r1 if r1 >= r2 (signed)
11 ADDI |>= r1, #0x16

```

### 4.8.2 Manual Status Flag Control

The SIRC-1 provides syntax to manually control how status flags are updated during ALU and shift operations. This is particularly useful for advanced control flow and flag preservation. See Section 8.12 in Chapter 8 for complete details.

```
1 ; Update flags from shift instead of ALU
2 ADDI [S] r2, #1, LSL #2 ; Flags from shift, not addition
3
4 ; Preserve flags across operations
5 CMPI r1, r2 ; Set comparison flags
6 ADDI [N] r3, #1 ; Increment without changing flags
7 BRAN |= still_uses_cmpi_flags ; Branch uses CMPI flags
```

### 4.8.3 Testing Bits

```
1 ; Test if bit 5 is set in r1
2 TSAI r1, #0x0020 ; AND with 0x0020, update flags only
3
4 ; Branch if bit was set (result non-zero)
5 BRAN != bit_was_set
```

### 4.8.4 Enabling Protected Mode

```
1 ; Entering user mode (from supervisor mode)
2 ORRI sr, #0x0100 ; Set protected mode bit
3 ; Now running in protected mode
4
5 ; Attempting to write privileged register will fault
6 ; ADDI ah, #0x05 ; This would cause exception!
```

### 4.8.5 Saving and Restoring Status

```
1 ; Save status register
2 LOAD r7, sr
3
4 ; Do some operations that change flags
5 ADDI r1, r2, r3
6 CMPI r1, #100
7
8 ; Restore original status
9 LOAD sr, r7
```

## Chapter 5

# Exception and Interrupt Handling

### 5.1 Overview

An exception is an event that causes the program flow to be diverted to an address stored in a predefined vector.

Some are directly caused by an instruction, some are a side effect of an instruction and some are asynchronously caused by external hardware.

They are managed by the exception coprocessor, which will take control of the processor when there is a pending exception.

### 5.2 Exception Types

#### 5.2.1 Faults

Faults are priority 7 exceptions raised by the CPU itself in response to error conditions or debug events. They use vectors in the 0x01–0x0F range. Unlike hardware and software exceptions, faults cannot be masked or deferred: when a fault condition is detected, the fault is dispatched at the next instruction boundary regardless of the current exception level.

The T bit (TraceMode) in the status register is always cleared when entering any exception handler, including fault handlers. This ensures that trace faults do not fire inside handlers. RETE restores the saved status register, so if T was set before the exception, tracing resumes after RETE returns.

#### Fault Categories

Not all faults behave the same way with respect to the faulting instruction and the RETE return address. There are three categories:

**Retryable faults** The instruction that triggered the fault is cancelled before it takes effect. The link register stores the address of the faulting instruction. RETE returns to that instruction so it can be retried (possibly after the fault handler has corrected the conditions that caused the fault, e.g. mapping a page in response to a bus fault).

*Bus Fault, Bus Protection Fault, Alignment Fault, Segment Overflow Fault, Invalid Opcode Fault, Privilege Violation Fault.*

**Post-instruction faults** The instruction completes normally and its effects are committed before the fault fires. The link register stores the address of the *next* instruction. RETE advances to that instruction; the traced instruction is not re-executed.

*Instruction Trace Fault.*

**Exception dispatch faults** Raised during the exception handling machinery itself, not during normal instruction execution. The link register stores the program counter at the point the fault was detected within the exception handler. Recovery may require careful state management; these faults often indicate a hardware misconfiguration or a programming error in an exception handler.

*Level Five Hardware Exception Conflict, Double Fault.*

### Fault Descriptions

- **Bus Fault** (*Retryable*) — Raised when an external device asserts the bus error CPU pin.  
This could happen, for example, if an unmapped address is presented by the CPU and another chip detects this and raises an error. It could also be used to implement virtual memory: the instruction is aborted and the program counter is not incremented, so the address can be re-calculated and the operation retried.
- **Bus Protection Fault** (*Retryable*) — Raised when an external device asserts the bus protection error CPU pin.  
This indicates that the requested address and device are valid, but the operation is disallowed. Examples include writing to ROM or accessing a protected memory region while the CPU is in protected mode.
- **Alignment Fault** (*Retryable*) — Raised when fetching instructions if the program counter contains an odd word address.  
This is to simplify fetching: the second word of an instruction is always at `p1 | 0x1`, which ensures that instructions can never overflow a segment boundary.
- **Segment Overflow Fault** (*Retryable*) — Raised when a computed address would fall outside the current segment.  
Only raised if the `TrapOnAddressOverflow` SR bit is set, since some programs intentionally rely on address wrap-around. Also raised if the program counter overflows; the phase field of the fault metadata register distinguishes the two cases.
- **Invalid Opcode Fault** (*Retryable*) — Raised when a co-processor call targets a non-existent co-processor or uses an invalid opcode.  
Can be used for forward compatibility: programs written for a future co-processor (e.g. a floating-point unit) will trap on older hardware, and the handler can emulate the operation in software.
- **Privilege Violation Fault** (*Retryable*) — Raised when a privileged operation is attempted while not in supervisor mode:
  1. Writing to the high word of any address register
  2. Writing to the high byte of the status register

### 3. Triggering an exception with a vector below 0x80

- **Instruction Trace Fault** (*Post-instruction*) — Raised after every instruction completes when the **TraceMode** SR bit is set.

Used for single-step debugging. Because the fault fires after the instruction commits, RETE returns to the instruction *following* the traced one. The T bit is cleared on entry to the fault handler (as with all exceptions), so the handler itself is not traced. RETE restores the saved SR, which re-enables tracing unless the handler explicitly clears T from the saved status register via ETTR before returning.

- **Level Five Hardware Exception Conflict** (*Exception dispatch*) — Raised when a level five hardware exception arrives while one is already being serviced.

The windowed link register scheme has no spare register above level 6, so a second L5 cannot be handled normally. Rather than silently dropping it (which could mask a hardware misconfiguration), the CPU raises this fault. Level 5 interrupts should be treated as NMIs.

- **Double Fault** (*Exception dispatch*) — Raised when a fault occurs while another fault is already being handled.

When the CPU is already at fault level (priority 7) and another fault occurs, there is no additional banked link register available. The fault link register (register 6) is overwritten with the new return address, losing the original fault's return address. The fault metadata register (register 7) is also overwritten; it encodes a double-fault flag along with both the current and original fault types.

The CPU jumps to vector 0x08. This is typically a last-chance handler that dumps registers and resets. If a fault occurs while handling a double fault, another double fault is raised, continuously overwriting the link registers.

## 5.2.2 Hardware Exceptions (Interrupts)

A hardware exception occurs when interrupt pins are signalled by external hardware. These are also referred to as "interrupts". When a hardware exception occurs the current instruction is completed, and control is transferred to different vectors based on the interrupt "priority". The vectors for hardware exceptions are in the 0x10-0x50 range.

## 5.2.3 User Exceptions (Traps)

A user exception occurs when the program executes an EXCP instruction that signals the exception coprocessor that it should trigger an exception. They are usually used by programs running in protected mode to switch to supervisor mode in a controlled way. They are also referred to as "traps". The vectors for user exceptions are in the 0x60-0xFF range.

## 5.3 Exception Registers

The SIRC-1 exception coprocessor maintains several special-purpose registers for exception handling.

### 5.3.1 Link Registers

The exception unit uses banked link registers to store the processor state when an exception occurs. There are eight link registers in total, indexed 0-7:

- Register 0: Software exceptions (traps)
- Registers 1-5: Hardware exceptions (interrupts), one for each priority level
- Register 6: Faults (abort exceptions)
- Register 7: Fault metadata (stores bus address and execution phase information, see section 5.3.1 for the format)

Each link register stores two pieces of information:

- **Return Address:** The 32-bit address to return to when the exception handler completes
- **Return Status Register:** The 16-bit status register value at the time of the exception

When an exception occurs, the appropriate link register (determined by the exception priority level) is populated with the current program counter and status register values. For abort exceptions (faults), the return address is the address of the faulting instruction so it can be retried. For regular exceptions (hardware and software exceptions), the return address is the address after the current instruction.

#### Link Register Preservation

**Important:** Link registers can be overwritten when a fault occurs during the handling of another fault (a double fault). In this case, the fault link register (register 6) and the fault metadata register (register 7) is overwritten. This means the original return address is lost.

A valid way of handling this is just to declare the CPU state as invalid, do some last chance error handling (e.g. dumping the registers) and reset.

For robust fault handling that can recover from double faults, fault handlers should save the link register contents early in the handler:

```

1 :fault_handler
2     ; Immediately save link register to memory before doing anything
3     ; that might trigger another fault
4     ETFR #6                                ; Save level 6 exception registers to a
        and r7
5     STOR (s)-, r7
6     STOR (s)-, ah
7     STOR (s)-, al
8     ETFR #7                                ; Save level 7 (metadata) exception
        registers to a and r7
9     STOR (s)-, r7
10    STOR (s)-, ah
11    STOR (s)-, al
12
13    ; Now safe to perform operations that might fault
14    ; ...
15
```

```

16      ; Restore and return
17      LOAD al, +(s),
18      LOAD ah, +(s)
19      LOAD r7, +(s)
20      ETTR #7                                ; Restore level 6 exception registers from
      a and r7
21      LOAD al, +(s)
22      LOAD ah, +(s)
23      LOAD r7, +(s)
24      ETTR #6                                ; Restore level 7 (metadata) exception
      registers from a and r7
25
26      ; Now save to use RETE
27      RETE

```

However, if the stack pointer is invalid in this case, the double fault will still cause the CPU state to be corrupted. You could provide an even more robust implementation by storing/loading from hardcoded memory addresses that are known to always be valid.

### Fault Metadata Register Format

The fault metadata register (register 7) stores additional information about faults beyond just the return address. The `return_status_register` field is used to encode fault-specific metadata in the following format:

- **Bits 0-2:** Execution phase when the fault occurred (3 bits)
  - 0x0: Instruction fetch
  - 0x3: Effective address calculation
  - Other values indicate other pipeline phases
- **Bit 3:** Double fault flag (1 bit) - set to 1 if this is a double fault
- **Bits 4-7:** Current fault type (4 bits) - the type of fault that occurred
- **Bits 8-11:** Original fault type (4 bits) - only meaningful for double faults, indicates the fault type that was being handled when the double fault occurred
- **Bits 12-15:** Unused (reserved for future use)

Fault types use the following values:

- **0x01:** Bus fault
- **0x02:** Alignment fault
- **0x03:** Segment overflow fault
- **0x04:** Invalid opcode fault
- **0x05:** Privilege violation fault
- **0x06:** Instruction trace fault

- **0x07:** Level five hardware exception conflict
- **0x08:** Double fault
- **0x09:** Bus protection fault

The `return_address` field of register 7 stores the bus address that was being accessed when the fault occurred (for faults that involve memory access). For faults like invalid opcode that occur before bus access, this field will be zero.

Exception handlers can use the **ETFR** instruction to read the fault metadata register and decode this information to determine how to handle the fault appropriately.

### 5.3.2 Cause Register

The cause register is a 16-bit internal register used to communicate between different CPU units and coordinate exception handling. It has the following structure:

- **Bits 15-12 (First Nibble):** Coprocessor ID (0x1 for exception unit, 0x0 for processing unit)
- **Bits 11-8 (Second Nibble):** Coprocessor opcode (the operation to execute)
- **Bits 7-0 (Third and Fourth Nibbles):** Vector or parameter value

The cause register is automatically populated by the exception unit when a pending exception needs to be serviced. The priority of exceptions is determined by examining the first nibble of the vector ID, calculated as 7 minus the value of the first nibble (e.g., 0x00 is priority 7, 0x40 is priority 3, 0x60 and above are all priority 1).

### 5.3.3 Hardware Interrupt Enable Bits

The status register contains 4 individual enable bits (bits 9-12 in the privileged byte) that control whether each maskable hardware interrupt line can trigger an exception:

- Bit 9 (E1): Hardware Interrupt Line 1 (Level 1, lowest maskable priority)
- Bit 10 (E2): Hardware Interrupt Line 2 (Level 2)
- Bit 11 (E3): Hardware Interrupt Line 3 (Level 3)
- Bit 12 (E4): Hardware Interrupt Line 4 (Level 4, highest maskable priority)

**Note:** Hardware Interrupt Line 5 (Level 5) is a Non-Maskable Interrupt (NMI) and cannot be disabled. It will always trigger an exception when asserted, regardless of the enable bit settings. Bit 13 is used for the Exception Active flag instead.

When an interrupt line is disabled (bit = 0), interrupts on that line are completely ignored and not queued. When enabled (bit = 1), the interrupt can trigger an exception if its priority is higher than the currently executing exception level.

#### **Hierarchical Exception Handling:**

Exceptions in SIRC-1 are strictly hierarchical. A hardware interrupt can only interrupt the current execution if its priority level (exception level) is *higher* than the currently active exception level. This prevents:

- An interrupt handler from being interrupted by its own interrupt (same level)
- Lower priority interrupts from interrupting higher priority handlers
- Software exceptions from running during any hardware interrupt (would corrupt link register 0)

The exception unit maintains a `current_exception_level` field that tracks the active exception level (0-7). When not handling any exception, this is 0. When handling an exception at level N, only exceptions at level > N can interrupt.

**Level 5 NMI-like Behavior:**

Level 5 hardware exceptions behave similarly to a Non-Maskable Interrupt (NMI). They cannot be disabled via the interrupt enable bits in the status register, and will preempt any lower-priority exception handler. However, Level 5 is *not* fully non-maskable. Because faults carry priority 7 (higher than Level 5's priority 6), a L5 interrupt that arrives while a fault is being handled is *queued* in the pending hardware exceptions register rather than serviced immediately.

Once the fault handler returns via **RETE** and the exception level drops back below 6, the pending L5 interrupt fires normally.

This means Level 5 response latency is not deterministic when a fault is in progress. For timing-critical use of the NMI line, fault handlers should be kept as short as possible.

Keep in mind that if a second L5 interrupt arrives *while the CPU is already executing an L5 handler* (exception level 6), it is not queued. It will raise a Level Five Hardware Exception Conflict fault instead. See the fault descriptions above.

These two rules compose in a non-obvious way: if a fault is raised *inside* an L5 handler and a second L5 interrupt arrives while that fault is being serviced, the interrupt is queued (priority 7 blocks priority 6). Once the fault handler returns via **RETE** and the exception level drops back to 6, the queued L5 fires — and since the CPU is already at level 6 at that point, a Level Five Hardware Exception Conflict fault is raised then, not during the inner fault handler.

## 5.4 Exception Processing

### 5.4.1 Exception Priority

Exceptions are prioritized as follows (highest to lowest):

1. Priority 7: Faults (abort exceptions)
2. Priority 6: Level 5 hardware exceptions (NMI-like)
3. Priority 5: Level 4 hardware exceptions
4. Priority 4: Level 3 hardware exceptions
5. Priority 3: Level 2 hardware exceptions
6. Priority 2: Level 1 hardware exceptions
7. Priority 1: Software exceptions (traps)

### 5.4.2 Exception Handling Flow

When an exception occurs, the following steps are executed:

1. For hardware exceptions, the exception unit checks if the interrupt line is enabled in the status register. If disabled, the interrupt is ignored and not queued.
2. The exception unit checks if the exception priority level is higher than the current active exception level. If not, the exception is deferred until the current handler completes.
3. For level 5 hardware exceptions being raised when one is already being handled (both have exception level 6), a Level Five Hardware Exception Conflict fault is raised instead.
4. The current program counter, status register, and current exception level are stored in the appropriate link register based on the exception priority level.
5. The Protected Mode bit in the status register is cleared, switching the CPU to supervisor mode.
6. The current exception level is updated to the level of the exception being serviced.
7. The vector address is computed by multiplying the vector ID by 2 (since vectors are 32-bit addresses stored as two 16-bit words).
8. The program counter is set to the vector address, transferring control to the exception handler.

#### Returning from Exceptions:

The RETE (Return from Exception) instruction restores the saved state from the link register:

1. The return address is loaded from the link register into the program counter
2. The saved status register value is restored (including interrupt enable bits)
3. The saved exception level is restored to `current_exception_level`
4. Execution resumes at the return address

After RETE, any pending interrupts that are now eligible to run (higher priority than the restored exception level and enabled) will be serviced before the next instruction executes.

### 5.4.3 Vector Table

The exception vectors are stored starting at address 0x0. Each vector is a 32-bit address stored as two consecutive 16-bit words. The vector table layout is:

- **0x00:** Reset vector
- **0x01:** Bus fault
- **0x02:** Alignment fault
- **0x03:** Segment overflow fault
- **0x04:** Invalid opcode fault

- **0x05:** Privilege violation fault
- **0x06:** Instruction trace fault
- **0x07:** Level five hardware exception conflict
- **0x08:** Double fault
- **0x09:** Bus protection fault
- **0x0A-0x0F:** Reserved for future privileged exceptions
- **0x10:** Level 5 hardware exception vector
- **0x20:** Level 4 hardware exception vector
- **0x30:** Level 3 hardware exception vector
- **0x40:** Level 2 hardware exception vector
- **0x50:** Level 1 hardware exception vector
- **0x60-0xFF:** User exception vectors (128 user-accessible trap vectors)

## 5.5 Return from Exception

To return from an exception handler, the **RETE** (Return from Exception) instruction is used. This instruction performs the following operations:

1. Reads the link register corresponding to the current interrupt mask level (minus 1, since the mask is set to the exception level)
2. Restores the status register from the link register (including the interrupt mask and protected mode bit)
3. Restores the program counter from the link register
4. Execution continues from the restored address

For abort exceptions (faults), **RETE** returns to the faulting instruction, allowing it to be retried. For regular exceptions, **RETE** returns to the instruction following the one that was executing when the exception occurred.

### 5.5.1 Accessing Link Registers

The exception coprocessor provides instructions to transfer data to and from the link registers:

- **ETFR (Exception Transfer From Register):** Copies the return address and/or status register from a link register to the address register and/or R7
- **ETTR (Exception Transfer To Register):** Copies the address register and/or R7 to a link register's return address and/or status register

These instructions allow exception handlers to examine or modify the saved processor state before returning, enabling features like system calls, context switching, and debuggers.

## 5.6 Exception Coprocessor Instructions

The exception coprocessor (coprocessor ID 0x1) provides several specialized instructions for exception handling and system control.

| Instruction | Opcode | Privilege  | Purpose                                          |
|-------------|--------|------------|--------------------------------------------------|
| <b>EXCP</b> | 0x1    | User       | Trigger a software exception (trap)              |
| <b>WAIT</b> | 0x9    | Supervisor | Enter wait state until an exception occurs       |
| <b>RETE</b> | 0xA    | Supervisor | Return from an exception handler                 |
| <b>RSET</b> | 0xB    | Supervisor | Reset CPU state and jump to reset vector         |
| <b>ETFR</b> | 0xC    | Supervisor | Copy saved exception state from a link register  |
| <b>ETTR</b> | 0xD    | Supervisor | Write CPU register state back to a link register |

Table 5.1: Exception Coprocessor Instruction Summary

For full syntax, operation details, and worked examples, see Chapter 16.

### 5.6.1 Internal Instructions

The following opcodes are used internally by the exception unit and are not directly accessible via user instructions:

- **Fault (0xE)**: Automatically invoked when a fault condition is detected
- **HardwareException (0xF)**: Automatically invoked when a hardware interrupt is signaled
- **None (0x0)**: No operation, used when no exception is pending

These internal operations follow the standard exception handling flow described in the Exception Processing section.

# **Part II**

## **Instruction Set Architecture**



## Chapter 6

# Instruction Formats

### 6.1 Overview

All SIRCIS instructions are exactly 32 bits (4 bytes, or 2 words) in length. This fixed instruction size simplifies instruction fetch and decode logic, though it may result in larger code size compared to variable-length instruction sets.

The SIRC-1 supports three distinct instruction formats:

1. Immediate
2. Short Immediate (with Shift)
3. Register

Note: operandless assembler meta-instructions such as **NOOP** still assemble to one of these three CPU instruction formats.

Each format is optimized for different types of operations and addressing modes.

### 6.2 Format Overview

| Format          | Typical Use                                           |
|-----------------|-------------------------------------------------------|
| Immediate       | Operations with a 16-bit constant value               |
| Short Immediate | Operations with an 8-bit constant with optional shift |
| Register        | Operations on multiple registers with optional shift  |

Table 6.1: Instruction Format Summary

### 6.3 Common Fields

All instruction formats share some common fields:

**Opcode (6 bits)** Identifies the operation to perform. With 6 bits, up to 64 distinct operations can be encoded.

**Condition Flags (4 bits)** Specifies under what conditions the instruction should execute. See **Chapter 9** for details.

**Additional Flags (2 bits)** Used in memory operations to specify address register pairs (a, p, s, l) and in ALU operations to specify how the status register is updated.

## 6.4 Immediate Format

### 6.4.1 Structure

The immediate format encodes a full 16-bit immediate value within the instruction.

Note: Due to the large 16-bit operand, shift is not supported in this format.

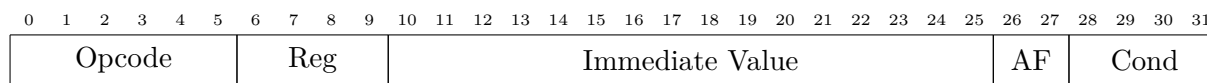


Figure 6.1: Immediate Instruction Format

**Opcode (bits 31–26)** 6-bit operation code

**Register (bits 25–22)** 4-bit destination/source register identifier

**Immediate (bits 21–6)** 16-bit signed or unsigned immediate value

**Additional Flags (bits 5–4)** 2-bit address register pair selector (for memory operations)

- 00 = l (link register)
- 01 = a (address register)
- 10 = s (stack pointer)
- 11 = p (program counter)

or 2-bit status register update source selector (for ALU operations)

- 00 = Status register not updated
- 01 = Status register updated from ALU operation
- 10 = Status register updated from shift operation
- 11 = Reserved for future use

**Condition (bits 3–0)** 4-bit condition code

### 6.4.2 Encoding Examples

| Instruction       | Opcode           | Reg           | Immediate                        | AF           | Cond          | Hex        |
|-------------------|------------------|---------------|----------------------------------|--------------|---------------|------------|
| ADDI r1, #100     | 0x00<br>(000000) | 0x1<br>(0001) | 0x0064<br>(0000 0000 0110 0100)  | 0b01<br>(01) | 0x0<br>(0000) | 0x00401910 |
| CMPI r2, #0x8000  | 0x0A<br>(001010) | 0x2<br>(0010) | 0x8000<br>(1000 0000 0000 0000)  | 0b01<br>(01) | 0x0<br>(0000) | 0x28A00010 |
| LOAD r3, (#16, a) | 0x14<br>(010100) | 0x3<br>(0011) | 0x0010<br>(0000 0000 0001 0000)  | 0b01<br>(01) | 0x0<br>(0000) | 0x50C00410 |
| STOR (#-2, s), r4 | 0x10<br>(010000) | 0x4<br>(0100) | 0xFFFFE<br>(1111 1111 1111 1110) | 0b10<br>(10) | 0x0<br>(0000) | 0x413FFFA0 |
| ADDI == r5, #42   | 0x00<br>(000000) | 0x5<br>(0101) | 0x002A<br>(0000 0000 0010 1010)  | 0b01<br>(01) | 0x1<br>(0001) | 0x01400A91 |
| SUBI >= r6, #10   | 0x02<br>(000010) | 0x6<br>(0110) | 0x000A<br>(0000 0000 0000 1010)  | 0b01<br>(01) | 0xB<br>(1011) | 0x0980029B |

Table 6.2: Immediate Format Encoding Examples

#### Notes:

- Condition code examples: 0x0 (0000) = Always, 0x1 (0001) = Equal (==), 0xB (1011) = Signed >=
- AF field for memory ops: 00=l, 01=a, 10=s, 11=p; for ALU ops: 00=no update, 01=update from ALU, 10=update from shift
- Negative immediates use two's complement (#-2 = 0xFFFFE)

## 6.5 Short Immediate Format (with Shift)

### 6.5.1 Structure

The short immediate format sacrifices immediate value width to include shift information, allowing for more powerful single-instruction operations.

Only supported for ALU instructions - cannot be used with any of the branching, or load/store instructions.

|    |   |   |   |   |   |     |   |   |   |     |    |    |    |    |    |    |    |    |    |    |    |    |      |    |    |    |    |    |    |    |    |
|----|---|---|---|---|---|-----|---|---|---|-----|----|----|----|----|----|----|----|----|----|----|----|----|------|----|----|----|----|----|----|----|----|
| 0  | 1 | 2 | 3 | 4 | 5 | 6   | 7 | 8 | 9 | 10  | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 | 21 | 22 | 23   | 24 | 25 | 26 | 27 | 28 | 29 | 30 | 31 |
| Op |   |   |   |   |   | Reg |   |   |   | Imm |    |    |    |    |    | SO | ST | SA |    |    |    | AF | Cond |    |    |    |    |    |    |    |    |

Figure 6.2: Short Immediate with Shift Format

**Opcode (bits 31–26)** 6-bit operation code

**Register (bits 25–22)** 4-bit destination/source register identifier

**Immediate (bits 21–14)** 8-bit signed or unsigned immediate value

**Shift Operand (bit 13)** Determines how the shift amount field is interpreted

- 0 = Shift amount field contains the literal shift amount
- 1 = Shift amount field refers to register that contains shift amount

**Shift Type (bits 12–10)** Type of shift operation (see Table 6.6 for encoding details)

**Shift Amount (bits 9–6)** Shift amount (or register containing shift amount)

**Additional Flags (bits 5–4)** 2-bit status register update source selector

- 00 = Status register not updated
- 01 = Status register updated from ALU operation
- 10 = Status register updated from shift operation
- 11 = Reserved for future use

**Condition (bits 3–0)** 4-bit condition code

### 6.5.2 Encoding Examples

| Instruction             | Op               | Reg           | Imm                | SO       | ST           | SA            | AF           | Cond          | Hex        |
|-------------------------|------------------|---------------|--------------------|----------|--------------|---------------|--------------|---------------|------------|
| ADDI r1, #2, LSL #3     | 0x20<br>(100000) | 0x1<br>(0001) | 0x02<br>(00000010) | 0<br>(0) | 001<br>(001) | 0x3<br>(0011) | 0b01<br>(01) | 0x0<br>(0000) | 0x804084D0 |
| ORRI r2, #1, LSL #10    | 0x25<br>(100101) | 0x2<br>(0010) | 0x01<br>(00000001) | 0<br>(0) | 001<br>(001) | 0xA<br>(1010) | 0b01<br>(01) | 0x0<br>(0000) | 0x94804690 |
| ANDI r3, #7, ASR #2     | 0x24<br>(100100) | 0x3<br>(0011) | 0x07<br>(00000111) | 0<br>(0) | 100<br>(100) | 0x2<br>(0010) | 0b01<br>(01) | 0x0<br>(0000) | 0x90C1D090 |
| SUBI r4, #5, LSL r2     | 0x22<br>(100010) | 0x4<br>(0100) | 0x05<br>(00000101) | 1<br>(1) | 001<br>(001) | 0x2<br>(0010) | 0b01<br>(01) | 0x0<br>(0000) | 0x89016490 |
| XORI != r7, #15, LSR #4 | 0x26<br>(100110) | 0x7<br>(0111) | 0x0F<br>(00001111) | 0<br>(0) | 010<br>(010) | 0x4<br>(0100) | 0b01<br>(01) | 0x2<br>(0010) | 0x99C3C912 |
| CMPI NS r1, #8, RTL r3  | 0x2A<br>(101010) | 0x1<br>(0001) | 0x08<br>(00001000) | 1<br>(1) | 101<br>(101) | 0x3<br>(0011) | 0b01<br>(01) | 0x5<br>(0101) | 0xA84234D5 |
| SHFT r3, RTR #5         | 0x25<br>(100101) | 0x3<br>(0011) | 0x00<br>(00000000) | 0<br>(0) | 110<br>(110) | 0x5<br>(0101) | 0b10<br>(10) | 0x0<br>(0000) | 0x94C01960 |

Table 6.3: Short Immediate Format Encoding Examples

#### Notes:

- SO (Shift Operand): 0 = literal shift amount, 1 = register contains shift amount (see rows 4 and 6)
- ST (Shift Type): 000=none, 001=LSL, 010=LSR, 011=ASL, 100=ASR, 101=ROL, 110=ROR
- AF field for ALU: 00=no update, 01=update from ALU, 10=update from shift (see SHFT row 7)
- Condition code examples: 0x0 = Always, 0x2 = Not Equal (!=), 0x5 = Negative (<0)
- SHFT is a meta instruction using ORRI opcode (0x25) with AF=0b10

## 6.6 Register Format

### 6.6.1 Structure

The register format allows operations on up to three registers with optional shift.

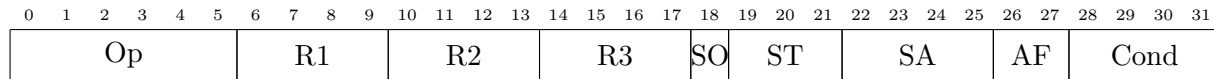


Figure 6.3: Register Instruction Format

**Opcode (bits 31–26)** 6-bit operation code

**Register 1 (bits 25–22)** 4-bit destination register

**Register 2 (bits 21–18)** 4-bit source A register

**Register 3 (bits 17–14)** 4-bit source B register

**Shift Operand (bit 13)** Determines how the shift amount field is interpreted

- 0 = Shift amount field contains the literal shift amount
- 1 = Shift amount field refers to register that contains shift amount

**Shift Type (bits 12–10)** Type of shift operation (see Table 6.6 for encoding details)

**Shift Amount (bits 9–6)** Shift amount (or register containing shift amount)

**Additional Flags (bits 5–4)** 2-bit address register pair selector (for memory operations)

- 00 = l (link register)
- 01 = a (address register)
- 10 = s (stack pointer)
- 11 = p (program counter)

or 2-bit status register update source selector (for ALU operations)

- 00 = Status register not updated
- 01 = Status register updated from ALU operation
- 10 = Status register updated from shift operation
- 11 = Reserved for future use

**Condition (bits 3–0)** Condition code

### 6.6.2 Register Operand Semantics

For most three-operand instructions:

- R1 = Destination
- R2 = First source operand

- R3 = Second source operand (optional)

Note: If R3 is not specified, the assembler will use R1 as both destination and first source operand when encoding the instruction:

```
1 ADDR r1, r2, r3 ; r1 = r2 + r3
2 ADDR r1, r2 ; r1 = r1 + r2 (R3 implicitly = R1)
```

### 6.6.3 Encoding Examples

| Instruction                | Op               | R1            | R2            | R3            | SO       | ST           | SA            | AF           | Cond          | Hex        |
|----------------------------|------------------|---------------|---------------|---------------|----------|--------------|---------------|--------------|---------------|------------|
| ADDR r1, r2, r3            | 0x30<br>(110000) | 0x1<br>(0001) | 0x2<br>(0010) | 0x3<br>(0011) | 0<br>(0) | 000<br>(000) | 0x0<br>(0000) | 0b01<br>(01) | 0x0<br>(0000) | 0xC048C010 |
| ADDR r1, r2, r3, LSL #2    | 0x30<br>(110000) | 0x1<br>(0001) | 0x2<br>(0010) | 0x3<br>(0011) | 0<br>(0) | 001<br>(001) | 0x2<br>(0010) | 0b01<br>(01) | 0x0<br>(0000) | 0xC048C490 |
| LOAD r1, (r2, a)           | 0x15<br>(010101) | 0x1<br>(0001) | 0x0<br>(0000) | 0x2<br>(0010) | 0<br>(0) | 000<br>(000) | 0x0<br>(0000) | 0b01<br>(01) | 0x0<br>(0000) | 0x54408010 |
| STOR -(r2, s), r1          | 0x13<br>(010011) | 0x0<br>(0000) | 0x1<br>(0001) | 0x2<br>(0010) | 0<br>(0) | 000<br>(000) | 0x0<br>(0000) | 0b10<br>(10) | 0x0<br>(0000) | 0x4C048020 |
| CMPR r4, r5                | 0x3A<br>(111010) | 0x4<br>(0100) | 0x4<br>(0100) | 0x5<br>(0101) | 0<br>(0) | 000<br>(000) | 0x0<br>(0000) | 0b01<br>(01) | 0x0<br>(0000) | 0xE9114010 |
| XORR r2, r3, r4, RTR #1    | 0x36<br>(110110) | 0x2<br>(0010) | 0x3<br>(0011) | 0x4<br>(0100) | 0<br>(0) | 110<br>(110) | 0x1<br>(0001) | 0b01<br>(01) | 0x0<br>(0000) | 0xD88D1850 |
| SUBR > r1, r2, r3, ASR r4  | 0x32<br>(110010) | 0x1<br>(0001) | 0x2<br>(0010) | 0x3<br>(0011) | 1<br>(1) | 100<br>(100) | 0x4<br>(0100) | 0b01<br>(01) | 0xD<br>(1101) | 0xC848F11D |
| ANDR CS r5, r6, r7, LSL r1 | 0x34<br>(110100) | 0x5<br>(0101) | 0x6<br>(0110) | 0x7<br>(0111) | 1<br>(1) | 001<br>(001) | 0x1<br>(0001) | 0b01<br>(01) | 0x3<br>(0011) | 0xD159E453 |
| SHFT r1, ASL #3            | 0x25<br>(100101) | 0x1<br>(0001) | 0x0<br>(0000) | 0x0<br>(0000) | 0<br>(0) | 011<br>(011) | 0x3<br>(0011) | 0b10<br>(10) | 0x0<br>(0000) | 0x94400CE0 |

Table 6.4: Register Format Encoding Examples

#### Notes:

- When R3 is not specified in assembly, it defaults to R1 (e.g., ADDR r4, r5 uses R1=0x4, R2=0x4, R3=0x5)
- SO (Shift Operand): 0 = literal shift amount, 1 = register contains shift amount (see rows 7 and 8)
- ST (Shift Type): 000=none, 001=LSL, 010=LSR, 011=ASL, 100=ASR, 101=ROL, 110=ROR
- AF for memory ops: 00=l, 01=a, 10=s, 11=p; for ALU ops: 00=no update, 01=update from ALU, 10=update from shift (see SHFT row 9)
- Condition code examples: 0x0 = Always, 0xD = Signed > (>), 0x3 = Carry Set (CS)
- SHFT is a meta instruction using ORRI opcode (0x25) with AF=0b10

## 6.7 Operand Encoding Details

### 6.7.1 Condition Codes

| Code | Mnemonic | Condition                                      |
|------|----------|------------------------------------------------|
| 0000 | (none)   | Always (unconditional)                         |
| 0001 | ==       | Equal ( $Z = 1$ )                              |
| 0010 | !=       | Not Equal ( $Z = 0$ )                          |
| 0011 | CS       | Carry Set ( $C = 1$ )                          |
| 0100 | CC       | Carry Clear ( $C = 0$ )                        |
| 0101 | NS       | Negative Set ( $N = 1$ )                       |
| 0110 | NC       | Negative Clear ( $N = 0$ )                     |
| 0111 | OS       | Overflow Set ( $V = 1$ )                       |
| 1000 | OC       | Overflow Clear ( $V = 0$ )                     |
| 1001 | HI       | Unsigned Higher ( $C = 1$ AND $Z = 0$ )        |
| 1010 | LO       | Unsigned Lower or Same ( $C = 0$ OR $Z = 1$ )  |
| 1011 | >=       | Signed Greater or Equal ( $N = V$ )            |
| 1100 | <        | Signed Less Than ( $N \neq V$ )                |
| 1101 | »        | Signed Greater Than ( $Z = 0$ AND $N = V$ )    |
| 1110 | «        | Signed Less or Equal ( $Z = 1$ OR $N \neq V$ ) |
| 1111 | NV       | Never (never executes)                         |

Table 6.5: Condition Code Encoding

For more details on condition codes see **Chapter 9**

### 6.7.2 Shift Types

| Code | Type                   | Mnemonic |
|------|------------------------|----------|
| 000  | None (no shift)        | –        |
| 001  | Logical Shift Left     | LSL      |
| 010  | Logical Shift Right    | LSR      |
| 011  | Arithmetic Shift Left  | ASL      |
| 100  | Arithmetic Shift Right | ASR      |
| 101  | Rotate Left            | RTL      |
| 110  | Rotate Right           | RTR      |
| 111  | Reserved               | –        |

Table 6.6: Shift Type Encoding

For more details on shift operations see **Chapter 8**

## 6.8 Opcode Encoding Patterns

The SIRCIS instruction set uses systematic opcode patterns to simplify decoding:

### 6.8.1 ALU Instructions

- **0x0\_\_**: Immediate operand
- **0x1\_\_**: Memory operations
- **0x2\_\_**: Short immediate with shift
- **0x3\_\_**: Register operand

### 6.8.2 Test vs. Save

All ALU operations have both a "save result" and "test only" variant:

- **0x\_\_0-0x\_\_7**: Save result to destination
- **0x\_\_8-0x\_\_F**: Test only (update flags, don't save result)

To turn a "save result" instruction to a "test only" instruction, you can simply add 0x8 to the opcode.

For example:

- **0x04**: **ANDI** – AND immediate, save result
- **0x0C**: **TSAI** – Test AND immediate (don't save result)

## 6.9 Instruction Fetch and Alignment

### 6.9.1 Fetch Process

Since instructions are 32 bits and the data bus is 16 bits wide, each instruction requires two fetch cycles:

1. Fetch high word (bits 31–16) from address in PC
2. Increment PC by 1
3. Fetch low word (bits 15–0) from address in PC
4. Increment PC by 1

### 6.9.2 Alignment Requirements

Instructions span two consecutive words and must begin at an even word address. The program counter (`p1`) must always be even.

Attempting to fetch an instruction from an odd word address will trigger an alignment fault exception.

## 6.10 Format Selection Guidelines

When writing assembly code, the assembler automatically selects the appropriate format based on operands:

| Assembly                          | Format Used     | Notes                   |
|-----------------------------------|-----------------|-------------------------|
| <code>ADDI r1, #1000</code>       | Immediate       | Full 16-bit immediate   |
| <code>ADDI r1, #10, LSL #2</code> | Short Immediate | 8-bit immediate + shift |
| <code>ADDR r1, r2, r3</code>      | Register        | Three register operands |

Table 6.7: Format Selection Examples

The choice of format affects instruction encoding but is generally transparent to the programmer, as the assembler handles the details.



## Chapter 7

# Addressing Modes

### 7.1 Overview

The SIRC-1 CPU supports seven addressing modes that specify how operands are accessed. These modes determine whether an operand is an immediate value, register contents, or data in memory at a computed address.

| Mode               | Description                                            |
|--------------------|--------------------------------------------------------|
| Immediate          | Operand is a constant value encoded in the instruction |
| Register Direct    | Operand is the contents of a specified register        |
| Indirect Immediate | Memory address = address register + immediate offset   |
| Indirect Register  | Memory address = address register + register offset    |
| Post-Increment     | Indirect register, then increment address register     |
| Pre-Decrement      | Decrement address register, then use as indirect       |
| Short Immediate    | 8-bit constant with optional shift                     |

Table 7.1: Addressing Modes Summary

### 7.2 Immediate Addressing

#### 7.2.1 Description

The operand is a constant value 16-bit value encoded directly in the instruction.

#### 7.2.2 Syntax

```
1 ADDI r1, #100      ; 16-bit immediate
2 ADDI r2, #-50       ; Signed 16-bit immediate
```

### 7.2.3 Effective Value

16-bit value encoded into the instruction.

### 7.2.4 Usage

- Loading constant values
- Arithmetic with known values
- Bit manipulation with masks

## 7.3 Register Direct Addressing

### 7.3.1 Description

The operand is the contents of a specified register. This is the most common mode for ALU operations in a load/store architecture.

### 7.3.2 Syntax

```
1 ADDR r1, r2, r3      ; r1 = r2 + r3
2 ADDR r4, r5          ; r4 = r4 + r5
```

### 7.3.3 Effective Value

The 16-bit value currently stored in the specified register.

### 7.3.4 Usage

- All ALU operations between registers
- Register-to-register data movement
- Testing register values

## 7.4 Indirect Immediate Addressing

### 7.4.1 Description

The effective address is computed by adding a 16-bit signed immediate offset to an address register pair. The operand is then loaded from or stored to this computed address.

### 7.4.2 Syntax

```
1 LOAD r1, (#8, a)      ; Load from address (a + 8)
2 STOR (#-4, s), r2     ; Store to address (s - 4)
3 LDEA p, (#100, p)    ; Jump relative (pc = pc + 100)
```

### 7.4.3 Effective Address

$$EA = \text{AddressRegister} + \text{Offset} \quad (7.1)$$

The address register can be: **l** (link), **a** (address), **s** (stack), or **p** (program counter).

### 7.4.4 Usage

- Accessing structure fields (fixed offset from base)
- Array element access (if index is constant)
- Stack frame access (local variables)
- Position-independent code (PC-relative addressing)

## 7.5 Indirect Register Addressing

### 7.5.1 Description

Similar to indirect immediate, but the offset is taken from a register instead of being encoded as an immediate value. This enables runtime-computed offsets.

### 7.5.2 Syntax

```

1 LOAD r1, (r2, a)      ; Load from address (a + r2)
2 STOR (r4, s), r3      ; Store to address (s + r4)

```

### 7.5.3 Effective Address

$$EA = \text{AddressRegister} + \text{IndexRegister} \quad (7.2)$$

### 7.5.4 Usage

- Array indexing with variable index
- Table lookups
- Dynamic offset calculations

## 7.6 Post-Increment Addressing

### 7.6.1 Description

The effective address is computed using indirect register addressing, then the address register pair is incremented after the memory operation.

This mode is particularly useful for iterating through arrays or implementing stack operations.

### 7.6.2 Syntax

```
1 LOAD r1, (r2, s)+      ; Load from (s + r2), then s = s + 1
2 LOAD r1, (#2, s)+      ; Load from (s + 2), then s = s + 1
```

### 7.6.3 Operation Sequence

1. Compute  $EA = \text{AddressRegister} + \text{Offset}$
2. Perform memory operation (load)
3.  $\text{AddressRegister} = \text{AddressRegister} + 1$

### 7.6.4 Usage

- Stack pop operations
- Forward array traversal
- Buffer reading

## 7.7 Pre-Decrement Addressing

### 7.7.1 Description

The address register pair is decremented before computing the effective address for the memory operation.

This mode is the complement of post-increment and is useful for stack push operations.

### 7.7.2 Syntax

```
1 STOR -(r2, s), r1      ; s = s - 1, then store to (s + r2)
2 STOR -(#2, s), r3      ; s = s - 1, then store to (s + 2)
```

### 7.7.3 Operation Sequence

1.  $\text{AddressRegister} = \text{AddressRegister} - 1$
2. Compute  $EA = \text{AddressRegister} + \text{Offset}$
3. Perform memory operation (store)

### 7.7.4 Usage

- Stack push operations
- Backward array traversal
- Buffer writing

## 7.8 Short Immediate Addressing

### 7.8.1 Description

Similar to immediate addressing, but limited to 8-bit values so shift information can fit into the instruction encoding.

### 7.8.2 Syntax

```
1 ADDI r1, #10, LSL #2 ; r1 = (r1 << 2) + 10
2 SUBI r2, #1, LSL #8 ; r2 = (r2 << 8) - 1
```

### 7.8.3 Effective Value

8-bit value encoded into the instruction.

The shift is applied to the source operand, not the constant.

### 7.8.4 Usage

- Loading constant values
- Arithmetic with known values
- Bit manipulation with masks

## 7.9 Address Register Selection

Many addressing modes require specifying which address register pair to use. The 2-bit address register field encodes:

| Bits | Symbol | Register Pair             |
|------|--------|---------------------------|
| 00   | l      | Link Register (lh, ll)    |
| 01   | a      | Address Register (ah, al) |
| 10   | s      | Stack Pointer (sh, sl)    |
| 11   | p      | Program Counter (ph, pl)  |

Table 7.2: Address Register Encoding

## 7.10 Addressing Mode Examples

### 7.10.1 Structure Access

```
1 ; Assume 'a' points to a struct:
2 ; struct Point {
3 ;     int16_t x; // offset 0
4 ;     int16_t y; // offset 2
```

```
5 ; }
6
7 LOAD r1, (#0, a)      ; r1 = point.x
8 LOAD r2, (#2, a)      ; r2 = point.y
```

### 7.10.2 Array Iteration

```
1 ; Iterate array of 16-bit values
2 ; Assume 'a' points to array start
3 ; r7 = counter
4 loop:
5     LOAD r1, (#0, a)+    ; Load element, advance by 1
6     ; ... process r1 ...
7     SUBI r7, #1
8     BRAN|!= loop
```

### 7.10.3 Stack Frame

```
1 :begin_subroutine
2
3 ; Function prologue - save registers
4 STOR -(#2, s), r1
5 STOR -(#2, s), r2
6 STOR -(#2, s), r3
7
8 ; Access parameters (above saved regs)
9 LOAD r4, (#6, s)        ; First parameter
10 LOAD r5, (#8, s)        ; Second parameter
11
12 ; Function epilogue - restore registers
13 LOAD r3, (#0, s)+
14 LOAD r2, (#0, s)+
15 LOAD r1, (#0, s)+
16 RETS
```

## 7.11 Addressing Mode Restrictions

### 7.11.1 Privilege Mode Restrictions

In protected mode, indirect addressing modes that would write to the high byte of an address register are restricted:

- **LDEA with both registers:** Privileged in protected mode
- **LDEA with low register only:** Allowed (can't change segment)
- **Post-increment/Pre-decrement:** Can only modify low register in protected mode

### 7.11.2 Alignment

- All memory addresses refer to 16-bit words
- There are no alignment restrictions for general memory I/O
- All instructions span 2 words and must begin at an even word address
- Unaligned instruction fetches (odd word address) trigger an alignment fault exception



## Chapter 8

# Shift Operations

### 8.1 Overview

The SIRC-1 CPU supports powerful shift operations that can be applied to operands in many instructions. Shifts can multiply or divide values by powers of 2, extract bit fields, or perform bit rotations.

Shift operations are encoded within Short Immediate and Register format instructions, allowing a value to be shifted before being used in an ALU operation—all within a single instruction.

Shift operations are only applied to source operands. A shift cannot be applied to the result of an operation before register writeback.

**Note:** By default, when shifts are used with ALU instructions (apart from the SHFT meta instruction), the status register flags are updated based on the ALU operation result, not the shift. However, you can manually control this behavior using the status override syntax (see Section 8.12).

### 8.2 Shift Types

#### 8.2.1 Shift Type Encoding

| Code | Mnemonic      | Type                   | Description                      |
|------|---------------|------------------------|----------------------------------|
| 000  | NUL (or none) | None                   | No shift performed               |
| 001  | LSL           | Logical Shift Left     | Shift left, fill with zeros      |
| 010  | LSR           | Logical Shift Right    | Shift right, fill with zeros     |
| 011  | ASL           | Arithmetic Shift Left  | Same as LSL (shifts in zeros)    |
| 100  | ASR           | Arithmetic Shift Right | Shift right, preserving sign bit |
| 101  | RTL           | Rotate Left            | Rotate left through carry        |
| 110  | RTR           | Rotate Right           | Rotate right through carry       |
| 111  | —             | Reserved               | Reserved for future use          |

Table 8.1: Shift Type Encoding

## 8.3 Shift Syntax

A shift postamble can come after any short immediate or register instruction. It is applied in the "Decode and Register Fetch" phase of instruction execution, before any ALU operations or memory address calculation occurs.

It is applied to the first source operand of an instruction. If using the three operand form of instruction, it will be the second operand. If using the two operand form of instruction, the second operand is inferred to be the same as the destination.

See this table for some examples of which operand is shifted:

| Instruction             | How the CPU Interprets It | First Source Operand |
|-------------------------|---------------------------|----------------------|
| ADDR r1, r2, r3, LSL #1 | ADDR r1, r2, r3, LSL #1   | r2                   |
| ADDR r1, r2, LSL #1     | ADDR r1, r1, r2, LSL #1   | r1                   |
| ADDI r1, #2, LSL #1     | ADDI r1, r1, #2, LSL #1   | r1                   |

Table 8.2: Examples of first source operands

## 8.4 Shift Type Details

### 8.4.1 Logical Shift Left (LSL)

**Operation:** Shift bits to the left, filling vacated bits with zeros.

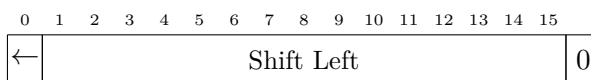


Figure 8.1: Logical Shift Left

- Bit 15 is shifted into the carry flag
- Bit 0 receives 0
- Equivalent to multiplying by  $2^n$  where  $n$  is the shift count

**Example:**

```

1 ; Multiply r1 by 4
2 ADDI r1, #0, LSL #2 ; r1 = (r1 << 2) = r1 * 4
3
4 ; Set bit 10
5 ORRI r3, #1, LSL #10 ; r3 |= (r3 << 10) with immediate 1 = 0x0400

```

### 8.4.2 Logical Shift Right (LSR)

**Operation:** Shift bits to the right, filling vacated bits with zeros.

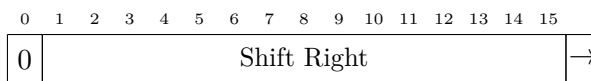


Figure 8.2: Logical Shift Right

- Bit 0 is shifted into the carry flag
- Bit 15 receives 0
- Equivalent to unsigned division by  $2^n$
- Use for unsigned values only

**Example:**

```

1 ; Divide unsigned value by 8
2 ADDI r1, #0, LSR #3 ; r1 = (r1 >> 3) = r1 / 8 (unsigned)
3
4 ; Extract upper byte
5 ADDI r3, #0, LSR #8 ; r3 = (r3 >> 8)

```

**8.4.3 Arithmetic Shift Left (ASL)**

**Operation:** Identical to LSL. Shift bits to the left, filling with zeros.

- Equivalent to signed multiplication by  $2^n$
- Can cause signed overflow if sign bit changes
- Overflow flag is set if sign bit changes during shift

**Example:**

```

1 ; Multiply signed value by 2
2 ADDI r1, #0, ASL #1 ; r1 = (r1 << 1) = r1 * 2 (signed)

```

**8.4.4 Arithmetic Shift Right (ASR)**

**Operation:** Shift bits to the right, preserving the sign bit (bit 15).

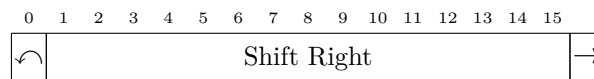


Figure 8.3: Arithmetic Shift Right

- Bit 0 is shifted into the carry flag
- Bit 15 (sign bit) is copied into bit 15 and bit 14
- Equivalent to signed division by  $2^n$  (rounds toward negative infinity)
- Use for signed values only

**Example:**

```

1 ; Divide signed value by 4
2 ADDI r1, #0, ASR #2 ; r1 = (r1 >> 2) = r1 / 4 (signed)
3
4 ; Average two signed values
5 ADDR r3, r4, r5
6 ADDI r3, #0, ASR #1 ; r3 = (r3 >> 1) = (r4 + r5) / 2

```

### 8.4.5 Rotate Left (RTL)

**Operation:** Rotate bits to the left through the carry flag.

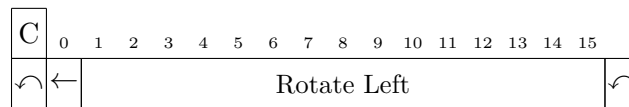


Figure 8.4: Rotate Left

- Bit 15 moves into the carry flag
- Previous carry flag value moves into bit 0
- No data is lost
- Useful for multi-word shifts and bit manipulation

**Example:**

```

1 ; Rotate r1 left by 1
2 ADDI r1, #0, ROL #1
3
4 ; Multi-word left shift (32-bit value in r1:r2)
5 ADDI r2, #0, LSL #1 ; Shift low word, bit 15 -> carry
6 ADDI r1, #0, RTL #1 ; Shift high word, carry -> bit 0

```

### 8.4.6 Rotate Right (RTR)

**Operation:** Rotate bits to the right through the carry flag.

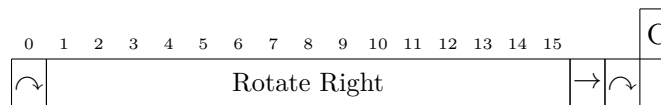


Figure 8.5: Rotate Right

- Bit 0 moves into the carry flag
- Previous carry flag value moves into bit 15
- No data is lost

**Example:**

```

1 ; Rotate r1 right by 1
2 ADDI r1, #0, RTR #1
3
4 ; Check if bit 0 is set
5 ADDI r2, #0, RTR #1 ; Bit 0 -> carry
6 ; Now branch on carry to test bit 0

```

## 8.5 Shift Operands

The Shift Operand (SO) bit determines what value is shifted:

| SO Bit | Mode                    | Description                                              |
|--------|-------------------------|----------------------------------------------------------|
| 0      | Shift by Immediate      | R2 (first source operand) is shifted by a literal amount |
| 1      | Shift Count in Register | R2 is shifted by the amount in the shift count register  |

Table 8.3: Shift Operand Modes

### 8.5.1 Mode 0: Shift by Immediate

The first source operand (R2) is shifted by a literal amount before the ALU operation.

```

1 ; Add (r1 * 4) + 1 to r1
2 ADDI r1, #1, LSL #2 ; r1 = r1 + (r2 << 2) + 1
3 ; SO=0: r2 is shifted by literal 2

```

### 8.5.2 Mode 1: Shift by Register Count

The shift count is taken from the shift amount field (interpreted as a register), allowing variable-length shifts. R2 (first source) is still what gets shifted.

```

1 ; Variable shift
2 ; Shift amount register contains shift count
3 ADDR r1, r2, r3, LSL r4 ; r1 = (r2 << r4) + r3
4 ; SO=1: shift r2 by value in r4
5
6 ; Barrel shifter emulation
7 ADDR r4, r5, r6, ASR r7 ; r4 = (r5 >> r7) + r6 (arithmetic)

```

## 8.6 Shift Count

The shift count is a 4-bit field, allowing shifts of 0–15 positions.

- Shift count of 0 means no shift is performed
- Maximum shift count is 15
- Shift counts  $\geq 16$  would be meaningless for 16-bit values

## 8.7 Status Flag Effects

### 8.7.1 Default Behavior

When shift operations are used with ALU instructions, the status flags are **by default** updated based on the **ALU operation result**, not the shift operation. For example, in `CMPI r2, #1, LSL #2`, the flags reflect the comparison result, not the shift.

To update flags based on the shift operation instead, use the `[S]` status override modifier (see Section 8.12), or use the **SHFT** meta-instruction (Chapter refch:meta-instructions).

### 8.7.2 Carry Flag

For all shifts and rotates:

- The last bit shifted out is placed in the carry flag
- For shifts of more than 1 position, only the final bit shifted out matters
- If shift count is 0, carry flag is not modified

### 8.7.3 Zero Flag

Set if the result after shifting is zero.

### 8.7.4 Negative Flag

Set if bit 15 of the result is 1 (result is negative when interpreted as signed).

### 8.7.5 Overflow Flag

- For **ASL**: Set if the sign bit changes during the shift (signed overflow)
- For other shifts: Behavior is undefined; typically unchanged or cleared

## 8.8 Common Use Cases

### 8.8.1 Multiplication by Constants

```
1 ; Multiply by 2
2 ADDI r1, #0, LSL #1    ; r1 = (r1 << 1) = r1 * 2
3
4 ; Multiply by 3
5 ADDR r1, r2, r2, LSL #1    ; r1 = (r2 << 1) + r2 = r2 * 3
6
7 ; Multiply by 5
8 ADDR r1, r2, r2, LSL #2    ; r1 = (r2 << 2) + r2 = r2 * 5
```

### 8.8.2 Division by Powers of 2

```

1 ; Unsigned divide by 4
2 LOAD r1, r2          ; r1 = r2
3 ADDI r1, #0, LSR #2  ; r1 = (r1 >> 2) = r1 / 4 (unsigned)
4
5 ; Signed divide by 8
6 LOAD r1, r2          ; r1 = r2
7 ADDI r1, #0, ASR #3  ; r1 = (r1 >> 3) = r1 / 8 (signed)

```

### 8.8.3 Bit Field Extraction

```

1 ; Extract bits 8-11 from r1
2 LOAD r2, r1          ; r2 = r1
3 ADDI r2, #0, LSR #8  ; Shift down
4 ANDI r2, #0x0F        ; Mask to 4 bits
5
6 ; Extract and sign-extend bit 7
7 LOAD r2, r1          ; r2 = r1
8 ADDI r2, #0, LSL #8  ; Shift bit 7 to bit 15
9 ADDI r2, #0, ASR #8  ; Arithmetic shift to sign-extend

```

### 8.8.4 Bit Manipulation

```

1 ; Set bit N (where N is in r2)
2 LOAD r3, #1          ; r3 = 1
3 ADDI r3, #0, LSL r2  ; r3 = (r3 << r2) = 1 << N
4 ORRR r1, r3          ; r1 |= (1 << N)
5
6 ; Clear bit N
7 LOAD r3, #1          ; r3 = 1
8 ADDI r3, #0, LSL r2  ; r3 = (r3 << r2) = 1 << N
9 LOAD r4, #0xFFFF
10 XORR r3, r4, r3      ; r3 = 0xFFFF ^ (1 << N) = ~(1 << N)
11 ANDR r1, r1, r3      ; r1 = r1 & ~(1 << N)
12
13 ; Test bit N
14 LOAD r3, #1          ; r3 = 1
15 ADDI r3, #0, LSL r2  ; r3 = (r3 << r2) = 1 << N
16 TSAR r1, r3          ; Test r1 & (1 << N)
17 BRAN|!= bit_was_set ; Branch if bit set

```

## 8.9 Multi-Word Shifts

For operations on values larger than 16 bits, shifts and rotates can be combined:

### 8.9.1 32-bit Left Shift

```
1 ; Shift 32-bit value in r1:r2 left by 1
2 ; r1 = high word, r2 = low word
3 ADDI r2, #0, LSL #1 ; Shift low, MSB -> carry
4 ADDI r1, #0, RTL #1 ; Rotate high with carry
```

### 8.9.2 32-bit Right Shift (Unsigned)

```
1 ; Shift 32-bit value in r1:r2 right by 1 (unsigned)
2 ADDI r1, #0, LSR #1 ; Shift high, LSB -> carry
3 ADDI r2, #0, RTR #1 ; Rotate low with carry
```

### 8.9.3 32-bit Right Shift (Signed)

```
1 ; Shift 32-bit value in r1:r2 right by 1 (signed)
2 ADDI r1, #0, ASR #1 ; Arithmetic shift high
3 ADDI r2, #0, RTR #1 ; Rotate low with carry
```

## 8.10 Performance Considerations

- All shift operations complete in the same number of cycles regardless of shift count
- Shifts can be performed "for free" as part of another instruction (no extra cycles)
- Using shifts for multiplication/division by powers of 2 is much faster than using a multiply/divide instruction (if one existed)
- Barrel shifter implementation in hardware allows any shift count in constant time

## 8.11 Limitations

- Maximum shift count is 15 (4-bit field)
- For shifts > 15, multiple shift instructions must be used
- Rotates include the carry flag, so rotating by 16 does not equal original value
- Some shift types (e.g., reserved type 111) may have undefined behavior

## 8.12 Status Flag Update Control

By default, ALU instructions with shift operations update the status register flags based on the **ALU operation result**, not the shift operation. However, the SIRC-1 provides manual control over which operation updates the flags.

### 8.12.1 Status Override Syntax

A status override modifier can be added immediately after an instruction mnemonic using square brackets:

| Modifier | Meaning | Description                             |
|----------|---------|-----------------------------------------|
| [A]      | ALU     | Update flags from ALU result (default)  |
| [S]      | Shift   | Update flags from shift result, not ALU |
| [N]      | None    | Do not update status flags at all       |

Table 8.4: Status Flag Override Modifiers

### 8.12.2 Examples

#### Default Behavior (ALU flags):

```

1 ; Without override, flags reflect the ALU addition result
2 ADDI r2, #1, LSL #2          ; r2 = (r2 << 2) + 1
3                               ; Flags: based on addition result

```

#### Shift Flag Override:

```

1 ; Update flags based on shift result instead of ALU
2 ADDI [S] r2, #1, LSL #2      ; r2 = (r2 << 2) + 1
3                               ; Flags: based on (r2 << 2), not the +1
4
5 ; Check if shifted value is zero
6 CMPI [S] r3, #0, LSR #4      ; Compare (r3 >> 4) with 0
7                               ; Flags: based on shift, not comparison

```

#### No Flag Update:

```

1 ; Perform operation without affecting status flags
2 ADDI [N] r1, #5, LSL #1      ; r1 = (r1 << 1) + 5
3                               ; Flags: unchanged
4
5 ; Useful when preserving flags from previous operation
6 CMPI r4, r5                  ; Set flags based on comparison
7 ADDI [N] r6, #1              ; Increment r6 without changing flags
8 BRAN |== equal_branch        ; Branch using flags from CMPI

```

### 8.12.3 Use Cases

#### Testing Shift Results

When you need to know properties of a shifted value without performing a subsequent comparison:

```

1 ; Check if r1 << 3 would be negative
2 ADDI [S] r1, #0, LSL #3      ; r1 = r1 << 3, flags from shift
3 BRAN |NS overflow_detected   ; Branch if sign bit set

```

## Preserving Flags

When performing operations that should not affect condition codes:

```
1 ; Test condition
2 CMPI r1, r2 ; Compare r1 and r2
3
4 ; Do some work without affecting flags
5 ADDI[N] r3, #1 ; Increment counter (flags unchanged)
6 LOAD[N] r4, (#0, a)+ ; Load next value (flags unchanged)
7
8 ; Branch using original comparison flags
9 BRAN|>= r1_was_greater
```

## SHFT Meta-Instruction

The **SHFT** meta-instruction (see Chapter 16) uses the [S] modifier internally:

```
1 SHFT r1, ASL #3 ; Expands to: ORRI[S] r1, #0, ASL #3
```

This provides a clean syntax for shift-only operations with flag updates.

### 8.12.4 Assembler Syntax Notes

- The status override modifier must appear immediately after the instruction mnemonic with no spaces: **ADDI**[S]
- The modifier is optional; omitting it defaults to [A] (ALU flag updates)
- The override applies to the entire instruction, including any conditional execution
- Explicit [A] is rarely needed but can improve code readability

## Chapter 9

# Condition Codes

### 9.1 Overview

Most SIRCIS instructions can be executed conditionally based on the state of the condition flags in the status register. This allows for efficient implementation of conditional logic without requiring explicit branch instructions.

The 4-bit condition code field in each instruction specifies under what conditions the instruction should execute. If the condition is not met, the instruction behaves as a **NOOP** (no operation).

### 9.2 Condition Code Encoding

| Code | Mnemonic     | Condition (when instruction executes)          |
|------|--------------|------------------------------------------------|
| 0000 | AL (or none) | Always (unconditional)                         |
| 0001 | ==           | Equal ( $Z = 1$ )                              |
| 0010 | !=           | Not Equal ( $Z = 0$ )                          |
| 0011 | CS           | Carry Set ( $C = 1$ )                          |
| 0100 | CC           | Carry Clear ( $C = 0$ )                        |
| 0101 | NS           | Negative Set ( $N = 1$ )                       |
| 0110 | NC           | Negative Clear ( $N = 0$ )                     |
| 0111 | OS           | Overflow Set ( $V = 1$ )                       |
| 1000 | OC           | Overflow Clear ( $V = 0$ )                     |
| 1001 | HI           | Unsigned Higher ( $C = 1$ AND $Z = 0$ )        |
| 1010 | LO           | Unsigned Lower or Same ( $C = 0$ OR $Z = 1$ )  |
| 1011 | >=           | Signed Greater or Equal ( $N = V$ )            |
| 1100 | «            | Signed Less Than ( $N \neq V$ )                |
| 1101 | »            | Signed Greater Than ( $Z = 0$ AND $N = V$ )    |
| 1110 | <=           | Signed Less or Equal ( $Z = 1$ OR $N \neq V$ ) |
| 1111 | NV           | Never (never executes)                         |

Table 9.1: Condition Code Encoding

## 9.3 Condition Code Categories

### 9.3.1 Simple Conditions

These conditions test a single status flag:

**Always (default)** No condition code specified; instruction always executes.

**== (Equal)** Executes when  $Z = 1$ . Typically used after **CMP** to test equality.

**!= (Not Equal)** Executes when  $Z = 0$ . Typical use: test inequality.

**CS (Carry Set)** Executes when  $C = 1$ . Can indicate unsigned overflow or borrow.

**CC (Carry Clear)** Executes when  $C = 0$ . No unsigned overflow/borrow.

**NS (Negative Set)** Executes when  $N = 1$ . Tests if result is negative (signed).

**NC (Negative Clear)** Executes when  $N = 0$ . Tests if result is positive or zero (signed).

**OS (Overflow Set)** Executes when  $V = 1$ . Signed overflow occurred.

**OC (Overflow Clear)** Executes when  $V = 0$ . No signed overflow.

### 9.3.2 Unsigned Comparisons

These conditions are used after comparing unsigned integers:

**HI (Unsigned Higher)** Executes when  $C = 1$  AND  $Z = 0$ . Tests if first operand  $>$  second operand (unsigned).

**LO (Unsigned Lower or Same)** Executes when  $C = 0$  OR  $Z = 1$ . Tests if first  $\leq$  second (unsigned).

**Note:** For unsigned comparisons:

- Use HI and LO after **CMP**
- HI tests if first  $>$  second (unsigned)
- LO tests if first  $\leq$  second (unsigned)
- CS is equivalent to unsigned  $\geq$
- CC is equivalent to unsigned  $<$

### 9.3.3 Signed Comparisons

These conditions are used after comparing signed integers:

**$\geq$  (Signed Greater or Equal)** Executes when  $N = V$ . First operand  $\geq$  second operand (signed).

**$\llcorner$  (Signed Less Than)** Executes when  $N \neq V$ . First operand  $<$  second operand (signed).

**$\gg$  (Signed Greater Than)** Executes when  $Z = 0$  AND  $N = V$ . First operand  $>$  second operand (signed).

**$\leq$  (Signed Less or Equal)** Executes when  $Z = 1$  OR  $N \neq V$ . First operand  $\leq$  second operand (signed).

### 9.3.4 Special Conditions

**Never (NV)** Never executes; equivalent to **NOOP**. Can be used for instruction padding or to temporarily disable instructions during debugging.

## 9.4 Assembly Syntax

Condition codes are specified by appending a pipe character (|) followed by the condition mnemonic to the instruction:

```

1 ; Unconditional
2 ADDI r1, #10
3
4 ; Conditional - execute only if equal
5 ADDI|== r2, #20
6
7 ; Conditional branch
8 BRAN!= loop_start
9
10 ; Multiple conditions
11 ADDR|HI r3, r4, r5
12 STOR|NS (#0, s), r6

```

## 9.5 Usage with Compare Instructions

The most common use of condition codes is with compare (**CMP**) instructions:

### 9.5.1 Unsigned Comparison

```

1 ; if (r1 > r2) { r3 = r4; } (unsigned)
2 CMPR r1, r2 ; Compare r1 and r2
3 LOAD|HI r3, r4 ; r3 = r4 if r1 > r2 (unsigned)
4
5 ; if (r1 <= 100) { branch } (unsigned)
6 CMPI r1, #100
7 BRAN|LO target

```

### 9.5.2 Signed Comparison

```

1 ; if (r1 >= r2) { r3 = r4; } (signed)
2 CMPR r1, r2 ; Compare r1 and r2
3 LOAD|>= r3, r4 ; r3 = r4 if r1 >= r2 (signed)
4
5 ; if (r1 < -10) { branch } (signed)
6 CMPI r1, #-10
7 BRAN|<< negative_case

```

### 9.5.3 Equality Testing

```

1 ; if (r1 == r2) { branch }
2 CMPR r1, r2
3 BRAN|== equal_case
4
5 ; if (r1 != 0) { branch }
6 CMPI r1, #0
7 BRAN|!= non_zero_case

```

## 9.6 Condition Code Truth Tables

### 9.6.1 After Unsigned Comparison (CMPR rA, rB)

| Relationship            | Z | C | Condition | Code | Meaning        |
|-------------------------|---|---|-----------|------|----------------|
| $rA = rB$               | 1 | * | ==        | 0001 | Equal          |
| $rA \neq rB$            | 0 | * | !=        | 0010 | Not Equal      |
| $rA > rB$ (unsigned)    | 0 | 1 | HI        | 1001 | Higher         |
| $rA < rB$ (unsigned)    | 0 | 0 | CC        | 0100 | Lower          |
| $rA \geq rB$ (unsigned) | * | 1 | CS        | 0011 | Higher or Same |
| $rA \leq rB$ (unsigned) | * | * | LO        | 1010 | Lower or Same  |

Table 9.2: Unsigned Comparison Results

### 9.6.2 After Signed Comparison (CMPR rA, rB)

| Relationship          | Z | N | V | Condition | Code | Meaning          |
|-----------------------|---|---|---|-----------|------|------------------|
| $rA = rB$             | 1 | * | * | ==        | 0001 | Equal            |
| $rA \neq rB$          | 0 | * | * | !=        | 0010 | Not Equal        |
| $rA > rB$ (signed)    | 0 | 0 | 0 | »         | 1101 | Greater          |
| $rA > rB$ (signed)    | 0 | 1 | 1 | »         | 1101 | Greater          |
| $rA < rB$ (signed)    | * | 0 | 1 | «         | 1100 | Less             |
| $rA < rB$ (signed)    | * | 1 | 0 | «         | 1100 | Less             |
| $rA \geq rB$ (signed) | * | 0 | 0 | >=        | 1011 | Greater or Equal |
| $rA \geq rB$ (signed) | * | 1 | 1 | >=        | 1011 | Greater or Equal |
| $rA \leq rB$ (signed) | 1 | * | * | <=        | 1110 | Less or Equal    |
| $rA \leq rB$ (signed) | 0 | 0 | 1 | <=        | 1110 | Less or Equal    |
| $rA \leq rB$ (signed) | 0 | 1 | 0 | <=        | 1110 | Less or Equal    |

Table 9.3: Signed Comparison Results

## 9.7 Advanced Usage

### 9.7.1 Conditional Assignment

```

1 ; max = (a > b) ? a : b (unsigned)
2 CMPR r1, r2           ; Compare a and b
3 LOAD|HI r3, r1        ; r3 = a if a > b
4 LOAD|LO r3, r2        ; r3 = b if a <= b

```

### 9.7.2 Conditional Increment

```

1 ; if (condition) count++
2 CMPI r1, #0
3 ADDI|!= r2, #1        ; Increment r2 if r1 != 0

```

### 9.7.3 Conditional Function Call

```

1 ; if (r1 > 0) call_function()
2 CMPI r1, #0
3 LJSR|>> (#0, a)      ; Call function if r1 > 0 (signed)

```

### 9.7.4 Loop Construction

```

1 ; for (i = 10; i > 0; i--)
2 ADDI r1, #0, #10      ; i = 10
3 loop:
4   ; ... loop body ...
5   SUBI r1, #1          ; i--
6   BRAN|HI loop        ; Continue if i > 0 (unsigned)

```

## 9.8 Condition Code Selection Guide

| Intent               | After CMP        | Condition Code |
|----------------------|------------------|----------------|
| If equal             | CMPR a, b        | ==             |
| If not equal         | CMPR a, b        | !=             |
| If a > b (unsigned)  | CMPR a, b        | HI             |
| If a ≥ b (unsigned)  | CMPR a, b        | CS             |
| If a < b (unsigned)  | CMPR a, b        | CC             |
| If a ≤ b (unsigned)  | CMPR a, b        | LO             |
| If a > b (signed)    | CMPR a, b        | »              |
| If a ≥ b (signed)    | CMPR a, b        | >=             |
| If a < b (signed)    | CMPR a, b        | «              |
| If a ≤ b (signed)    | CMPR a, b        | <=             |
| If negative          | CMPI a, #0       | NS             |
| If positive or zero  | CMPI a, #0       | NC             |
| If overflow occurred | After arithmetic | OS             |
| If no overflow       | After arithmetic | OC             |

Table 9.4: Condition Code Selection Guide

## 9.9 Performance Considerations

- Conditional instructions do not branch, avoiding pipeline flushes
- When a condition is false, the instruction still takes 6 cycles but performs no operation
- For short conditional sequences, conditional execution can be faster than branching
- For longer conditional blocks, explicit branches may be more efficient

## 9.10 Common Pitfalls

- **Signed vs. Unsigned:** Using unsigned conditions (HI, LO) for signed values or vice versa produces incorrect results
- **Flag Clobbering:** Flags are modified by most ALU instructions. Ensure the condition flags haven't been modified between the comparison and conditional instruction:

```

1 ; WRONG - flags clobbered
2 CMPR r1, r2
3 ADDI r3, #100           ; Modifies flags!
4 BRAN|HI target         ; Tests wrong flags
5
6 ; CORRECT - flags preserved
7 CMPR r1, r2
8 BRAN|HI target
9 ADDI r3, #100           ; After the branch

```

- **Inverted Logic:** Remember that `CMP A, B` computes  $A - B$ , so condition codes test  $A$  relative to  $B$ , not  $B$  relative to  $A$



## Part III

# SIRCIS Instruction Reference



## Chapter 10

# Reading Instruction Descriptions

This part of the manual describes each SIRCIS instruction in a consistent format. The goal is to make each instruction entry usable as an implementation contract, not only as programmer guidance.

### 10.1 Instruction Entry Fields

**Mnemonic** The assembly mnemonic or mnemonic family. Immediate and register variants are listed together when they perform the same operation.

**Opcodes** The 6-bit opcode values for each supported instruction format.

**Syntax** The accepted assembler forms. Meta-instructions and assembler conveniences are called out separately from real CPU instruction encodings.

**Operands** The permitted operand kinds and addressing modes for the instruction.

**Operation** Pseudocode for the architectural effect of the instruction. Arithmetic is performed on 16-bit word values unless otherwise stated.

**Write-back** Whether the computed result is written to a register, discarded, or used to update control state.

**Status Flags** The effect on the condition flags in the status register.

**Condition Codes** All normal instructions may be conditional. If the condition is false, the instruction has no architectural side effects unless a later instruction description explicitly states otherwise.

**Exceptions** Faults or traps that can be raised by the instruction. When an exception can occur, the entry should state whether partial side effects are possible.

**Timing** The base execution time and any circumstances that add wait states.

**Privilege** Whether the instruction is available in protected mode, supervisor mode, or both.

## 10.2 Notation

| Notation | Meaning                                                |
|----------|--------------------------------------------------------|
| rD       | Destination general-purpose register                   |
| rS       | Source general-purpose register                        |
| rS1      | First source register, usually the left-hand operand   |
| rS2      | Second source register, usually the right-hand operand |
| #imm16   | 16-bit immediate word encoded in immediate format      |
| #imm8    | 8-bit immediate encoded in short immediate format      |
| shift    | Optional SIRCIS shift definition                       |
| SR.X     | Status register field X                                |

Table 10.1: Instruction Description Notation

## 10.3 Status Flag Effect Symbols

Flag tables use the following symbols:

| Symbol | Meaning                                          |
|--------|--------------------------------------------------|
| *      | Updated from the instruction result              |
| 0      | Cleared to zero                                  |
| 1      | Set to one                                       |
| -      | Preserved                                        |
| S      | Updated from the shifter result when [S] is used |

Table 10.2: Status Flag Effect Symbols

## 10.4 Conditional Execution

Every encoded instruction contains a condition-code field. If the condition evaluates true, the instruction executes normally. If the condition evaluates false, the instruction is skipped: destination registers, address registers, memory, the program counter, link register, coprocessor state, and status flags are not modified by that instruction.

## 10.5 Status Update Overrides

ALU instructions can select the source of status flag updates with the status update field:

| Syntax   | Encoded AF | Effect                               |
|----------|------------|--------------------------------------|
| [N]      | 00         | Preserve status flags                |
| [A]      | 01         | Update flags from the ALU result     |
| [S]      | 10         | Update flags from the shifter result |
| Reserved | 11         | Reserved for future use              |

Table 10.3: Status Update Source Encoding

When no override is written, ALU instructions use **[A]**. Instructions that do not support status updates must not use status update override syntax.



# Chapter 11

## Instruction Summary

### 11.1 Overview

The SIRCIS instruction set consists of 64 possible opcodes (6-bit opcode field), of which approximately 52 are documented and assigned. The instructions are organized into logical groups:

- **ALU Instructions (0x00–0x0F, 0x20–0x2F, 0x30–0x3F):** Arithmetic, logical, and comparison operations
- **Memory Instructions (0x10–0x17):** Load and store operations
- **Control Flow (0x18–0x1F):** Branches, jumps, and address loading
- **Coprocessor (0x0F, 0x2F, 0x3F):** Coprocessor interface

## 11.2 Complete Instruction List

| Opcode | Mnemonic | Format          | Flags | Operation                     |
|--------|----------|-----------------|-------|-------------------------------|
| 0x00   | ADDI     | Immediate       | NZCV  | Add immediate                 |
| 0x01   | ADCI     | Immediate       | NZCV  | Add immediate with carry      |
| 0x02   | SUBI     | Immediate       | NZCV  | Subtract immediate            |
| 0x03   | SBCI     | Immediate       | NZCV  | Subtract immediate with carry |
| 0x04   | ANDI     | Immediate       | NZ    | AND immediate                 |
| 0x05   | ORRI     | Immediate       | NZ    | OR immediate                  |
| 0x06   | XORI     | Immediate       | NZ    | XOR immediate                 |
| 0x07   | LOAD     | Immediate       | –     | Load immediate (move)         |
| 0x08   | –        | –               | –     | Undocumented                  |
| 0x09   | –        | –               | –     | Undocumented                  |
| 0x0A   | CMPI     | Immediate       | NZCV  | Compare immediate             |
| 0x0B   | –        | –               | –     | Undocumented                  |
| 0x0C   | TSAI     | Immediate       | NZ    | Test AND immediate            |
| 0x0D   | –        | –               | –     | Undocumented                  |
| 0x0E   | TSXI     | Immediate       | NZ    | Test XOR immediate            |
| 0x0F   | COPI     | Immediate       | –     | Coprocessor call immediate    |
|        |          |                 |       |                               |
| 0x10   | STOR     | Indirect Imm    | –     | Store to (addr + imm)         |
| 0x11   | STOR     | Indirect Reg    | –     | Store to (addr + reg)         |
| 0x12   | STOR     | Pre-Dec         | –     | Store with pre-decrement      |
| 0x13   | STOR     | Pre-Dec Reg     | –     | Store with pre-dec (reg)      |
| 0x14   | LOAD     | Indirect Imm    | –     | Load from (addr + imm)        |
| 0x15   | LOAD     | Indirect Reg    | –     | Load from (addr + reg)        |
| 0x16   | LOAD     | Post-Inc        | –     | Load with post-increment      |
| 0x17   | LOAD     | Post-Inc Reg    | –     | Load with post-inc (reg)      |
| 0x18   | LDEA     | Indirect Imm    | –     | Load effective address        |
| 0x19   | LDEA     | Indirect Reg    | –     | Load effective address (reg)  |
| 0x1A   | BRAN     | Indirect Imm    | –     | Branch (imm displacement)     |
| 0x1B   | BRAN     | Indirect Reg    | –     | Branch (reg displacement)     |
| 0x1C   | LJSR     | Indirect Imm    | –     | Long jump to subroutine       |
| 0x1D   | LJSR     | Indirect Reg    | –     | Long jump to subroutine (reg) |
| 0x1E   | BRSR     | Indirect Imm    | –     | Branch to subroutine          |
| 0x1F   | BRSR     | Indirect Reg    | –     | Branch to subroutine (reg)    |
|        |          |                 |       |                               |
| 0x20   | ADDI     | Short Imm+Shift | NZCV  | Add short immediate           |
| 0x21   | ADCI     | Short Imm+Shift | NZCV  | Add short imm with carry      |
| 0x22   | SUBI     | Short Imm+Shift | NZCV  | Subtract short immediate      |
| 0x23   | SBCI     | Short Imm+Shift | NZCV  | Subtract short imm with carry |
| 0x24   | ANDI     | Short Imm+Shift | NZ    | AND short immediate           |
| 0x25   | ORRI     | Short Imm+Shift | NZ    | OR short immediate            |
| 0x26   | XORI     | Short Imm+Shift | NZ    | XOR short immediate           |
| 0x27   | LOAD     | Short Imm+Shift | –     | Undocumented                  |
| 0x28   | –        | –               | –     | Undocumented                  |
| 0x29   | –        | –               | –     | Undocumented                  |
| 0x2A   | CMPI     | Short Imm+Shift | NZCV  | Compare short immediate       |
| 0x2B   | –        | –               | –     | Undocumented                  |
| 0x2C   | TSAI     | Short Imm+Shift | NZ    | Test AND short immediate      |
| 0x2D   | –        | –               | –     | Undocumented                  |
| 0x2E   | TSXI     | Short Imm+Shift | NZ    | Test XOR short immediate      |
| 0x2F   | COPI     | Short Imm+Shift | –     | Coprocessor call short imm    |
|        |          |                 |       |                               |
| 0x30   | ADDR     | Register        | NZCV  | Add register                  |
| 0x31   | ADCR     | Register        | NZCV  | Add register with carry       |
| 0x32   | SUBR     | Register        | NZCV  | Subtract register             |
| 0x33   | SBCR     | Register        | NZCV  | Subtract register with carry  |
| 0x34   | ANDR     | Register        | NZ    | AND register                  |
| 0x35   | ORRR     | Register        | NZ    | OR register                   |
| 0x36   | XORR     | Register        | NZ    | XOR register                  |
| 0x37   | LOAD     | Register        | –     | Load from register (move)     |
| 0x38   | –        | –               | –     | Undocumented                  |
| 0x39   | –        | –               | –     | Undocumented                  |
| 0x3A   | CMPR     | Register        | NZCV  | Compare register              |
| 0x3B   | –        | –               | –     | Undocumented                  |
| 0x3C   | TSAR     | Register        | NZ    | Test AND register             |
| 0x3D   | –        | –               | –     | Undocumented                  |
| 0x3E   | TSXR     | Register        | NZ    | Test XOR register             |
| 0x3F   | COPR     | Register        | –     | Coprocessor call register     |

## 11.3 Instruction Grouping Patterns

### 11.3.1 Opcode Organization

The opcode space is systematically organized:

- **0x0\_\_**: Immediate operand format
- **0x1\_\_**: Memory and control flow operations
- **0x2\_\_**: Short immediate with shift format
- **0x3\_\_**: Register operand format

### 11.3.2 Save vs. Test Variants

Many ALU instructions have two variants:

- **+0x00 to +0x07**: Save result to destination register
- **+0x08 to +0x0F**: Test only (update flags, discard result)

Examples:

- 0x04 = **ANDI** (save result), 0x0C = **TSAI** (test only)
- 0x06 = **XORI** (save result), 0x0E = **TSXI** (test only)

## 11.4 Meta-Instructions

Several commonly-used operations are implemented as meta-instructions (pseudo-instructions) that assemble to specific opcodes:

| Meta-Instruction | Assembles To            | Purpose                |
|------------------|-------------------------|------------------------|
| <b>NOOP</b>      | <b>ADDI r1, #0</b>      | No operation           |
| <b>RETS</b>      | <b>LDEA p, (#0, 1)</b>  | Return from subroutine |
| <b>WAIT</b>      | <b>COPI r1, #0x1900</b> | Wait for interrupt     |
| <b>RETE</b>      | <b>COP #0x1A00</b>      | Return from exception  |

Table 11.2: Meta-Instructions

## 11.5 Instruction Timing

All instructions use exactly 6 execution phases and complete in 6 clock cycles when no bus wait states are required:

1. Instruction Fetch (High Word)
2. Instruction Fetch (Low Word)

3. Decode and Register Fetch
4. Execute/Address Calculate
5. Memory Access (or NOP)
6. Write Back

This fixed sequence simplifies hardware design. Delayed bus acknowledgement may hold a phase and add wait cycles.

## 11.6 Next Chapters

The following chapters provide detailed documentation for each instruction group:

- **Chapter 12:** ALU instructions (arithmetic, logical, compare)
- **Chapter 13:** Load and store operations
- **Chapter 14:** Branches, jumps, and address operations
- **Chapter 15:** Coprocessor interface
- **Chapter 16:** Meta-instructions and pseudo-ops

# Chapter 12

## ALU Instructions

### 12.1 Overview

The ALU (Arithmetic Logic Unit) instructions perform arithmetic and logical operations on register operands. All ALU instructions follow the load/store architecture principle: they operate only on registers, not directly on memory.

ALU instructions are available in three format variants:

- **Immediate:** Operation with 16-bit constant
- **Short Immediate with Shift:** Operation with 8-bit constant and optional shift
- **Register:** Operation between registers with optional shift

### 12.2 ALU Instruction Families

#### 12.2.1 Arithmetic Instructions

- **ADD** – Addition
- **ADC** – Add with Carry
- **SUB** – Subtraction
- **SBC** – Subtract with Carry (Borrow)

#### 12.2.2 Logical Instructions

- **AND** – Bitwise AND
- **ORR** – Bitwise OR
- **XOR** – Bitwise XOR (Exclusive OR)

#### 12.2.3 Data Movement

- **LOAD** – Load immediate or copy register

### 12.2.4 Test and Compare

- **CMP** – Compare (subtract without saving result)
- **TSA** – Test AND (AND without saving result)
- **TSX** – Test XOR (XOR without saving result)

## 12.3 Legal Forms

ALU instructions are register-only operations. Immediate operands are encoded in the instruction word; memory operands are not legal for this instruction family.

| Family | Immediate       | Short Immediate       | Register                   | Writes Result |
|--------|-----------------|-----------------------|----------------------------|---------------|
| ADD    | ADDI rD, #imm16 | ADDI rD, #imm8, shift | ADDR rD, rS1, rS2[, shift] | Yes           |
| ADC    | ADCI rD, #imm16 | ADCI rD, #imm8, shift | ADCR rD, rS1, rS2[, shift] | Yes           |
| SUB    | SUBI rD, #imm16 | SUBI rD, #imm8, shift | SUBR rD, rS1, rS2[, shift] | Yes           |
| SBC    | SBCI rD, #imm16 | SBCI rD, #imm8, shift | SBCR rD, rS1, rS2[, shift] | Yes           |
| AND    | ANDI rD, #imm16 | ANDI rD, #imm8, shift | ANDR rD, rS1, rS2[, shift] | Yes           |
| ORR    | ORRI rD, #imm16 | ORRI rD, #imm8, shift | ORRR rD, rS1, rS2[, shift] | Yes           |
| XOR    | XORI rD, #imm16 | XORI rD, #imm8, shift | XORR rD, rS1, rS2[, shift] | Yes           |
| CMP    | CMPI rS, #imm16 | CMPI rS, #imm8, shift | CMPR rS1, rS2[, shift]     | No            |
| TSA    | TSAI rS, #imm16 | TSAI rS, #imm8, shift | TSAR rS1, rS2[, shift]     | No            |
| TSX    | TSXI rS, #imm16 | TSXI rS, #imm8, shift | TSXR rS1, rS2[, shift]     | No            |

Table 12.1: Legal ALU Instruction Forms

**Register shorthand:** When a register-form ALU instruction omits **rS1**, the assembler encodes **rD** as both the destination and first source operand. For example, **ADDR r1, r2** is encoded as **ADDR r1, r1, r2**.

**Short-immediate LOAD:** Opcode 0x27 is a valid CPU encoding, but the assembler does not expose a **LOAD rD, #imm8, shift** form. In the CPU datapath the short-immediate shift applies to the destination register side of the ALU input, while **LOAD** writes the unshifted immediate operand. The encoded shift therefore has no useful effect on the loaded value.

## 12.4 Common ALU Semantics

| Topic                  | Definition                                                                                                                                                             |
|------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Condition false        | The instruction is skipped and has no side effects. Registers and status flags are preserved.                                                                          |
| Immediate format       | The left operand is the named register. The right operand is the encoded 16-bit immediate.                                                                             |
| Short immediate format | The left operand is the named register after the encoded shift is applied. The right operand is the encoded 8-bit immediate.                                           |
| Register format        | The left operand is <b>rS1</b> after the encoded shift is applied. The right operand is <b>rS2</b> .                                                                   |
| Test variants          | <b>CMP</b> , <b>TSA</b> , and <b>TSX</b> compute a result for flag updates but do not write it back.                                                                   |
| LOAD                   | <b>LOAD</b> writes the selected operand to <b>rD</b> and does not update status flags.                                                                                 |
| Exceptions             | ALU instructions do not access memory and do not raise bus, protection, or alignment faults. Invalid or reserved encodings raise the normal invalid-instruction fault. |

Table 12.2: Common ALU Semantics

## 12.5 Status Flag Updates

Most ALU instructions update the status register flags:

**Arithmetic (ADD, ADC, SUB, SBC, CMP)** Update all four flags: N, Z, C, V

**Logical (AND, ORR, XOR, TSA, TSX)** Update N and Z; clear C and V

**Load (LOAD)** Do not update flags

| Family | N | Z | C | V | Default AF | Meaning                                                            |
|--------|---|---|---|---|------------|--------------------------------------------------------------------|
| ADD    | * | * | * | * | [A]        | C is carry out; V is signed overflow.                              |
| ADC    | * | * | * | * | [A]        | Adds incoming C; C is carry out.                                   |
| SUB    | * | * | * | * | [A]        | C is set when a borrow occurs.                                     |
| SBC    | * | * | * | * | [A]        | Subtracts incoming C as borrow input; C is set on outgoing borrow. |
| CMP    | * | * | * | * | [A]        | Same flags as SUB; result is discarded.                            |
| AND    | * | * | 0 | 0 | [A]        | C and V are cleared.                                               |
| ORR    | * | * | 0 | 0 | [A]        | C and V are cleared.                                               |
| XOR    | * | * | 0 | 0 | [A]        | C and V are cleared.                                               |
| TSA    | * | * | 0 | 0 | [A]        | Same flags as AND; result is discarded.                            |
| TSX    | * | * | 0 | 0 | [A]        | Same flags as XOR; result is discarded.                            |
| LOAD   | - | - | - | - | [N]        | Status flags are preserved.                                        |

Table 12.3: ALU Status Flag Effects

See Table 10.2 for the flag-effect symbol definitions. If [N] is encoded, all status flags are preserved. If [S] is encoded, flags are updated from the shifter result instead of the ALU result; see Chapter 8 for the shifter flag rules.

### 12.5.1 Manual Flag Update Control

ALU instructions support manual control over how status flags are updated using the status override syntax. See Section 8.12 in Chapter 8 for complete details.

- INSTR[A] – Update flags from ALU result (default behavior)
- INSTR[S] – Update flags from shift result instead of ALU
- INSTR[N] – Do not update status flags

#### Example:

```

1 ; Update flags from shift operation
2 ADDI [S] r1, #0, LSL #3          ; Flags reflect (r1 << 3), not the ALU
3
4 ; Preserve flags during operation
5 CMPI r2, r3                    ; Set comparison flags
6 ADDI [N] r4, #1                 ; Increment without changing flags
7 BRAN |>= branch_if_r2_gte_r3    ; Use flags from CMPI

```

## 12.6 ADD – Addition

### ADDI / ADDR – Add Immediate / Add Register

**Opcodes:** 0x00 (Imm), 0x20 (Short Imm), 0x30 (Reg)

**Syntax:**

```

1 ADDI rD, #imm16                ; rD = rD + imm16
2 ADDI rD, #imm8, shift          ; rD = (rD << shift) + imm8
3 ADDR rD, rS1, rS2              ; rD = rS1 + rS2
4 ADDR rD, rS1, rS2, shift       ; rD = (rS1 << shift) + rS2

```

**Operation:**

```

result = operand1 + operand2
destination = result
SR.N = result[15]
SR.Z = (result == 0)
SR.C = carry_out
SR.V = signed_overflow

```

**Description:**

Adds two values and stores the result in the destination register. In short immediate format with shift, the destination register is shifted before adding the immediate. The carry flag is set if an unsigned overflow occurs (carry out of bit 15). The overflow flag is set if a signed overflow occurs (the sign of the result is incorrect for the operation).

**Flags:** N Z C V

**Example:**

```

1 ; Simple addition
2 ADDI r1, #100                ; r1 = r1 + 100
3
4 ; Add two registers
5 ADDR r3, r1, r2              ; r3 = r1 + r2
6
7 ; Scaled addition (multiply by 3)
8 ADDR r4, r5, r5, LSL #1      ; r4 = r5 + (r5 * 2) = r5 * 3
9
10 ; Build large constant
11 ADDI r1, #1, LSL #12        ; r1 = (r1 << 12) + 1

```

**Notes:**

- For multi-precision arithmetic, use **ADC** to propagate carry
- Adding zero (**ADDI** r1, #0) can be used to move values or test flags
- Shift operations allow efficient constant multiplication

## 12.7 ADC – Add with Carry

### ADCI / ADCR – Add with Carry

**Opcodes:** 0x01 (Imm), 0x21 (Short Imm), 0x31 (Reg)

**Syntax:**

|   |                                 |                                              |
|---|---------------------------------|----------------------------------------------|
| 1 | <b>ADCI</b> rD, #imm16          | <i>; rD = rD + imm16 + C</i>                 |
| 2 | <b>ADCI</b> rD, #imm8, shift    | <i>; rD = (rD &lt;&lt; shift) + imm8 + C</i> |
| 3 | <b>ADCR</b> rD, rS1, rS2        | <i>; rD = rS1 + rS2 + C</i>                  |
| 4 | <b>ADCR</b> rD, rS1, rS2, shift | <i>; rD = (rS1 &lt;&lt; shift) + rS2 + C</i> |

**Operation:**

result = operand1 + operand2 + SR.C

destination = result

SR.N = result[15]

SR.Z = (result == 0)

SR.C = carry\_out

SR.V = signed\_overflow

**Description:**

Adds two values plus the carry flag and stores the result. In short immediate format with shift, the destination register is shifted before adding the immediate. This instruction is used for multi-precision (32-bit, 48-bit, etc.) arithmetic to propagate the carry from lower-order additions to higher-order additions.

**Flags:** N Z C V

**Example:**

```

1 ; 32-bit addition (r1:r2 = r3:r4 + r5:r6)
2 ; Low words
3 ADDR r2, r4, r6           ; r2 = r4 + r6, set carry
4
5 ; High words (with carry propagation)
6 ADCR r1, r3, r5           ; r1 = r3 + r5 + carry

```

**Notes:**

- Always use after **ADD** for multi-word arithmetic
- The carry flag must be in the correct state before **ADC**
- Can be used to add 1 conditionally based on carry flag

## 12.8 SUB – Subtraction

### SUBI / SUBR – Subtract Immediate / Subtract Register

**Opcodes:** 0x02 (Imm), 0x22 (Short Imm), 0x32 (Reg)

**Syntax:**

|   |                                 |                             |
|---|---------------------------------|-----------------------------|
| 1 | <b>SUBI</b> rD, #imm16          | ; rD = rD - imm16           |
| 2 | <b>SUBI</b> rD, #imm8, shift    | ; rD = (rD << shift) - imm8 |
| 3 | <b>SUBR</b> rD, rS1, rS2        | ; rD = rS1 - rS2            |
| 4 | <b>SUBR</b> rD, rS1, rS2, shift | ; rD = (rS1 << shift) - rS2 |

**Operation:**

```
result = operand1 - operand2
destination = result
SR.N = result[15]
SR.Z = (result == 0)
SR.C = borrow
SR.V = signed_overflow
```

**Description:**

Subtracts the second operand from the first and stores the result. In short immediate format with shift, the destination register is shifted before subtracting the immediate. The carry flag is set if a borrow occurs (operand2 > operand1 for unsigned). The overflow flag indicates signed overflow.

**Flags:** N Z C V

**Example:**

```
1 ; Decrement by 1
2 SUBI r1, #1           ; r1 = r1 - 1
3
4 ; Subtract registers
5 SUBR r3, r1, r2       ; r3 = r1 - r2
6
7 ; Loop counter
8 SUBI r7, #1           ; Decrement counter
9 BRAN != loop         ; Branch if not zero
```

**Notes:**

- Carry is set when the subtraction requires a borrow
- For multi-precision subtraction, use **SBC**
- Subtracting a value from itself (**SUBR** r1, r1, r1) clears the register

## 12.9 SBC – Subtract with Carry (Borrow)

### SBCI / SBCR – Subtract with Carry

**Opcodes:** 0x03 (Imm), 0x23 (Short Imm), 0x33 (Reg)

**Syntax:**

|   |                                 |                                 |
|---|---------------------------------|---------------------------------|
| 1 | <b>SBCI</b> rD, #imm16          | ; rD = rD - imm16 - C           |
| 2 | <b>SBCR</b> rD, rS1, rS2        | ; rD = rS1 - rS2 - C            |
| 3 | <b>SBCR</b> rD, rS1, rS2, shift | ; rD = (rS1 << shift) - rS2 - C |

**Operation:**

```
result = operand1 - operand2 - SR.C
destination = result
SR.N = result[15]
SR.Z = (result == 0)
SR.C = borrow
SR.V = signed_overflow
```

**Description:**

Subtracts the second operand and borrow from the first operand. In short immediate format with shift, the destination register is shifted before subtracting the immediate. Used for multi-precision subtraction to propagate borrows from lower-order to higher-order subtractions.

**Flags:** N Z C V

**Example:**

```
1 ; 32-bit subtraction (r1:r2 = r3:r4 - r5:r6)
2 ; Low words
3 SUBR r2, r4, r6 ; r2 = r4 - r6, set/clear carry
4
5 ; High words (with borrow propagation)
6 SBCR r1, r3, r5 ; r1 = r3 - r5 - borrow
```

**Notes:**

- Always use after **SUB** for multi-word subtraction
- Carry is treated as the borrow input for multi-word subtraction

## 12.10 AND – Bitwise AND

### ANDI / ANDR – AND Immediate / AND Register

**Opcodes:** 0x04 (Imm), 0x24 (Short Imm), 0x34 (Reg)

**Syntax:**

|   |                                 |                                              |
|---|---------------------------------|----------------------------------------------|
| 1 | <b>ANDI</b> rD, #imm16          | <i>; rD = rD &amp; imm16</i>                 |
| 2 | <b>ANDI</b> rD, #imm8, shift    | <i>; rD = (rD &lt;&lt; shift) &amp; imm8</i> |
| 3 | <b>ANDR</b> rD, rS1, rS2        | <i>; rD = rS1 &amp; rS2</i>                  |
| 4 | <b>ANDR</b> rD, rS1, rS2, shift | <i>; rD = (rS1 &lt;&lt; shift) &amp; rS2</i> |

**Operation:**

result = operand1 AND operand2

destination = result

SR.N = result[15]

SR.Z = (result == 0)

SR.C = 0

SR.V = 0

**Description:**

Performs bitwise AND operation. Each bit in the result is 1 only if the corresponding bits in both operands are 1. In short immediate format with shift, the destination register is shifted before ANDing with the immediate. Commonly used for bit masking and clearing specific bits.

**Flags:** N Z (C and V cleared)

**Example:**

```

1 ; Mask lower byte
2 ANDI r1, #0x00FF           ; r1 = r1 & 0xFF
3
4 ; Clear bit 5
5 ANDI r2, # ~(1<<5)         ; r2 = r2 & 0xFFDF
6
7 ; Check if two registers share any set bits
8 ANDR r3, r1, r2             ; r3 = r1 & r2
9 CMPI r3, #0
10 BRAN != common_bits       ; Branch if any bits in common

```

**Notes:**

- ANDing with 0xFFFF leaves value unchanged
- ANDing with 0x0000 clears the register
- Use **TSA** to test bits without modifying the register

## 12.11 ORR – Bitwise OR

### ORRI / ORRR – OR Immediate / OR Register

**Opcodes:** 0x05 (Imm), 0x25 (Short Imm), 0x35 (Reg)

**Syntax:**

|   |                                 |                                          |
|---|---------------------------------|------------------------------------------|
| 1 | <b>ORRI</b> rD, #imm16          | <i>; rD = rD   imm16</i>                 |
| 2 | <b>ORRI</b> rD, #imm8, shift    | <i>; rD = (rD &lt;&lt; shift)   imm8</i> |
| 3 | <b>ORRR</b> rD, rS1, rS2        | <i>; rD = rS1   rS2</i>                  |
| 4 | <b>ORRR</b> rD, rS1, rS2, shift | <i>; rD = (rS1 &lt;&lt; shift)   rS2</i> |

**Operation:**

result = operand1 OR operand2

destination = result

SR.N = result[15]

SR.Z = (result == 0)

SR.C = 0

SR.V = 0

**Description:**

Performs bitwise OR operation. Each bit in the result is 1 if either corresponding bit in the operands is 1. In short immediate format with shift, the destination register is shifted before ORing with the immediate. Commonly used for setting specific bits.

**Flags:** N Z (C and V cleared)

**Example:**

```

1 ; Set bit 7
2 ORRI r1, #0x0080          ; r1 = r1 | 0x80
3
4 ; Set multiple bits
5 ORRI r2, #0xF000          ; Set upper nibble
6
7 ; Combine two registers
8 ORRR r3, r1, r2           ; r3 = r1 | r2
9
10 ; Set bit N (variable)
11 ADDI r4, #1
12 ADDR r4, r4, r5, LSL      ; r4 = 1 << r5
13 ORRR r1, r4               ; r1 |= (1 << r5)

```

**Notes:**

- ORing with 0x0000 leaves value unchanged
- ORing with 0xFFFF sets all bits
- Use for enabling flag bits or combining bit fields

## 12.12 XOR – Bitwise Exclusive OR

### XORI / XORR – XOR Immediate / XOR Register

**Opcodes:** 0x06 (Imm), 0x26 (Short Imm), 0x36 (Reg)

**Syntax:**

```

1 XORI rD, #imm16                ; rD = rD ^ imm16
2 XORI rD, #imm8, shift          ; rD = (rD << shift) ^ imm8
3 XORR rD, rS1, rS2              ; rD = rS1 ^ rS2
4 XORR rD, rS1, rS2, shift      ; rD = (rS1 << shift) ^ rS2

```

**Operation:**

result = operand1 XOR operand2

destination = result

SR.N = result[15]

SR.Z = (result == 0)

SR.C = 0

SR.V = 0

**Description:**

Performs bitwise XOR (exclusive OR) operation. Each bit in the result is 1 if the corresponding bits in the operands are different. In short immediate format with shift, the destination register is shifted before XORing with the immediate. Used for toggling bits, bitwise comparison, and simple encryption.

**Flags:** N Z (C and V cleared)

**Example:**

```

1 ; Toggle bit 3
2 XORI r1, #0x0008                ; r1 = r1 ^ 0x08
3
4 ; Invert all bits
5 XORI r2, #0xFFFF                ; r2 = ~r2
6
7 ; Check if registers are equal
8 XORR r3, r1, r2                  ; r3 = r1 ^ r2
9 ; If r3 == 0, then r1 == r2
10
11 ; Swap using XOR (no temporary)
12 XORR r1, r1, r2                  ; r1 = r1 ^ r2
13 XORR r2, r1, r2                  ; r2 = r1 ^ r2 = (r1 ^ r2) ^ r2 = r1
14 XORR r1, r1, r2                  ; r1 = (r1 ^ r2) ^ r1 = r2

```

**Notes:**

- XORing with 0x0000 leaves value unchanged
- XORing with 0xFFFF inverts all bits (bitwise NOT)
- XORing a value with itself always gives 0
- Use **TSX** to check for differences without modifying register

## 12.13 LOAD – Load Immediate / Move Register

### LOAD – Load/Move

**Opcodes:** 0x07 (Imm), 0x27 (Short Imm), 0x37 (Reg)

**Syntax:**

```
1 LOAD rD, #imm16           ; rD = imm16 (via add)
2 LOAD rD, rS               ; rD = rS (via add)
```

**Operation:**

; Implemented as:  $rD = 0 + \text{operand}$   
 destination = operand  
 ; Flags NOT updated

**Description:**

Loads an immediate value into a register or copies a register value. This is actually implemented as an ADD with an implicit zero operand, but flags are not updated. This is the primary way to load constants or move data between registers. The CPU defines a short-immediate **LOAD** encoding at opcode 0x27, but the assembler does not expose it because its shift field does not affect the loaded value.

**Flags:** None

**Example:**

```
1 ; Load constant
2 LOAD r1, #1234           ; r1 = 1234
3
4 ; Copy register
5 LOAD r2, r1              ; r2 = r1
6
7 ; Load with arithmetic
8 LOAD r3, #1
9 ADDI r3, #0, LSL #8      ; r3 = (r3 << 8) + 0 = 256
10
11 ; Zero a register
12 LOAD r4, #0             ; r4 = 0
13
14 ; Load large constant (combine)
15 LOAD r5, #0x12
16 ADDI r5, #0, LSL #8      ; r5 = (r5 << 8) + 0 = 0x1200
17 ORRI r5, #0x34          ; r5 = 0x1234
```

**Notes:**

- Does not update flags, unlike **ADD**
- Most efficient way to move data between registers
- For 16-bit constants, use immediate form
- For larger or shifted constants, use short immediate with shift



## 12.14 CMP – Compare

### CMPI / CMPR – Compare Immediate / Compare Register

**Opcodes:** 0x0A (Imm), 0x2A (Short Imm), 0x3A (Reg)

**Syntax:**

```

1 CMPI rS, #imm16                ; Compare rS with imm16
2 CMPI rS, #imm8, shift          ; Compare (rS << shift) with imm8
3 CMPR rS1, rS2                  ; Compare rS1 with rS2
4 CMPR rS1, rS2, shift           ; Compare (rS1 << shift) with rS2

```

**Operation:**

```

result = operand1 - operand2
; Result is discarded, only flags updated
SR.N = result[15]
SR.Z = (result == 0)
SR.C = borrow
SR.V = signed_overflow

```

**Description:**

Performs subtraction but discards the result, only updating the status flags. In short immediate format with shift, the source register is shifted before comparing with the immediate. This is the primary instruction for implementing conditional execution and branches. The flags can be tested with condition codes to determine the relationship between the operands.

**Flags:** N Z C V

**Example:**

```

1 ; Test if r1 is zero
2 CMPI r1, #0
3 BRAN|== is_zero                ; Branch if r1 == 0
4
5 ; Test if r1 > r2 (unsigned)
6 CMPR r1, r2
7 BRAN|HI r1_greater
8
9 ; Test if r1 >= r2 (signed)
10 CMPR r1, r2
11 BRAN|>= r1_greater_or_equal
12
13 ; Range check: if (r1 >= 10 && r1 < 20)
14 CMPI r1, #10
15 BRAN|<< out_of_range          ; Branch if r1 < 10
16 CMPI r1, #20
17 BRAN|>= out_of_range          ; Branch if r1 >= 20
18 ; r1 is in range [10, 20)

```

**Notes:**

- Does not modify any register, only flags
- Use with conditional instructions or branches
- Remember: **CMP** A, B computes A - B
- For signed comparisons, use >=, <, », «
- For unsigned comparisons, use >, CS, CC, <=



## 12.15 TSA – Test AND

### TSAI / TSAR – Test AND Immediate / Test AND Register

**Opcodes:** 0x0C (Imm), 0x2C (Short Imm), 0x3C (Reg)

**Syntax:**

```

1 TSAI rS, #imm16                ; Test rS & imm16
2 TSAI rS, #imm8, shift          ; Test (rS << shift) & imm8
3 TSAR rS1, rS2                  ; Test rS1 & rS2
4 TSAR rS1, rS2, shift           ; Test (rS1 << shift) & rS2

```

**Operation:**

```

result = operand1 AND operand2
; Result is discarded, only flags updated
SR.N = result[15]
SR.Z = (result == 0)
SR.C = 0
SR.V = 0

```

**Description:**

Performs bitwise AND but discards the result, only updating flags. In short immediate format with shift, the source register is shifted before ANDing with the immediate. Used to test if specific bits are set without modifying the register. The zero flag indicates whether any tested bits were set.

**Flags:** N Z (C and V cleared)

**Example:**

```

1 ; Test if bit 5 is set
2 TSAI r1, #0x0020                ; Test r1 & 0x20
3 BRAN != bit5_set                ; Branch if bit 5 was set
4
5 ; Test if any upper byte bits are set
6 TSAI r2, #0xFF00
7 BRAN != has_upper_bits
8
9 ; Test if r1 and r2 have any bits in common
10 TSAR r1, r2
11 BRAN == no_common_bits          ; Branch if no bits in common
12
13 ; Test multiple bits
14 TSAI r3, #0xF000                ; Test upper nibble
15 BRAN == upper_clear              ; Branch if all tested bits clear

```

**Notes:**

- Does not modify any register, only flags
- Z=1 means all tested bits were 0
- Z=0 means at least one tested bit was 1
- Use for bit testing without side effects

## 12.16 TSX – Test XOR

### TSXI / TSXR – Test XOR Immediate / Test XOR Register

**Opcodes:** 0x0E (Imm), 0x2E (Short Imm), 0x3E (Reg)

**Syntax:**

```

1 TSXI rS, #imm16           ; Test rS ^ imm16
2 TSXI rS, #imm8, shift    ; Test (rS << shift) ^ imm8
3 TSXR rS1, rS2           ; Test rS1 ^ rS2
4 TSXR rS1, rS2, shift    ; Test (rS1 << shift) ^ rS2

```

**Operation:**

```

result = operand1 XOR operand2
; Result is discarded, only flags updated
SR.N = result[15]
SR.Z = (result == 0)
SR.C = 0
SR.V = 0

```

**Description:**

Performs bitwise XOR but discards the result, only updating flags. In short immediate format with shift, the source register is shifted before XORing with the immediate. Used to test if bits match a specific pattern or to compare values for equality. Z=1 indicates the values are identical.

**Flags:** N Z (C and V cleared)

**Example:**

```

1 ; Test if r1 equals specific value
2 TSXI r1, #0x1234           ; Test r1 ^ 0x1234
3 BRAN |== is_0x1234        ; Branch if r1 == 0x1234
4
5 ; Test if two registers are equal
6 TSXR r1, r2               ; Test r1 ^ r2
7 BRAN |== registers_equal   ; Branch if r1 == r2
8
9 ; Test if bit pattern matches
10 TSXI r3, #0xFF           ; Mask to lower byte
11 TSXI r3, #0xAA           ; Test if lower byte == 0xAA
12 BRAN |== pattern_match

```

**Notes:**

- Does not modify any register, only flags
- Z=1 means values are identical (all bits match)
- Can be used as equality test (alternative to CMP for == only)
- More efficient than XOR + CMP for simple equality tests

## 12.17 Summary

The ALU instructions form the computational core of the SIRCIS instruction set. Key points:

- All ALU operations work only on registers (load/store architecture)
- Three format variants provide flexibility (immediate, short+shift, register)
- Test variants (CMP, TSA, TSX) allow non-destructive testing
- Shift operations can be combined with ALU ops for powerful single-cycle operations
- Consistent flag behavior makes conditional execution predictable

## Chapter 13

# Memory Access Instructions

### 13.1 Overview

Memory access instructions transfer data between registers and memory. Following the load/store architecture, these are the only instructions that can access memory—all arithmetic and logical operations must work on registers.

The SIRC-1 provides two primary memory operations:

- **LOAD** – Read from memory into a register
- **STOR** – Write from a register to memory

Both instructions support multiple addressing modes:

- Indirect with immediate offset: (**#offset**, **addr**)
- Indirect with register offset: (**reg**, **addr**)
- Post-increment: (**#offset**, **addr**)**+**
- Pre-decrement: **-(#offset, addr)**

**Important:** When using indirect addressing modes (with memory access), **LOAD** and **STOR** instructions support optional shift operations on the data being loaded or stored. See Section 13.6 for details. Note that shifts are **not** supported when loading immediate values or copying register values directly.

### 13.2 Effective Address Calculation

For all memory operations, the effective address (EA) is calculated as:

$$EA = \text{AddressRegister} + \text{Offset} \quad (13.1)$$

Where:

- Address Register is one of: **l**, **a**, **s**, or **p**
- Offset is either an immediate value or register contents

The resulting 24-bit address is used to access memory.

## 13.3 LOAD Instructions

### LOAD – Load from Memory

**Opcodes:** 0x14 (Indirect Imm), 0x15 (Indirect Reg), 0x16 (Post-Inc Imm), 0x17 (Post-Inc Reg)

**Syntax:**

```

1  LOAD rD, (#offset, addr)      ; rD = memory[addr + offset]
2  LOAD rD, (rS, addr)          ; rD = memory[addr + rS]
3  LOAD rD, (#offset, addr)+    ; rD = memory[addr + offset]; addr +=
    1
4  LOAD rD, (rS, addr)+          ; rD = memory[addr + rS]; addr += 1
5
6  ; With optional shift (memory addressing modes only)
7  LOAD rD, (#offset, addr), shift ; Load and shift
8  LOAD rD, (rS, addr), shift     ; Load and shift
9  LOAD rD, (#offset, addr)+, shift ; Load, shift, then increment
10 LOAD rD, (rS, addr)+, shift    ; Load, shift, then increment

```

**Description:**

Loads a 16-bit value from memory into the destination register. Post-increment forms update the address register after the load, useful for array traversal.

**Examples:**

```

1  ; Load from fixed offset
2  LOAD r1, (#4, a)              ; r1 = memory[a + 4]
3
4  ; Load using variable offset
5  LOAD r2, (r3, a)              ; r2 = memory[a + r3]
6
7  ; Array iteration
8  LOAD r4, (#0, a)+             ; r4 = *a; a += 1
9
10 ; Stack pop
11 LOAD r5, (#0, s)+             ; r5 = *s; s += 1
12
13 ; Load and shift (for bit manipulation)
14 LOAD r3, (r2, a), LSL #2      ; r3 = memory[a + r2] << 2
15 LOAD r6, (#0, a), ASR #2      ; r6 = memory[a] >> 2 (signed)

```

## 13.4 STOR Instructions

### STOR – Store to Memory

**Opcodes:** 0x10 (Indirect Imm), 0x11 (Indirect Reg), 0x12 (Pre-Dec Imm), 0x13 (Pre-Dec Reg)

**Syntax:**

```

1 STOR (#offset, addr), rS      ; memory[addr + offset] = rS
2 STOR (rD, addr), rS          ; memory[addr + rD] = rS
3 STOR -(#offset, addr), rS     ; addr -= 1; memory[addr + offset] =
    rS
4 STOR -(rD, addr), rS          ; addr -= 1; memory[addr + rD] = rS
5
6 ; With optional shift (memory addressing modes only)
7 STOR (#offset, addr), rS, shift ; Shift and store
8 STOR (rD, addr), rS, shift      ; Shift and store
9 STOR -(#offset, addr), rS, shift ; Decrement, shift, and store
10 STOR -(rD, addr), rS, shift     ; Decrement, shift, and store

```

**Description:**

Stores a 16-bit value from a register into memory. Pre-decrement forms update the address register before the store, useful for stack push operations.

**Examples:**

```

1 ; Store to fixed offset
2 STOR (#8, a), r1          ; memory[a + 8] = r1
3
4 ; Store using variable offset
5 STOR (r3, a), r2          ; memory[a + r3] = r2
6
7 ; Store with shift (data shifted before storing)
8 STOR (r2, a), r1, LSL #1   ; memory[a + r2] = r1 << 1
9 STOR (#0, a), r1, LSR #3   ; memory[a] = r1 >> 3
10
11 ; Stack push
12 STOR -(#0, s), r4         ; s -= 1; memory[s] = r4

```

## 13.5 Common Memory Access Patterns

### 13.5.1 Structure Field Access

```

1 ; struct { int16_t x, y, z; } point;
2 ; 'a' points to point
3 LOAD r1, (#0, a)          ; r1 = point.x
4 LOAD r2, (#1, a)          ; r2 = point.y
5 LOAD r3, (#2, a)          ; r3 = point.z

```

### 13.5.2 Array Iteration

```
1 ; Process array of 16-bit values
2 ; 'a' = array base, r7 = count
3 loop:
4     LOAD r1, (#0, a)+          ; Load element, advance
5     ; ... process r1 ...
6     SUBI r7, #1
7     BRAN != loop
```

### 13.5.3 Stack Operations

```
1 ; Push registers
2 STOR -(#0, s), r1
3 STOR -(#0, s), r2
4 STOR -(#0, s), r3
5
6 ; Pop registers (reverse order)
7 LOAD r3, (#0, s)+
8 LOAD r2, (#0, s)+
9 LOAD r1, (#0, s)+
```

## 13.6 Shift Operations with Memory Instructions

When using indirect addressing modes (memory access), both **LOAD** and **STOR** instructions support optional shift operations. This feature allows data to be shifted during the load or store operation, which is particularly useful for bit manipulation and working with the CPU's word-addressed memory architecture.

### 13.6.1 LOAD with Shift

When loading from memory, the data read from memory is shifted **after** being loaded but **before** being written to the destination register:

```
1 LOAD r3, (r2, a), LSL #2      ; r3 = (memory[a + r2]) << 2
2 LOAD r4, (#0, a), LSR #4      ; r4 = (memory[a]) >> 4
3 LOAD r5, (r2, a), ASL #3      ; r5 = (memory[a + r2]) << 3
4 LOAD r6, (#0, a), ASR #2      ; r6 = (memory[a]) >> 2 (sign-extend)
```

### 13.6.2 STOR with Shift

When storing to memory, the source register value is shifted **before** being written to memory. The original register value is **not** modified:

```
1 STOR (r2, a), r1, LSL #1      ; memory[a + r2] = r1 << 1 (r1 unchanged)
2 STOR (#0, a), r1, LSR #3      ; memory[a] = r1 >> 3 (r1 unchanged)
3 STOR (r2, a), r1, ASL #2      ; memory[a + r2] = r1 << 2 (r1 unchanged)
4 STOR (#0, a), r1, ASR #4      ; memory[a] = r1 >> 4 (r1 unchanged)
```

### 13.6.3 Supported Shift Types

All standard shift types are supported:

- **LSL** – Logical Shift Left (fill with zeros)
- **LSR** – Logical Shift Right (fill with zeros)
- **ASL** – Arithmetic Shift Left (same as LSL)
- **ASR** – Arithmetic Shift Right (sign-extend)
- **RTL** – Rotate Left (through carry)
- **RTR** – Rotate Right (through carry)

### 13.6.4 Shift Limitations

**Important:** Shift operations are **only** supported when using indirect addressing modes (accessing memory). The following **LOAD** modes do **not** support shifts:

- Loading immediate values: `LOAD r1, #0x1234` – No shift allowed
- Copying registers: `LOAD r1, r2` – No shift allowed

For these non-memory operations, use regular ALU instructions with shifts instead (e.g., `ADDI r1, #0, LSL #2`).

### 13.6.5 Use Cases

#### Word-Addressed Memory

The SIRC-1 uses word addressing, so shifting during load/store can save cycles when converting between byte and word addresses:

```
1 ; Convert byte offset to word address and load
2 LOAD r3, (r2, a), LSR #1      ; Divide byte offset by 2
```

#### Bit Manipulation

Shifting during memory access enables efficient bit manipulation:

```
1 ; Extract upper byte from memory
2 LOAD r1, (#0, a), LSR #8      ; r1 = (memory[a] >> 8) & 0xFF
3
4 ; Store scaled value
5 STOR (#0, a), r2, LSL #2      ; memory[a] = r2 * 4
```

### Combined with Auto-Increment/Decrement

Shifts work seamlessly with post-increment and pre-decrement modes:

```
1 ; Load, shift, and advance pointer
2 LOAD r3, (r2, a)+, LSL #1      ; r3 = (memory[a + r2]) << 1; a += 1
3
4 ; Decrement, shift, and store
5 STOR -(r2, a), r1, LSR #2      ; a -= 1; memory[a + r2] = r1 >> 2
```

#### 13.6.6 Status Flags

**Note:** Unlike shifts in ALU instructions, shifts performed during **LOAD** and **STOR** operations can **not** update the status register flags. If you need flag updates based on a shift result, use an ALU instruction or the **SHFT** meta-instruction (see Chapter 16).

## Chapter 14

# Control Flow Instructions

### 14.1 Overview

Control flow instructions modify the program counter to change the execution flow. The SIRC-1 provides several types of control flow operations:

- **BRAN** – Branch (unconditional or conditional jump)
- **LJMP** – Long Jump (transfer address register to PC)
- **LJSR** – Long Jump to Subroutine (with link)
- **BRSR** – Branch to Subroutine (relative, with link)
- **LDEA** – Load Effective Address (can be used for jumps)

All control flow instructions can be conditional, allowing efficient implementation of if-then-else and loop constructs.

## 14.2 Branch Instructions

### BRAN – Branch

**Opcodes:** 0x1A (Imm Displacement), 0x1B (Reg Displacement)

**Syntax:**

```
1 BRAN (#disp, p)           ; PC = PC + disp
2 BRAN (rS, p)              ; PC = PC + rS
3 BRAN|cond (#disp, p)      ; Conditional branch
```

**Description:**

Branches to a new location by adding a displacement to the program counter. The displacement can be an immediate value (for fixed jumps) or a register value (for computed jumps). Branches are typically PC-relative. **Examples:**

```
1 ; Unconditional branch forward
2 BRAN (#20, p)           ; Jump ahead 20 words
3
4 ; Backward branch (loop)
5 loop:
6     ; ... loop body ...
7     BRAN|!= (#-16, p)    ; Jump back if not zero
8
9 ; Conditional branch
10 CMPI r1, #10
11 BRAN|>= (#8, p)        ; Branch if r1 >= 10
```

## 14.3 Long Jump

### LJMP – Long Jump

**Opcodes:** 0x18 (Imm Offset), 0x19 (Reg Offset)

**Syntax:**

```

1 LJMP src                      ; PC = src
2 LJMP src, #offset             ; PC = src + offset
3 LJMP src, rS                  ; PC = src + rS

```

**Description:**

Jumps to an absolute address by transferring the contents of an address register to the program counter. This is a meta-instruction that assembles to **LDEA** with **p** as the hardcoded destination. Unlike **BRAN** which uses PC-relative addressing, **LJMP** performs absolute jumps using address registers.

The instruction supports optional immediate or register offsets. Note that shift operations cannot be applied to the displacement operand for this instruction.

**Privilege:** Privileged instruction (like **LDEA**).

**Examples:**

```

1 ; Simple absolute jump
2 LOAD ah, #0x0040
3 LOAD al, #0x0000                ; a = 0x00400000
4 LJMP a                          ; Jump to 0x00400000
5
6 ; Jump with immediate offset
7 LJMP a, #16                     ; Jump to a + 16
8
9 ; Jump with register offset (jump table)
10 LOAD r1, #8                    ; Table index * 2
11 LJMP b, r1                     ; Jump to b + r1

```

## 14.4 Subroutine Call Instructions

### LJSR – Long Jump to Subroutine

**Opcodes:** 0x1C (Imm), 0x1D (Reg)

**Syntax:**

```

1 LJSR src                      ; l = PC; PC = src
2 LJSR src, #offset             ; l = PC; PC = src + offset
3 LJSR src, rS                  ; l = PC; PC = src + rS

```

**Description:**

Calls a subroutine by saving the return address to the link register (l) and jumping to the target address. The return address can later be restored using **RETS**. Supports optional immediate or register offsets, similar to **LJMP**.

**Examples:**

```

1 ; Call function at address in 'a'
2 LOAD ah, #0x0040
3 LOAD al, #0x0000          ; a = 0x00400000
4 LJSR a                    ; Call 0x00400000
5
6 ; Call with offset
7 LJSR a, #function_offset  ; Call a + offset
8
9 ; Call with register displacement (function pointer table)
10 LOAD r1, #8
11 LJSR b, r1                ; Call b + r1
12
13 ; Return from subroutine
14 RETS                     ; Assembles to: LDEA p, (#0, l)

```

## 14.5 Load Effective Address

### LDEA – Load Effective Address

**Opcodes:** 0x18 (Imm), 0x19 (Reg)

**Syntax:**

```
1 LDEA dest, (#offset, src)      ; dest = src + offset
2 LDEA dest, (rS, src)          ; dest = src + rS
```

**Description:**

Computes an effective address and loads it into the destination address register pair. Can be used for pointer arithmetic or, when the destination is the program counter, as a jump instruction.

**Examples:**

```
1 ; Pointer arithmetic
2 LDEA a, (#16, a)              ; a = a + 16
3
4 ; Computed jump
5 LDEA p, (r1, p)               ; PC = PC + r1 (jump table)
6
7 ; Return from subroutine
8 LDEA p, (#0, l)               ; PC = l (RETS pseudo-instruction)
```

## 14.6 Control Flow Patterns

### 14.6.1 If-Then-Else

```
1 ; if (r1 > r2) { ... } else { ... }
2 CMPR r1, r2
3 BRAN|<= else_part
4 ; Then part
5 ; ...
6 BRAN (#end, p)
7 else_part:
8 ; Else part
9 ; ...
10 end:
```

### 14.6.2 While Loop

```
1 ; while (r1 > 0) { ... }
2 while_start:
3     CMPI r1, #0
4     BRAN|<= while_end
5     ; Loop body
6     ; ...
7     BRAN (#while_start, p)
8 while_end:
```

### 14.6.3 Switch/Jump Table

```
1 ; Switch on r1 (0-3)
2 LOAD r2, r1                ; r2 = r1
3 ADDI r2, #0, LSL #1        ; r2 = (r2 << 1) = r2 * 2 (word size)
4 LDEA p, (r2, p)            ; Jump via table
5
6 jump_table:
7     DW case0 - jump_table
8     DW case1 - jump_table
9     DW case2 - jump_table
10    DW case3 - jump_table
```

*(Additional control flow details would follow)*

## Chapter 15

# Coprocessor Instructions

### 15.1 Overview

The SIRC-1 CPU includes a coprocessor interface that allows delegation of certain operations to specialized coprocessors. The primary coprocessor is the Exception Unit (Coprocessor 1), which handles exceptions, interrupts, and privileged operations.

**COP – Coprocessor Call****Opcodes:** 0x0F (Imm), 0x2F (Short Imm), 0x3F (Reg)**Syntax:**

|   |                             |                                              |
|---|-----------------------------|----------------------------------------------|
| 1 | <b>COP</b> <b>I</b> #opcode | <i>; Call coprocessor with 16-bit opcode</i> |
| 2 | <b>COP</b> <b>R</b> rS      | <i>; Call coprocessor with register</i>      |
|   | <i>value</i>                |                                              |

**Description:**

Transfers control to a coprocessor. The coprocessor ID and operation code are encoded in the immediate value or register. Coprocessor operations may be privileged.

**Format:**

- Bits 15–12: Coprocessor ID
- Bits 11–0: Operation code

**Examples:**

```

1 ; Wait for interrupt (Exception Unit)
2 COPI #0x1900 ; Coprocessor 1, opcode 0x900
3 ; Assembled as WAIT meta-instruction
4
5 ; Return from exception
6 COPI #0x1A00 ; Coprocessor 1, opcode 0xA00
7 ; Assembled as RETE meta-instruction
8
9 ; User exception (system call)
10 COPI #0x1180 ; Coprocessor 1, opcode 0x180 (vector
    0x80)
11 ; Assembled as EXCP #0x80 meta-instruction

```

**15.2 Exception Unit (Coprocessor 1)**

The Exception Unit handles:

- Faults (abort exceptions, highest priority)
- Hardware exceptions (interrupts, multiple priority levels)
- Software exceptions (traps)
- Privileged operations

### 15.2.1 Common Operations

| Opcode        | Mnemonic | Operation                                 |
|---------------|----------|-------------------------------------------|
| 0x1100-0x11FF | EXCP     | User exception (trap) at vector 0x60-0xFF |
| 0x1900        | WAIT     | Wait for interrupt                        |
| 0x1A00        | RETE     | Return from exception                     |
| 0x1B00        | RSET     | System reset                              |
| 0x1C00-0x1C7F | ETFR     | Transfer from exception link register     |
| 0x1D00-0x1D7F | ETTR     | Transfer to exception link register       |

Table 15.1: Exception Unit Operations

See Chapter 5 for detailed documentation of exception handling and all exception coprocessor instructions.



## Chapter 16

# Meta-Instructions

### 16.1 Overview

Meta-instructions (also called pseudo-instructions) are assembly mnemonics that don't directly correspond to unique opcodes. Instead, they assemble to specific forms of existing instructions, providing more intuitive syntax for common operations.

### 16.2 NOOP – No Operation

**Syntax:** NOOP

**Assembles to:** ADDI r1, #0

**Description:** Does nothing for 6 cycles. Useful for timing delays, instruction padding, or placeholders during development.

**Example:**

```
1 NOOP                ; Wait 6 cycles
2 NOOP
3 NOOP                ; Total 18 cycles delay
```

### 16.3 RETS – Return from Subroutine

**Syntax:** RETS

**Assembles to:** LDEA p, (#0, 1)

**Description:** Returns from a subroutine by copying the link register to the program counter. Used after LJSR or BRSR.

**Example:**

```
1 my_function:
2   ; ... function body ...
3   RETS                ; Return to caller
```

### 16.4 LJMP – Long Jump

**Syntax:** LJMP src [, offset]

**Assembles to:** LDEA p, ... (various forms)

**Description:** Performs an absolute jump by transferring an address register to the program counter. Supports optional immediate or register offsets. Unlike **BRAN** which is PC-relative, **LJMP** uses absolute addressing via address registers.

**Examples:**

```
1 ; Simple jump to address in register
2 LJMP a ; Assembles to: LDEA p, (#0, a)
3
4 ; Jump with immediate offset
5 LJMP a, #16 ; Assembles to: LDEA p, (#16, a)
6
7 ; Jump with register displacement
8 LJMP b, r1 ; Assembles to: LDEA p, (r1, b)
```

## 16.5 EXCP – User Exception (Trap)

**Syntax:** EXCP #vector

**Assembles to:** COPI #0x1100 | vector

**Description:** Triggers a software exception (trap) at the specified user vector (0x60-0xFF). Used by user-mode programs to invoke system calls or request supervisor mode services.

**Example:**

```
1 ; System call to print character
2 LOAD r1, #'A' ; Character to print
3 EXCP #0x80 ; Trap to OS print routine
```

## 16.6 WAIT – Wait for Interrupt

**Syntax:** WAIT

**Assembles to:** COPI r1, #0x1900

**Description:** Puts the CPU into a low-power wait state until an exception occurs. Sets the waiting\_for\_exception flag in the exception unit. Only valid in supervisor mode.

**Example:**

```
1 main_loop:
2 WAIT ; Sleep until interrupt
3 ; ... process interrupt ...
4 BRAN main_loop
```

## 16.7 RETE – Return from Exception

**Syntax:** RETE

**Assembles to:** COPI #0x1A00

**Description:** Returns from an exception handler by restoring the saved program counter and status register. Only valid in supervisor mode.

**Example:**

```

1 exception_handler:
2     ; ... handle exception ...
3     RETE                               ; Return to interrupted code

```

## 16.8 RSET – System Reset

**Syntax:** RSET

**Assembles to:** COPI #0x1B00

**Description:** Performs a software reset of the processor by clearing the status register and jumping to the reset vector. Only valid in supervisor mode.

**Example:**

```

1 fatal_error:
2     ; Log error...
3     RSET                               ; Reset the system

```

## 16.9 ETFR – Exception Transfer From Register

**Syntax:**

```

1 ETFR #n                               ; Transfer both to a and r7
2 ETFR a, #n                            ; Transfer address to a only
3 ETFR r7, #n                           ; Transfer status to r7 only

```

**Assembles to:** COPI #0x1Cxy (where x = register select, y = link register)

**Description:** Copies the return address and/or status register from a link register (0-7) to CPU registers. Used in exception handlers to examine saved processor state. Only valid in supervisor mode.

**Example:**

```

1 alignment_fault_handler:
2     ETFR a, #6                         ; Get faulting return address
3     LOAD al, @fixed_address           ; Correct the address
4     ETTR #6, a                         ; Write corrected address back
5     RETE

```

## 16.10 ETTR – Exception Transfer To Register

**Syntax:**

```

1 ETTR #n                               ; Transfer both a and r7
2 ETTR #n, a                            ; Transfer a to address only
3 ETTR #n, r7                           ; Transfer r7 to status only

```

**Assembles to:** COPI #0x1Dxy (where x = register select, y = link register)

**Description:** Copies CPU registers to a link register's return address and/or status register. Used to modify saved processor state before returning from an exception. Only valid in supervisor mode.

**Example:**

```
1 privilege_fault_handler:
2     ETFR r7, #6                ; Get saved status register
3     ANDI r7, #0xFEFF          ; Clear protected mode bit
4     ETTR #6, r7                ; Save modified status back
5     RETE
```

## 16.11 SHFT – Shift with Status Update

**Syntax:** SHFT rD, shift

**Assembles to:** ORRI[S] rD, #0, shift

**Description:** Shifts the value in a register and updates the status register flags based on the shift result (not the ALU operation). This is useful when you need to shift a value and have the flags reflect the shifted result.

Normally, when using shift operations with ALU instructions like **CMPI** r2, #1, **LSL** #2, the status flags are updated according to the ALU operation result, not the shift. The **SHFT** instruction allows the status flags to be updated from the shift operation itself.

**Examples:**

```
1 ; Shift r1 left by 3 and update flags
2 SHFT r1, LSL #3                ; r1 = r1 << 3, flags = status of shift
3
4 ; Shift right and test result
5 SHFT r3, LSR #2                ; r3 = r3 >> 2
6 BRAN |== zero_result          ; Branch if shifted result is zero
7
8 ; Arithmetic shift with sign check
9 SHFT r6, ASR #4                ; r6 = r6 >> 4 (signed)
10 BRAN |NS negative_value      ; Branch if result is negative
```

**Notes:**

- Behind the scenes, **SHFT** uses the status override syntax (see Section 8.12) to force flag updates from the shift operation
- All shift types are supported (LSL, LSR, ASL, ASR, RTL, RTR)
- Both immediate and register-based shift counts are supported

(Note: Check assembler documentation for supported pseudo-instructions)

## Appendix A

# Opcode Map

### A.1 Complete Opcode Reference

This appendix provides a complete map of all 64 possible opcodes in the SIRCIS instruction set.

| Hex                             | Binary | Mnemonic | Description                         |
|---------------------------------|--------|----------|-------------------------------------|
| ALU Immediate (0x00–0x0F)       |        |          |                                     |
| 00                              | 000000 | ADDI     | Add Immediate                       |
| 01                              | 000001 | ADCI     | Add with Carry Immediate            |
| 02                              | 000010 | SUBI     | Subtract Immediate                  |
| 03                              | 000011 | SBCI     | Subtract with Carry Immediate       |
| 04                              | 000100 | ANDI     | AND Immediate                       |
| 05                              | 000101 | ORRI     | OR Immediate                        |
| 06                              | 000110 | XORI     | XOR Immediate                       |
| 07                              | 000111 | LOAD     | Load Immediate                      |
| 08                              | 001000 | –        | Undocumented                        |
| 09                              | 001001 | –        | Undocumented                        |
| 0A                              | 001010 | CMPI     | Compare Immediate                   |
| 0B                              | 001011 | –        | Undocumented                        |
| 0C                              | 001100 | TSAI     | Test AND Immediate                  |
| 0D                              | 001101 | –        | Undocumented                        |
| 0E                              | 001110 | TSXI     | Test XOR Immediate                  |
| 0F                              | 001111 | COPI     | Coprocessor Immediate               |
| Memory and Control (0x10–0x1F)  |        |          |                                     |
| 10                              | 010000 | STOR     | Store Indirect Immediate            |
| 11                              | 010001 | STOR     | Store Indirect Register             |
| 12                              | 010010 | STOR     | Store Pre-Decrement Immediate       |
| 13                              | 010011 | STOR     | Store Pre-Decrement Register        |
| 14                              | 010100 | LOAD     | Load Indirect Immediate             |
| 15                              | 010101 | LOAD     | Load Indirect Register              |
| 16                              | 010110 | LOAD     | Load Post-Increment Immediate       |
| 17                              | 010111 | LOAD     | Load Post-Increment Register        |
| 18                              | 011000 | LDEA     | Load Effective Address Immediate    |
| 19                              | 011001 | LDEA     | Load Effective Address Register     |
| 1A                              | 011010 | BRAN     | Branch Immediate                    |
| 1B                              | 011011 | BRAN     | Branch Register                     |
| 1C                              | 011100 | LJSR     | Long Jump to Subroutine Immediate   |
| 1D                              | 011101 | LJSR     | Long Jump to Subroutine Register    |
| 1E                              | 011110 | BRSR     | Branch to Subroutine Immediate      |
| 1F                              | 011111 | BRSR     | Branch to Subroutine Register       |
| ALU Short Immediate (0x20–0x2F) |        |          |                                     |
| 20                              | 100000 | ADDI     | Add Short Immediate                 |
| 21                              | 100001 | ADCI     | Add with Carry Short Immediate      |
| 22                              | 100010 | SUBI     | Subtract Short Immediate            |
| 23                              | 100011 | SBCI     | Subtract with Carry Short Immediate |
| 24                              | 100100 | ANDI     | AND Short Immediate                 |
| 25                              | 100101 | ORRI     | OR Short Immediate                  |
| 26                              | 100110 | XORI     | XOR Short Immediate                 |
| 27                              | 100111 | LOAD     | Load Short Immediate                |
| 28                              | 101000 | –        | Undocumented                        |
| 29                              | 101001 | –        | Undocumented                        |
| 2A                              | 101010 | CMPI     | Compare Short Immediate             |
| 2B                              | 101011 | –        | Undocumented                        |
| 2C                              | 101100 | TSAI     | Test AND Short Immediate            |
| 2D                              | 101101 | –        | Undocumented                        |
| 2E                              | 101110 | TSXI     | Test XOR Short Immediate            |

## A.2 Opcode Patterns

### A.2.1 Format Identification

- Bits 5–4 = 00: Immediate format (0x00–0x0F)
- Bits 5–4 = 01: Memory/Control operations (0x10–0x1F)
- Bits 5–4 = 10: Short Immediate with Shift (0x20–0x2F)
- Bits 5–4 = 11: Register format (0x30–0x3F)

### A.2.2 Save vs. Test

- Bit 3 = 0: Save result (0x00–0x07, 0x20–0x27, 0x30–0x37)
- Bit 3 = 1: Test only, discard result (0x08–0x0F, 0x28–0x2F, 0x38–0x3F)

### A.2.3 Memory Operations Decoding

For opcodes 0x10–0x1F, bits encode memory operation type:

- Bit 4 = 1: Always set for memory/control section
- Bits 3–2: Operation type
  - 00 = Store
  - 01 = Load
  - 10 = Load Effective Address
  - 11 = Load EA with Link (jump to subroutine)
- Bit 1: Determines if both registers in address pair are updated
- Bit 0: Immediate (0) or Register (1) offset

## A.3 Undocumented Instructions

The following opcodes are undocumented and should not be used in production code:

- 0x08, 0x09, 0x0B, 0x0D (Immediate format, test variants)
- 0x28, 0x29, 0x2B, 0x2D (Short Immediate format, test variants)
- 0x38, 0x39, 0x3B, 0x3D (Register format, test variants)

These opcodes may execute in hardware but their behavior is implementation-defined and may change between CPU revisions. See Appendix C for details.

## A.4 Quick Lookup Table

| Operation | Immediate | Short+Shift | Register |
|-----------|-----------|-------------|----------|
| ADD       | 0x00      | 0x20        | 0x30     |
| ADC       | 0x01      | 0x21        | 0x31     |
| SUB       | 0x02      | 0x22        | 0x32     |
| SBC       | 0x03      | 0x23        | 0x33     |
| AND       | 0x04      | 0x24        | 0x34     |
| OR        | 0x05      | 0x25        | 0x35     |
| XOR       | 0x06      | 0x26        | 0x36     |
| LOAD      | 0x07      | 0x27*       | 0x37     |
| CMP       | 0x0A      | 0x2A        | 0x3A     |
| TSA       | 0x0C      | 0x2C        | 0x3C     |
| TSX       | 0x0E      | 0x2E        | 0x3E     |
| COP       | 0x0F      | 0x2F        | 0x3F     |

Table A.2: Instruction Variant Quick Reference

\* Opcode 0x27 is an encoded CPU form but is not exposed by the assembler because its shift field has no useful effect on the loaded value.

## Appendix B

# Instruction Timing

### B.1 Overview

All SIRCIS instructions follow the six-stage execution phase model and complete in 6 clock cycles when no bus wait states are required. A bus operation may hold the current phase until it is acknowledged, adding wait cycles without adding execution phases. This fixed execution sequence greatly simplifies software development and hardware design.

### B.2 Six-Stage Execution Phases

For a detailed description of each phase, see Chapter 2.

1. **Instruction Fetch (High Word)** – 1 cycle
2. **Instruction Fetch (Low Word)** – 1 cycle
3. **Decode and Register Fetch** – 1 cycle
4. **Execute and Address Calculation** – 1 cycle
5. **Memory Access** – 1 cycle
6. **Write Back** – 1 cycle

## B.3 Timing Characteristics

| Instruction Type           | Cycles |
|----------------------------|--------|
| ALU (Register)             | 6      |
| ALU (Immediate)            | 6      |
| Load from Memory           | 6      |
| Store to Memory            | 6      |
| Branch (taken)             | 6      |
| Branch (not taken)         | 6      |
| Conditional (executed)     | 6      |
| Conditional (not executed) | 6      |
| NOOP                       | 6      |

Table B.1: Instruction Timing (All instructions)

The cycle counts in this appendix assume that every bus operation is acknowledged without a wait state.

## B.4 Conditional Execution

When a conditional instruction's condition is not met:

- The instruction still takes 6 cycles to complete when no bus wait states are required
- All execution phases run normally
- The write-back stage is disabled (no register or memory modification)
- Status flags are not updated

This means conditional instructions do not branch and avoid control-flow stalls, but failed conditions still consume execution time.

## B.5 Program Timing Examples

### B.5.1 Simple Loop

```

1 ; Process 10 elements (assuming no bus wait states)
2 ADDI r7, #0, #10 ; 6 cycles
3 loop:
4   LOAD r1, (#0, a)+ ; 6 cycles
5   ADDI r1, #1 ; 6 cycles
6   STOR (#-2, a), r1 ; 6 cycles
7   SUBI r7, #1 ; 6 cycles
8   BRAN |!= (#loop, p) ; 6 cycles

```

```

9 ; Total per iteration: 30 cycles
10 ; Total for 10 iterations: 6 + (10 * 30) = 306 cycles

```

### B.5.2 Function Call Overhead

```

1 ; Call function
2 LJSR (#0, a) ; 6 cycles
3
4 ; Function body executes...
5
6 ; Return
7 RETS ; 6 cycles
8 ; Total overhead: 12 cycles

```

### B.5.3 Conditional Block

```

1 CMPR r1, r2 ; 6 cycles
2 ADDR|HI r3, r4, r5 ; 6 cycles (exec if r1 > r2)
3 ADDR|LO r3, r6, r7 ; 6 cycles (exec if r1 <= r2)
4 ; Total: 18 cycles (one conditional executes, one doesn't)

```

## B.6 Performance Considerations

### B.6.1 No Pipeline Stalls

The SIRC-1 does not implement pipelining in the traditional sense where multiple instructions are in flight simultaneously. Instead:

- Each instruction completes fully before the next begins
- No data hazards or phase bubbles
- No branch prediction penalties
- Base execution time is predictable before bus wait states are considered

### B.6.2 Memory Access

- Memory access occupies one execution phase
- If BACK is not asserted, the CPU holds that phase until the bus operation is acknowledged
- Each wait state adds one clock cycle to the instruction

### B.6.3 Optimization Guidelines

#### 1. Use conditional execution for short sequences

```
1 ; Instead of:
2 CMPR r1, r2
3 BRAN|<= skip
4 ADDI r3, #1
5 skip:
6
7 ; Use:
8 CMPR r1, r2
9 ADDI|HI r3, #1 ; Saves branch overhead
```

#### 2. Combine operations with shifts

```
1 ; Instead of:
2 ADDR r1, r2, r2
3 ADDI r1, #0, LSL #1
4
5 ; Use:
6 ADDR r1, r2, r2, LSL #1 ; Multiply by 3 in one instruction
```

#### 3. Use post-increment/pre-decrement for iteration

```
1 ; Instead of:
2 LOAD r1, (#0, a)
3 ADDI a, #2
4
5 ; Use:
6 LOAD r1, (#0, a)+ ; Same result, fewer cycles
```

#### 4. Loop unrolling reduces overhead

```
1 ; Instead of processing one item per iteration,
2 ; process multiple items to reduce branch overhead
3 loop:
4     LOAD r1, (#0, a)+
5     LOAD r2, (#0, a)+
6     LOAD r3, (#0, a)+
7     LOAD r4, (#0, a)+
8     ; Process 4 items...
9     SUBI r7, #4
10    BRAN|HI loop
```

## B.7 Theoretical Performance

At a given clock frequency:

- **Instructions per second:**  $f_{clock}/6$
- **At 10 MHz:**  $\approx 1.67$  million instructions/second

- **At 25 MHz:**  $\approx$  4.17 million instructions/second

These are theoretical maximums assuming perfect code with no inefficiencies.

## B.8 Comparison to Other CPUs

| CPU            | Avg CPI | Notes                      |
|----------------|---------|----------------------------|
| SIRC-1         | 6.0     | Fixed, all instructions    |
| MOS 6502       | 2-7     | Variable by instruction    |
| Motorola 68000 | 4-158   | Highly variable            |
| ARM6           | 1.0     | Single-cycle with pipeline |
| MIPS R2000     | 1.0     | Single-cycle with pipeline |

Table B.2: Cycles Per Instruction Comparison

The SIRC-1's fixed 6-cycle execution makes it slower than pipelined RISC CPUs but more predictable. It's comparable to early microprocessors while being simpler to implement in hardware.



## Appendix C

# Undocumented Instructions

### C.1 Overview

The SIRCIS instruction set includes 12 undocumented instruction opcodes. These opcodes are valid and will execute in hardware, but their behaviour is not officially specified and may vary between CPU implementations or revisions.

**Warning:** Use of undocumented instructions is not recommended for production code.

### C.2 Undocumented Opcode List

| Opcode | Format    | Pattern        | Notes                  |
|--------|-----------|----------------|------------------------|
| 0x08   | Immediate | ADD-like + 0x8 | Test variant (no save) |
| 0x09   | Immediate | ADC-like + 0x8 | Test variant (no save) |
| 0x0B   | Immediate | SUB-like + 0x8 | Test variant (no save) |
| 0x0D   | Immediate | SBC-like + 0x8 | Test variant (no save) |
| 0x28   | Short Imm | ADD-like + 0x8 | Test variant (no save) |
| 0x29   | Short Imm | ADC-like + 0x8 | Test variant (no save) |
| 0x2B   | Short Imm | SUB-like + 0x8 | Test variant (no save) |
| 0x2D   | Short Imm | SBC-like + 0x8 | Test variant (no save) |
| 0x38   | Register  | ADD-like + 0x8 | Test variant (no save) |
| 0x39   | Register  | ADC-like + 0x8 | Test variant (no save) |
| 0x3B   | Register  | SUB-like + 0x8 | Test variant (no save) |
| 0x3D   | Register  | SBC-like + 0x8 | Test variant (no save) |

Table C.1: Undocumented Instruction Opcodes

## C.3 Likely Behavior

Based on the systematic opcode organization, these undocumented instructions likely behave as "test" variants of arithmetic instructions:

### C.3.1 0x08, 0x28, 0x38 (ADD Test)

Probably performs addition but discards the result, only updating flags. Similar to **CMP** but using ADD semantics instead of SUB.

```
1 ; Hypothetical behavior
2 result = operand1 + operand2 ; Calculate
3 ; result is discarded (not written to register)
4 ; Flags updated based on result
```

**Utility:** Limited. **CMP** already provides comparison functionality.

### C.3.2 0x09, 0x29, 0x39 (ADC Test)

Probably performs add-with-carry but discards the result, updating flags.

```
1 ; Hypothetical behavior
2 result = operand1 + operand2 + C
3 ; result is discarded
4 ; Flags updated
```

**Utility:** Possibly useful for testing carry propagation without modifying registers.

### C.3.3 0x0B, 0x2B, 0x3B (SUB Test)

Probably identical to **CMP** (which is already SUB without save).

**Utility:** None. Use **CMP** instead.

### C.3.4 0x0D, 0x2D, 0x3D (SBC Test)

Probably performs subtract-with-borrow but discards the result.

```
1 ; Hypothetical behavior
2 result = operand1 - operand2 - (1 - C)
3 ; result is discarded
4 ; Flags updated
```

**Utility:** Possibly useful for testing borrow propagation in multi-precision arithmetic.

## C.4 Implementation Notes

### C.4.1 Current Simulator

In the current SIRC-VM simulator (as of version 1.0):

- Undocumented instructions are recognized and decoded
- They execute following the "test variant" pattern (no write-back)

- Flag updates may or may not be implemented correctly
- Behavior may change in future versions

### C.4.2 Hardware Implementation

In a hardware implementation:

- The decoder treats bit 3 = 1 as "don't write back"
- ALU operations still execute normally
- Flags are still updated
- The write-back stage is simply disabled

This means the instructions *will* work, but their exact behavior depends on the hardware implementation details.

## C.5 Why Undocumented?

These instructions were left undocumented for several reasons:

1. **Limited utility:** Most have no practical use beyond documented instructions
2. **Incomplete semantics:** The designer hasn't fully defined their behavior
3. **Opcode space reservation:** May be used for future features
4. **Implementation flexibility:** Allows hardware optimizations

## C.6 Recommendations

### C.6.1 For Application Developers

**Do not use undocumented instructions in production code.**

- Behavior may change between CPU revisions
- Assemblers may reject them or emit warnings
- Code may not be portable
- Debugging tools may not recognize them

### C.6.2 For Compiler Writers

- Do not generate undocumented instructions
- Use only documented instructions for all operations
- If optimization requires new operations, request formal documentation

### C.6.3 For Hardware Implementers

- You may implement these opcodes following the logical pattern
- Document any implementation-specific behavior
- Consider reserving them for future standardization

### C.6.4 For Researchers/Hackers

If you must experiment with undocumented instructions:

- Test thoroughly on your specific hardware/simulator version
- Document your findings
- Isolate usage in test code, not production
- Be prepared for behavior changes

## C.7 Future Standardization

Some undocumented instructions may be formally specified in future SIRCIS revisions. Candidates for standardization include:

- **ADC Test (0x09/0x29/0x39):** For carry flag testing in multi-precision
- **SBC Test (0x0D/0x2D/0x3D):** For borrow flag testing in multi-precision

Others (SUB Test, which duplicates CMP) will likely remain undocumented or be repurposed.

## C.8 Opcode Space for Expansion

The SIRCIS instruction set has limited expansion space:

- 12 undocumented opcodes (could be formally specified)
- Reserved shift type (0x111) could enable additional shift modes
- Reserved address register encoding could enable more address modes
- Additional coprocessors could provide extended functionality

Any future expansion must maintain backward compatibility with existing code.

## Appendix D

# Example Programs

This appendix includes some sample programs to demonstrate how to use different features of the CPU.

### D.1 Byte Sieve

Listing D.1: Byte sieve example

```
1 ; https://rosettacode.org/wiki/Sieve\_of\_Eratosthenes
2
3 .EQU $MAX_PRIME #1000
4
5 ; Reserved space for 128x32 bit exception vectors
6 .ORG 0x0000
7 .DQ @main
8
9 .ORG 0x0200
10 :main
11
12 ; File I/O segment
13 LOAD    ah, #0x00F0
14 ; Array Start = 0x00F0_0000
15 LOAD    al, #0x0000
16
17 ; Stack segment (other end of mapped file)
18 LOAD    sh, #0x00F0
19 ; Array Start = 0x00F0_0000
20 LOAD    sl, #0xFFFF
21
22 LOAD    r7, #2
23 LOAD    r1, #2
24 LOAD    r2, #1
25
26 :1
27
28 STOR    (r1, a), r2
29 ADDI    r1, #2
30 CMPI    r1, $MAX_PRIME
31 BRAN | <= @1
```

```

32 LOAD    r1, #3
33 LOAD    r3, #1
34
35 :2
36
37 LOAD    r2, (r1, a)
38 CMPI    r2, #1
39 BRAN |== @4
40 ; THIS NUMBER IS A PRIME!
41 ; TODO: Log this somehow
42 STOR    -(s), r1
43 LOAD    r7, r1
44 LOAD    r2, r1
45
46 :3
47 STOR    (r2, a), r3
48 ADDR    r2, r1
49 CMPI    r2, $MAX_PRIME
50 BRAN |<= @3
51
52 :4
53 ADDI    r1, #2
54 CMPI    r1, $MAX_PRIME
55 BRAN |<= @2
56
57 :DONE
58
59 ; Halt CPU
60 COPI    r1, #0x14FF

```

## D.2 Faults

Listing D.2: Fault handling example

```

1 ;; Segments
2 .EQU $PROGRAM_SEGMENT          #0x0000
3 .EQU $SCRATCH_SEGMENT          #0x0001
4 .EQU $PROGRAM_SEGMENT_HIGH     #0x0002
5
6 ;; Devices
7 ; Debug
8 .EQU $DEBUG_DEVICE_SEGMENT      #0x000B
9 .EQU $DEBUG_DEVICE_BUS_ERROR    #0x0000
10 .EQU $DEBUG_DEVICE_EXCEPTION_L5 #0x0005
11 .EQU $DEBUG_DEVICE_BUS_PROTECTION_ERROR #0x0006
12
13
14 ;; Cpu Phases
15 .EQU $CPU_PHASE_INSTRUCTION_FETCH #0x0000
16 .EQU $CPU_PHASE_EFFECTIVE_ADDRESS #0x0003
17
18

```

```

19 ; Reserved space for 128x32 bit exception vectors
20 .ORG 0x0000
21 .DQ @init
22 .ORG 0x0002
23 .DQ @bus_fault_handler
24 .ORG 0x0004
25 .DQ @alignment_fault_handler
26 .ORG 0x0006
27 .DQ @segment_overflow_fault_handler
28 .ORG 0x0008
29 .DQ @invalid_opcode_fault_handler
30 .ORG 0x000A
31 .DQ @privilege_violation_fault_handler
32 .ORG 0x000C
33 .DQ @instruction_trace_fault_handler
34 .ORG 0x000E
35 .DQ @level_five_interrupt_conflict_handler
36 .ORG 0x0010
37 .DQ @double_fault_handler
38 .ORG 0x0012
39 .DQ @bus_protection_fault_handler
40
41 .ORG 0x0020
42 .DQ @level_five_interrupt_handler
43
44 .ORG 0x0200
45 :init
46 LOAD      r1, #1
47 LOAD      r2, #1
48 LOAD      r3, #1
49 LOAD      r4, #1
50 LOAD      r5, #1
51 LOAD      r6, #1
52 LOAD      r7, #1
53
54 ; Get the debug device to raise a fault on the bus
55 LOAD      ah, $DEBUG_DEVICE_SEGMENT
56 LOAD      al, $DEBUG_DEVICE_BUS_ERROR
57 STOR      (a), r1
58
59 ; Jumping to an odd address triggers an alignment fault
60 LOAD      pl, #0x0201
61
62 :after_odd_address
63
64 ; Loading/storing from an odd address should not trigger an alignment
   fault
65 LOAD      ah, $SCRATCH_SEGMENT
66 LOAD      al, #0x0
67 LOAD      r7, #0xCAFE
68 STOR      (#1, a), r7
69 LOAD      r7, (#1, a)
70
71

```

```

72 ; Calculating an effective address that overflows triggers a segment
    overflow fault
73 ; (but only if the SR is set up the right way)
74 LOAD    ah, $PROGRAM_SEGMENT
75 LOAD    al, #0xFFFE
76
77 ; This should not trigger fault
78 LOAD    r7, (#2, a)
79 ; Switch on trap on overflow bit
80 ORRI    sr, #0b0100_0000_0000_0000
81 ; This _should_ trigger a fault
82 LOAD    r7, (#2, a)
83 ; Switch off trap on overflow bit
84 ANDI    sr, #0b1011_1111_1111_1111
85
86 ; Segment overflow again, but this time the PC overflow causes it
87
88 LOAD    ah, $PROGRAM_SEGMENT_HIGH
89 LOAD    al, #0xFFFE
90
91 ; This should not trigger fault
92 LJMP    a
93
94 :return_from_testing_pc_overflow_no_fault
95
96 ; Switch on trap on overflow bit
97 ORRI    sr, #0b0100_0000_0000_0000
98
99 LOAD    ah, $PROGRAM_SEGMENT_HIGH
100 LOAD    al, #0xFFFE
101
102 ; This _should_ trigger a fault
103 LJMP    a
104
105 :return_from_testing_pc_overflow_with_fault
106
107 ; Switch off trap on overflow bit
108 ANDI    sr, #0b1011_1111_1111_1111
109
110 ; Instruction trace fault test
111 ; Set T bit - the next instruction will be traced and the handler will
    fire
112 ORRI    sr, #0b1000_0000_0000_0000
113 ; This instruction is traced
114 NOOP
115 ; T bit was cleared by the trace handler, so no further instructions are
    traced
116
117 ; Bus protection fault test
118 ; Get the debug device to raise a protection error on the bus
119 LOAD    ah, $DEBUG_DEVICE_SEGMENT
120 LOAD    al, $DEBUG_DEVICE_BUS_PROTECTION_ERROR
121 LOAD    r7, #0x1
122 STOR    (a), r7

```

```

123
124 ; There is no coprocessor at ID 4, so should trigger an invalid opcode
    fault
125 COPI    r1, #0x4000
126
127 ; Switch to non privileged mode
128 ORRI    sr, #0b0000_0001_0000_0000
129 ; Try to escape the current segment
130 LOAD    ph, #0xFEFE
131
132
133 ; Get the debug device to raise an interrupt (this one should be fine)
134 LOAD    r7, #0x1
135 LOAD    ah, $DEBUG_DEVICE_SEGMENT
136 LOAD    al, $DEBUG_DEVICE_EXCEPTION_L5
137 STOR    (a), r7
138
139 ; Halt CPU
140 COPI    r1, #0b0001_0100_1111_1111
141
142 .ORG 0x0300
143
144 :bus_fault_handler
145 ADDI    r1, #1
146 RETE
147
148 :bus_protection_fault_handler
149 ADDI    r5, #1
150 RETE
151
152 :alignment_fault_handler
153 ADDI    r2, #1
154
155 ; Fix up the return address because the original PC causes an alignment
    fault
156 ; Returning without fixing up the return address would cause an endless
    loop
157
158 ; Transfer current ELR into 'a' register
159 ETFR a, #6
160 ; Correct the lower word
161 LOAD    al, @after_odd_address
162 ; Transfer corrected address back to ELR
163 ETTR #6, a
164
165 RETE
166
167 :segment_overflow_fault_handler
168 ; Check the phase
169
170 ; Transfer exception metadata to r7
171 ETFR    r7, #7
172 ; Mask off the phase section of the status register
173 ANDI    r7, #0x7

```

```
174
175 CMPI    r7, $CPU_PHASE_INSTRUCTION_FETCH
176 BRAN |== @segment_overflow_fault_handler_pc
177 ; Mask off the phase section of the status register
178 ANDI    r7, #0x7
179
180 CMPI    r7, $CPU_PHASE_EFFECTIVE_ADDRESS
181 BRAN |== @segment_overflow_fault_handler_address
182
183 BRAN @segment_overflow_fault_handler_end
184
185 :segment_overflow_fault_handler_pc
186
187 ADDI    r3, #0x0F00
188
189 ; Fix up the return address because it isn't valid
190 ; Load address after fault
191 LOAD    ah, $PROGRAM_SEGMENT
192 LOAD    al, @return_from_testing_pc_overflow_with_fault
193 ; Transfer corrected address back to ELR
194 ETTR    #6, a
195
196 BRAN @segment_overflow_fault_handler_end
197
198 :segment_overflow_fault_handler_address
199
200 ADDI    r3, #0x000F
201
202 :segment_overflow_fault_handler_end
203
204 RETE
205
206 :invalid_opcode_fault_handler
207 ; A double fault will blat the EU link registers so we need to save them
   before we trigger it
208 ETFR #6
209
210 ADDI    r4, #1
211 ; Trigger another invalid opcode fault to test double fault handling
212 ; This will cause a fault while already handling a fault
213 COPI    r1, #0xF000
214
215 ; Restore link register
216 ETTR #6
217
218 RETE
219
220 :privilege_violation_fault_handler
221 ADDI    r5, #1
222
223 ; Make sure that the CPU is back in privileged mode again after handling
   this fault
224
225 ; Transfer current ELR into 'r7' register
```

```

226 ETFR      r7, #6
227 ; Correct the lower word
228 ANDI      r7, #0xFEFF
229 ; Transfer corrected SR back to ELR
230 ETTR      #6, r7
231
232 RETE
233
234 :double_fault_handler
235 ; A double fault occurred - this is typically a last-chance handler
236 ; In this example, we'll increment r4 to show we were here
237 ; and then return (though in production you'd probably reset)
238 ADDI      r4, #1
239 RETE
240
241 :level_five_interrupt_conflict_handler
242 ADDI      r6, #0x1
243 RETE
244
245 :level_five_interrupt_handler
246 ADDI      r6, #0x1000
247
248 ; Trigger another L5 interrupt while it is already being serviced
249 ; which will trigger a fault
250 LOAD      r7, #0x1
251 LOAD      ah, $DEBUG_DEVICE_SEGMENT
252 LOAD      al, $DEBUG_DEVICE_EXCEPTION_L5
253 STOR      (a), r7
254
255 RETE
256
257 :instruction_trace_fault_handler
258 ADDI      r6, #0x100
259 ; Clear T bit from the saved SR so RETE does not re-enable tracing
260 ETFR      r7, #6
261 ANDI      r7, #0x7FFF
262 ETTR      #6, r7
263 RETE
264
265 ; Trampoline for faults-high to come back
266 .ORG 0xFFFF0
267 LOAD      pl, @return_from_testing_pc_overflow_no_fault

```

Listing D.3: High-address fault handling example

```

1 ;; Segments
2 .EQU $PROGRAM_SEGMENT          #0x0000
3 .EQU $SCRATCH_SEGMENT          #0x0001
4 .EQU $PROGRAM_SEGMENT_HIGH     #0x0002
5
6 ; Section to test program counter overflow
7 ; without trying to execute the vector table
8 ; that lives at 0x0000 in the main segment
9

```

```
10 ; Note: This is not linked together with the main program
11 ; This is compiled as a separate program and mapped as a separate
    segment
12
13 .ORG 0x0000
14
15 LOAD    ah, $PROGRAM_SEGMENT
16 LOAD    al, #0xFFFF0
17 LJMP    a
18
19
20 ; TODO: Work out how to avoid padding this out
21 ; I had to pad it out because even though I said this segment has a
    length of 0xFFFF
22 ; I think the way files are mapped means that length is ignored
23 .ORG    0xFFFE
24 LOAD    r1, r1
```

## D.3 Hardware Exception

Listing D.4: Hardware exception example

```
1 ; Reserved space for 128x32 bit exception vectors
2
3
4 .EQU $FALSE                #0x0
5 .EQU $TRUE                 #0x1
6
7 ; When this character is typed, the program should exit
8 .EQU $EXPECTED_CHAR        #0x61
9
10 ;; Segments
11 .EQU $PROGRAM_SEGMENT      #0x0000
12 .EQU $SCRATCH_SEGMENT      #0x0001
13
14 ;; Devices
15 ; Serial
16 .EQU $SERIAL_DEVICE_SEGMENT #0x000A
17 .EQU $SERIAL_DEVICE_BAUD    #0x0000
18 .EQU $SERIAL_DEVICE_RECV_ENABLED #0x0001
19 .EQU $SERIAL_DEVICE_RECV_PENDING #0x0002
20 .EQU $SERIAL_DEVICE_RECV_DATA  #0x0003
21 .EQU $SERIAL_DEVICE_SEND_ENABLED #0x0004
22 .EQU $SERIAL_DEVICE_SEND_PENDING #0x0005
23 .EQU $SERIAL_DEVICE_SEND_DATA  #0x0006
24
25 ;; Scratch Variables
26 .EQU $MESSAGE_SEND_BASE      #0x0000
27 .EQU $MESSAGE_SEND_POINTER   #0x0001
28
29
30 ;; Exception Vectors
```

```

31 ; First vector is the reset vector
32 .ORG 0x0000
33 .DQ @start
34
35 .ORG 0x0080
36 .DQ @exception_handler_p3
37
38 ;;;;;; MAIN CODE SECTION
39
40 .ORG 0x0200
41 :start
42
43 ; Enable all hardware interrupts (set bits 9-13 of SR)
44 ORRI sr, #0b0001_1110_0000_0000
45
46 ; Setup routines
47 BRSR @setup_serial
48 BRSR @print_help
49 ; Make sure this is after the initial help message print (see the
    subroutine for more info)
50 BRSR @enable_serial_recv
51
52 :wait_for_char
53 ; Wait for exception (will spin until interrupted)
54 WAIT
55
56 ; Pending byte should be in r7 after exception handler runs
57 CMPI    r7, $EXPECTED_CHAR
58 BRAN|== @finish
59
60 BRAN @wait_for_char
61
62 :setup_serial
63
64 LOAD    r1, #9600
65 LOAD    ah, $SERIAL_DEVICE_SEGMENT
66 LOAD    al, $SERIAL_DEVICE_BAUD
67 STOR    (a), r1
68
69 RETS
70
71 :enable_serial_recv
72
73 ; When piping data to stdin, an EOF character gets sent which closes
    stdin
74 ; This means that we can't trigger any more interrupts manually after
    the data is piped
75 ; (stdin is currently the only way to externally trigger an interrupt)
76 ; If the data is coming in at the same type we are sending data out,
77 ; it ends up with the program gets stuck waiting for an interrupt and
    never finishes
78 ; TODO: Actually that shouldn't be a problem - better investigate this
    further (

```

```
79 ;   maybe the exception handler routines are conflicting with each other
80   )
81 LOAD    r1, #0x1
82 LOAD    al, $SERIAL_DEVICE_RECV_ENABLED
83 STOR    (a), r1
84
85 RETS
86
87 :print_help
88
89 LOAD    r1, @help_message
90 BRAN    @print
91
92 :print_exit
93
94 LOAD    r1, @exit_message
95 BRAN    @print
96
97 :print
98
99 LOAD    ah, $SCRATCH_SEGMENT
100 LOAD    al, $MESSAGE_SEND_BASE
101 STOR    (a), r1
102
103 ; Align pointer
104 LOAD    r1, #1
105
106 LOAD    ah, $SCRATCH_SEGMENT
107 LOAD    al, $MESSAGE_SEND_POINTER
108 STOR    (a), r1
109
110 LOAD    r1, $TRUE
111 LOAD    ah, $SERIAL_DEVICE_SEGMENT
112 LOAD    al, $SERIAL_DEVICE_SEND_ENABLED
113 STOR    (a), r1
114
115 :wait_for_print_finish
116
117 LOAD    ah, $SERIAL_DEVICE_SEGMENT
118 LOAD    al, $SERIAL_DEVICE_SEND_ENABLED
119 LOAD    r1, (a)
120
121 CMPI    r1, $TRUE
122 BRAN    |== @wait_for_print_finish
123
124 RETS
125
126 :finish
127
128 BRSR    @print_exit
129
130 ; Halt CPU
131 COPI    r1, #0b0001_0100_1111_1111
```

```

132
133 ;;;;;;;;; EXCEPTION HANDLERS
134
135 .ORG 0x0400
136 :exception_handler_p3
137
138 ; Save the link register
139 LDEA s, (l)
140
141 BRSR @read_pending_byte
142 BRSR @write_pending_byte
143
144 ; Restore the link register
145 LDEA l, (s)
146
147 RETE
148
149 :read_pending_byte
150 ; Check if there is something we need to read
151 LOAD    ah, $SERIAL_DEVICE_SEGMENT
152 LOAD    al, $SERIAL_DEVICE_RECV_PENDING
153 LOAD    r1, (a)
154
155 ; Return early if not
156 CMPI    r1, $FALSE
157 RETS |==
158
159 ; Read pending byte
160 LOAD    al, $SERIAL_DEVICE_RECV_DATA
161 LOAD    r7, (a)
162
163 LOAD    al, $SERIAL_DEVICE_RECV_PENDING
164 LOAD    r1, $FALSE
165 STOR    (a), r1
166 RETS
167
168 :write_pending_byte
169 ; Check if the device is ready to write
170 LOAD    ah, $SERIAL_DEVICE_SEGMENT
171 LOAD    al, $SERIAL_DEVICE_SEND_PENDING
172 LOAD    r1, (a)
173
174 ; Return early if not (there is already a send pending)
175 CMPI    r1, $TRUE
176 RETS |==
177
178 LOAD    ah, $SCRATCH_SEGMENT
179 LOAD    al, $MESSAGE_SEND_POINTER
180 LOAD    r2, (a)
181 LOAD    al, $MESSAGE_SEND_BASE
182 LOAD    al, (a)
183
184 LOAD    ah, $PROGRAM_SEGMENT
185 LOAD    r3, (r2, a)

```

```

186
187 ; If the message buffer (zero terminated) has been sent. Stop sending
188 CMPI    r3, #0
189 BRAN |== @stop_send
190
191 LOAD    ah, $SERIAL_DEVICE_SEGMENT
192 LOAD    al, $SERIAL_DEVICE_SEND_DATA
193 STOR    (a), r3
194
195 ; Increment by two because of lack of packing (a word actually is padded
    to a DW)
196 ADDI    r2, #2
197 LOAD    ah, $SCRATCH_SEGMENT
198 LOAD    al, $MESSAGE_SEND_POINTER
199 STOR    (a), r2
200
201 LOAD    ah, $SERIAL_DEVICE_SEGMENT
202 LOAD    al, $SERIAL_DEVICE_SEND_PENDING
203 LOAD    r1, $TRUE
204 STOR    (a), r1
205
206 RETS
207
208 :stop_send
209 LOAD    ah, $SERIAL_DEVICE_SEGMENT
210 LOAD    al, $SERIAL_DEVICE_SEND_ENABLED
211 LOAD    r1, $FALSE
212 STOR    (a), r1
213
214 RETS

```

Listing D.5: Hardware exception data

```

1  ;;;;;; DATA
2
3  ; TODO: A string data type that does this for us
4  :help_message
5  .DW #84
6  .DW #121
7  .DW #112
8  .DW #101
9  .DW #32
10 .DW #39
11 .DW #97
12 .DW #39
13 .DW #32
14 .DW #116
15 .DW #111
16 .DW #32
17 .DW #101
18 .DW #120
19 .DW #105
20 .DW #116
21 .DW #10

```

```

22 .DW #0
23
24 :exit_message
25 .DW #71
26 .DW #111
27 .DW #111
28 .DW #100
29 .DW #98
30 .DW #121
31 .DW #101
32 .DW #33
33 .DW #10
34 .DW #0

```

## D.4 Software Exception

Listing D.6: Software exception example

```

1 ; Reserved space for 128x32 bit exception vectors
2
3 ; First vector is the reset vector
4 .ORG 0x0000
5 .DQ @start
6
7 ; 64th vector (*2 words per vector)
8 .ORG 0x0080
9 .DQ @exception_handler
10 .DQ @exception_handler_that_should_be_ignored
11
12 .ORG 0x0200
13 :start
14
15 ; Reset register
16 LOAD    r1, #1
17 ; Trigger Software Exception
18 EXCP    #0x40
19
20 ; r1 should be 0xFF here after exception handler runs
21
22 ; Halt CPU
23 COPI    r1, #0x14FF
24
25 .ORG 0x0300
26 :exception_handler
27
28 ; Load a value to detect if this code runs
29 ; Final value of r1 should be this value
30 LOAD    r1, #0xFF
31 ; Try and trigger nested software exception (should be ignored)
32 EXCP    #0x41
33 ; Return from exception
34 RETE

```

```
35
36 .ORG 0x0500
37 :exception_handler_that_should_be_ignored
38
39 ; Load a value to detect if this code runs
40 ; Final value of r1 should _not_ be BB because this vector should never
   be jumped to
41 LOAD    r1, #0xBB
42 ; Return from exception
43 RETE
```

## D.5 Store Load

Listing D.7: Store/load example

```
1 ; Reserved space for 128x32 bit exception vectors
2
3 .ORG 0x0000
4 .DQ @init
5
6 .ORG 0x0200
7
8 :init
9 LOAD    r1, #0xCAFE
10 LOAD    r3, #0xBEEF
11 LOAD    r5, #0x000A
12
13 LOAD    ah, #0x0008
14 LOAD    al, #0xAAF0
15
16 STOR    (#0x000F, a), r1
17 LOAD    r2, (#0x000F, a)
18 STOR    (r5, a), r3
19 LOAD    r4, (r5, a)
20
21
22 ; Lets test jumping!
23
24 ; TODO: Add some tests that:
25 ; LJMP to another segment
26 ; LJMP with offset
27 ; LJMP with shifted offset
28 ; Same three things but with LJSR
29 ; Make a maze of jumps so if the jump goes to the wrong place it will
   land in the forbidden zone where every instruction jumps to the fail
   label
30
31 ; r7 will store the test result
32 LOAD    r7, #0
33 ; Use the stack address register pair to store fail address
34 LOAD    sh, #0x0
35 LOAD    sl, @fail
```

```

36 ; Address register pair used as part of test
37 LOAD ah, #0x0
38 LOAD al, #0x0
39
40 BRAN #12 ; Step 1
41 LJMP s ; FAIL!
42 LJMP s ; FAIL!
43 BRAN #10 ; Step 3
44 LJMP s ; FAIL!
45 LJMP s ; FAIL!
46 BRAN #-6 ; Step 2
47 LJMP s ; FAIL!
48 BRAN @ljmp_register_offset
49
50
51 :ljmp_register_offset
52 LOAD al, @ljmp_register_offset_anchor
53 LOAD r1, #-6
54 LOAD r2, #4
55 BRAN @ljmp_register_offset_anchor
56 LJMP s ; FAIL!
57 LJMP s ; FAIL!
58 LJMP s ; FAIL!
59 LJMP a, r2
60 LJMP s ; FAIL!
61 LJMP s ; FAIL!
62 :ljmp_register_offset_anchor
63 LJMP a, r1
64 LJMP s ; FAIL!
65 BRAN @ljmp_immediate_offset
66 LJMP s ; FAIL!
67 LJMP s ; FAIL!
68 LJMP s ; FAIL!
69 LJMP s ; FAIL!
70
71
72 :ljmp_immediate_offset
73 LOAD al, @ljmp_immediate_offset_anchor
74 BRAN @ljmp_immediate_offset_anchor
75 LJMP s ; FAIL!
76 LJMP s ; FAIL!
77 LJMP s ; FAIL!
78 LJMP a, #4
79 LJMP s ; FAIL!
80 LJMP s ; FAIL!
81 :ljmp_immediate_offset_anchor
82 LJMP a, #-6
83 LJMP s ; FAIL!
84 BRAN @ljsr_register_offset
85 LJMP s ; FAIL!
86 LJMP s ; FAIL!
87 LJMP s ; FAIL!
88 LJMP s ; FAIL!
89

```

```
90
91 :ljsr_register_offset
92 LOAD al, @ljsr_register_offset_anchor
93 LOAD r1, #-6
94 LOAD r2, #4
95 BRAN @ljsr_register_offset_anchor
96 LJSR s ; FAIL!
97 LJSR s ; FAIL!
98 LJSR s ; FAIL!
99 LJSR a, r2
100 LJSR s ; FAIL!
101 LJSR s ; FAIL!
102 :ljsr_register_offset_anchor
103 LJSR a, r1
104 LJSR s ; FAIL!
105 BRAN @ljsr_immediate_offset
106 LJSR s ; FAIL!
107 LJSR s ; FAIL!
108 LJSR s ; FAIL!
109 LJSR s ; FAIL!
110
111
112 :ljsr_immediate_offset
113 LOAD al, @ljsr_immediate_offset_anchor
114 BRAN @ljsr_immediate_offset_anchor
115 LJSR s ; FAIL!
116 LJSR s ; FAIL!
117 LJSR s ; FAIL!
118 LJSR a, #4
119 LJSR s ; FAIL!
120 LJSR s ; FAIL!
121 :ljsr_immediate_offset_anchor
122 LJSR a, #-6
123 LJSR s ; FAIL!
124 BRAN @finish
125 LJSR s ; FAIL!
126 LJSR s ; FAIL!
127 LJSR s ; FAIL!
128 LJSR s ; FAIL!
129
130
131 :finish
132 LOAD r7, #0x0FAB
133
134 LOAD sh, #0x0
135 LOAD sl, @pass
136 LJMP s
137
138 .ORG 0x0400
139
140 :fail
141 LOAD r7, #0xFA11
142
143 :pass
```

```
144  
145 ; Halt CPU  
146 COPI    r1, #0x14FF
```

